

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ  
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 11:55:54  
Файлов исходного кода: 82  
Суммарный объём кода: 1.17 MB  
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

# Билеты (учебный контент)

- 23. src/data/tickets/ticket1\_5.ts
- 24. src/data/tickets/ticket14\_20.ts
- 25. src/data/tickets/ticket21\_24.ts
- 26. src/data/tickets/ticket6\_13.ts

# Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

## Изменённые файлы

```text

```
A    .github/workflows/deploy-pages.yml
M    .github/workflows/rebuild-pdf.yml
M    index.html
M    public/export/full_code_all_pages.pdf
M    public/export/full_code_all_pages.txt
A    public/favicon.svg
M    src/App.tsx
M    src/components/Navbar.tsx
A    src/components/PythonCompiler.tsx
M    vite.config.ts
```

```

## `github/workflows/deploy-pages.yml`

### Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

```
deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
+env:
+  CI: false
```

## Универсальное пособие • полный исходный код

```
-----
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

## `github/workflows/rebuild-pdf.yml`

### Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
#   исходный код  →  full_code_all_pages.txt  (script)
#               |
#               |→  page-0001.png ... page-NNNN.png  (
#               |
#               |→  full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

- ```
        if [ -f "$policy" ]; then
            sudo sed -i 's/rights="none" pattern="\{P
        fi
    done
    convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density  
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| PNG-страниц | \$(ls tmp/export-pages.  
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
          - node-version: "22"
          + node-version: "24"
            cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....

## `index.html`
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
```

```
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
```

```
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```

```
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
```

```
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
```

```
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
```

```
import { chapters } from './data/content';
```

```
import { Navbar } from './components/Navbar';
```

```
import { Sidebar } from './components/Sidebar';
```

```
import { MobileToc } from './components/MobileToc';
```

```
import { ChapterView } from './components/ChapterView';
```

```
import { ChapterNav } from './components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
  <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

  {activeTab === "guide" ? (
    <>
      <MobileToc
        selectedChapterId={activeChapterId}
        setSelectedChapterId={goTo}
        currentTitle={activeChapter.title}
        index={index}
        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
        {/* Сайдбар только на десктопе – на мобильном скрывается */}
        <div className="hidden lg:block lg:w-80 shrink-0" >
          <Sidebar selectedChapterId={activeChapterId} />
        </div>

        <main id="main-content" className="flex-1 m-0 p-4" >
          {/* Шапка главы с быстрыми переходами */}
          <div className="flex items-center justify-between" >
            <div className="min-w-0" >
              <div className="text-[10px] uppercase" >
                {activeChapter.category || "Универсальное пособие"}
              </div>
              <h2 className="text-base sm:text-xl font-medium" >
                {activeChapter.title}
              </h2>
            </div>
            <ChapterNav prev={prev} next={next} index={index} />
          </div>

          <div className="h-1 w-full bg-slate-800 rounded" >
            <div
              className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
              style={{ width: `${progress}%` }}
            />

```

### Patch относительно `main`

```
```diff
```

```
diff --git a/src/App.tsx b/src/App.tsx
```

index 2091f33..4583756 100644

```
--- a/src/App.tsx
```

```
+++ b/src/App.tsx
```

```
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
```

```
import { ChapterNav } from "../components/ChapterNav";
```

```
import { SimulatorModal } from "../components/hints/Sim
```

```
import { SimulatorHub } from "../components/SimulatorHub";
```

```
+import { PythonCompiler } from "../components/PythonCom
```

```
export default function App() {
```

```
const [activeTab, setActiveTab] = useState<"guide" |
```

```
+   const [activeTab, setActiveTab] = useState<"guide" |
```

```
const [activeChapterId, setActiveChapterId] = useSta
```

```
const [modalVizId, setModalVizId] = useState<string>
```

```
@@ -95,7 +96,7 @@ export default function App() {
```

&lt;/main&gt;

</p>

$$- \quad ) : ($$

```
+      ) : activeTab === "simulator" ? (
```

```
<main className="px-4 sm:px-6 lg:px-10 py-6 lg
```

&lt;SimulatorHub

```
onOpen={setModalVizId}
```

```
@@ -105,6 +106,8 @@ export default function App() {
```

} }

13

&lt;/main&gt;

 $+ \quad ) : ($ 

```
+      <PythonCompiler />
```

11

```
<SimulatorModal vizId={modalVizId} onClose={() =>
```

```
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-slate-800">
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-extrabold">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>

  <div className="flex items-center w-full lg:w-auto" >
    >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <BookOpen className="w-4 h-4" /> Учебное пособие
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <button
      id="tab-compiler-btn"
      onClick={() => setActiveTab('compiler')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'compiler'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Code className="w-4 h-4" /> Компилятор
    </button>
  </div>
</div>
```

```
-----
        className="flex items-center justify-center
        er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
    >
        {busy ? <Loader2 className="w-4 h-4 animate
    </button>
    </div>
  </div>
</header>
);
};
},

```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >
      <Cpu className="w-4 h-4" /> Тренажёры

```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
      module as PyodideModule,
    }).then((module) => module.loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}
```

```

    }
  }, [code, stdin, running, runtimeReady]);

const reset = () => {
  setCode(STARTER_CODE);
  setStdin("");
  setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
  <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
    <div className="max-w-6xl mx-auto">
      <div className="flex flex-col sm:flex-row sm:items-center">
        <div>
          <div className="inline-flex items-center gap-2">
            <Terminal className="w-4 h-4" /> Боковая панель
          </div>
          <h2 className="text-2xl sm:text-3xl font-extrabold">
            <p className="text-slate-400 mt-2 max-w-2xl">
              Пишите небольшой код и запускайте его прямо в браузере,
              который переводит CPython в WebAssembly.
            </p>
          </div>
          <a
            href="https://pyodide.org/"
            target="_blank"
            rel="noreferrer"
            className="inline-flex items-center gap-1.5"
          >
            Как это работает <ExternalLink className="w-4 h-4" />
          </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
          <section className="rounded-2xl border border-slate-200 p-4">
            <div className="flex flex-wrap items-center">
              <div className="flex items-center gap-2">
                <span className="w-2.5 h-2.5 rounded-full bg-slate-200">

```

```

        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border"
        <label className="block px-4 pt-3 text-slate-500"
            Ввод для input() • по одной строке на клавишу
        </label>
        <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border"
            style={{border: '1px solid #333'}}
        />
    </div>
</section>

<section className="rounded-2xl border border-slate-800 border"
    <div className="flex items-center justify-between"
        <div className="flex items-center gap-2 text-slate-500"
            <Terminal className="w-4 h-4 text-emerald-500"
            />
        <span className={`inline-flex items-center gap-2`}
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500"
            {runtimeReady ? "готов" : "не загружен"}
            />
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4"
        x] leading-6 text-slate-300">
        {output}
    </pre>
    <div className="border-t border-slate-800 border"
    </div>
</section>
</div>

```

```

+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
+  const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+       <div className="max-w-6xl mx-auto">
+         <div className="flex flex-col sm:flex-row sm:items-center">
+           <div>
+             <div className="inline-flex items-center gap-2">
+               <Terminal className="w-4 h-4" /> Боковая панель
+             </div>
+             <h2 className="text-2xl sm:text-3xl font-extrabold">
+               <p className="text-slate-400 mt-2 max-w-2xl">
+                 Пишите небольшой код и запускайте его прямо в браузере,
+                 который переводит CPython в WebAssembly.
+               </p>
+             </div>
+             <a
+               href="https://pyodide.org/"
+               target="_blank"
+               rel="noreferrer"
+               className="inline-flex items-center gap-1.5"
+             >
+               Как это работает <ExternalLink className="text-slate-400" />
+             </a>
+           </div>
+
+           <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+             <section className="rounded-2xl border border-slate-200 p-4">
+               <div className="flex flex-wrap items-center gap-2">
+                 <div className="flex items-center gap-2">
+                   <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
+                     <span className="text-sm font-bold text-slate-400">
+                       <span className="text-[11px] text-slate-400">
+                         </div>
+                   <div className="flex items-center gap-2">
+                     <button
+                       type="button"
+                       onClick={reset}

```

```

+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-transparent border-slate-800 p-2"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4">
+         <div className="flex items-center justify-between">
+             <div className="flex items-center gap-2">
+                 <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+             </div>
+             <span className={`inline-flex items-center gap-2`}
+                 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+                 {runtimeReady ? "готов" : "не загружен"}}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-2">
+         </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+         <Info className="w-4 h-4 shrink-0 text-indigo-500 font-mono" />
+         <span>Ctrl или Cmd + Enter запускает код. L</span>
+     </div>
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+         <Info className="w-4 h-4 shrink-0 text-amber-500 font-mono" />

```

```
    },
    server: {
      host: "0.0.0.0",
      port: 5173,
      strictPort: true,
      // предпросмотр может открываться через внешний про
      allowedHosts: true,
    },
    preview: {
      host: "0.0.0.0",
      port: 4173,
      strictPort: true,
      allowedHosts: true,
    },
  },
});
\\
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
  },
}
```

---

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
  - "Душные" детали (код, формулы, сложные доказательства)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

## Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

# algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в о

| Команда              | Шаг       | Результат           |
|----------------------|-----------|---------------------|
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |

## Универсальное пособие • полный исходный код

-----  
известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).  
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`  
\* **Реестр интерактивов** — `src/components/vizRegistry`  
и для панели под главой, и для модальки из подсказки.

### ## Структура

...

|                    |                                           |
|--------------------|-------------------------------------------|
| src/               |                                           |
| App.tsx            | — оболочка, привязка глав к визуализации  |
| components/        | — интерактивные симуляторы (Действия)     |
| data/              | — контент пособия (главы/билеты)          |
| scripts/export/    | — пайплайн код → txt → png → pdf          |
| public/export/     | — сгенерированные TXT и PDF (обновляются) |
| .github/workflows/ | — автоматизация                           |

...

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте следующие теги и классы.

#### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и классы `Section` и `SectionTitle`.
- \* Подзаголовки: Используют тег `

### `.

#### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика: Используют тег `

` и классы `Text` и `TextBlock`.

#### ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

---

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

|               |                                |                             |
|---------------|--------------------------------|-----------------------------|
| Билет         | Тема                           | Статус                      |
| ---           | ---                            | ---                         |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. К                |
|               | ит вхолостую.                  |                             |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной глав               |
|               | огии**.                        |                             |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть бе |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет;         |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- \* `stack-dfs` → `StackViz.tsx` – **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` – **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` – **\*\*Динамическое программирование\*\***

---

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

|                                        |              |             |
|----------------------------------------|--------------|-------------|
| Глава                                  | id           | Комментарий |
| ---                                    | ---          | ---         |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e |             |

## Универсальное пособие • полный исходный код

|    |                                      |   |   |   |
|----|--------------------------------------|---|---|---|
| 17 | 17. Графы. Алгоритм Джонсона         | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП         | 4 | - | - |
| 22 | 22. Z-функция                        | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик             | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы       | - | - | - |
| -  | Алгоритмы на карте                   | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                   | - | - | - |
| -  | Мосты в графах: код + визуализация   | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)  
это 5 готовых компонентов, которые сейчас просто не...
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш...
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум inli...
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

-----

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

-----

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

---

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate-50">
```

```
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
```

```
      {activeTab === "guide" ? (
```

```
        <>
```

```
          <MobileToc
```

```
            selectedChapterId={activeChapterId}
```

```
            setSelectedChapterId={goTo}
```

```
            currentTitle={activeChapter.title}
```

```
            index={index}
```

```
            total={chapters.length}
```

```
          />
```

```
        <div className="flex flex-col lg:flex-row max-w-7xl">
```

```
          {/* Сайдбар только на десктопе – на мобильном скрывается */}
```

```
          <div className="hidden lg:block lg:w-80 shrink-0">
```

```
            <Sidebar selectedChapterId={activeChapterId}>
```

```
          </div>
```

```
          <main id="main-content" className="flex-1 min-w-0">
```

```
            {/* Шапка главы с быстрыми переходами */}
```

```
            <div className="flex items-center justify-between">
```

```
              <div className="min-w-0">
```

```
                <div className="text-[10px] uppercase">
```

```
                  {activeChapter.category || "Универсальное пособие"}>
```

```
                </div>
```

```
                <h2 className="text-base sm:text-xl font-medium">
```

```
                  {activeChapter.title}>
```

```
                </h2>
```

```
              </div>
```

```
              <ChapterNav prev={prev} next={next} index={index}>
```

```
            </div>
```

```
          <div className="h-1 w-full bg-slate-800 rounded">
```

```
            <div
```

---

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}])(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на них
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
      perTerm.set(id, usedTerm + 1);
      perSurface.set(surface, usedSurface + 1);
    }

    if (m.index > last) frag.appendChild(document.createElement("span"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("span"));
  textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
  const run = () => {
    const el = ref.current;
    if (!el) return;
    try {
      annotateTerms(el);
    }
  };
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

---

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

---

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
`
  }
]
```



```
<p class="text-xl font-mono text-white">
  двоичная куча. Пара – точка на декартовой
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
  платит  $\approx n \cdot \log n$  сравнений, counting по ма
  точка вставляется спуском от корня:  $x \leq уз$ 
  единственно. Весь процесс – в симуляторе под
    «соединить сам»</b> – кликаешь две точки,
  ь родитель», «получился бы цикл», «равные „
  «Миф  $2k/2k+1$ »</b> и <b>«Поиск  $\neq$  сортировка»</b>
  ждый запрещённый ход.</p>
```

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
  <p class="my-1 font-bold leading-none">
    <span class="text-2xl text-slate-300">И</span>
  </p>
  <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
  rder border-emerald-500 shadow-md">
    Аналогия 1: Split – таможня
  </div>
  <p class="text-slate-300 text-sm">S
  х»». Спуск от корня: узел, который вместе
  скрытия</b> – досматривается только одна ве
  </div>
```

```
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
  border-rose-500 shadow-md">
    Аналогия 2: Merge – стыковка
  </div>
```

```

        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}
}
}

</div>
</details>

<details class="bg-slate-900/40 rounded-lg" >
    <summary class="p-4 font-bold text-slate-300" >
        -b border-slate-700/50">
            Код: вставка, merge, erase (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300" >
            <pre class="bg-slate-950 p-4 rounded" >
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — левый
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)
    <pre class="bg-slate-950 p-4 rounded" >
def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else:
        root.left = root.right = None
        return root
    return root
    </pre>
        </div>
    </details>

```

```

        <p class="text-xs uppercase tracking-widest">Можно
        <p class="text-white font-bold">можно
        <p class="text-slate-400 text-xs mt-4">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
        <p class="text-xs uppercase tracking-widest">Можно
        <p class="text-emerald-300 font-bold">можно
        <p class="text-slate-400 text-xs mt-4">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
        <p class="text-xs uppercase tracking-widest">Можно
        <p class="text-white font-bold">0(n
        <p class="text-slate-400 text-xs mt-4">
    </div>
</div>
</div>

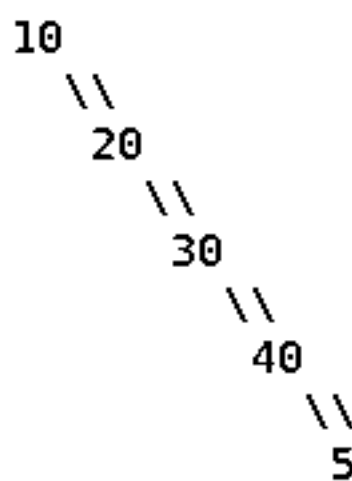
```

<!-- 1. Зачем -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border">
    <h3 class="text-lg font-bold text-white mb-3">
    <p class="text-slate-300 text-sm mb-4">В об
    ючи по возрастанию — и дерево вырождается в
    <pre class="bg-slate-950 p-4 rounded-lg over
    ading-relaxed">вставили 10, 20, 30, 40, 50

```



высота = 5, поиск 50 = 5 сравнений  
это уже не дерево, а односвязный

```

    50</pre>
    <p class="text-slate-300 text-sm mt-4">Есть
    <ul class="list-disc list-inside text-slate">
        <li><span class="text-white font-bold">
каждой вставки чинят дерево, чтобы оно <em>

```

```

      20      C      ----->      A      40
      /      \      <-----      /      \
     A        B      поворот 40      B        C

```

обход слева направо ДО: A 20 B 40 C

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же сам

```
<div class="grid grid-cols-1 md:grid-cols-2"
```

```
<div class="bg-slate-900 rounded-lg p-4"
```

```
<p class="text-emerald-400 font-bol
```

```
<p class="text-slate-300 text-xs">П
```

сле поворотов – сломать его поворотами нево.

```
</div>
```

```
<div class="bg-slate-900 rounded-lg p-4"
```

```
<p class="text-amber-400 font-bold"
```

```
<p class="text-slate-300 text-xs">П
```

одно и то же место, поэтому B просто перев

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- 4. Три случая -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
```

```
<h3 class="text-lg font-bold text-white mb-3"
```

```
<p class="text-slate-300 text-sm mb-4">Обоз
```

узел, который поднимаем, <span class="font

```
>g</span> – дед (родитель родителя). Смотри
```

```
<div class="overflow-x-auto -mx-2 px-2">
```

```
<table class="w-full text-xs sm:text-sm"
```

```
<thead>
```

```
<tr class="text-left text-slate
```

```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 pr-3 font-b
```

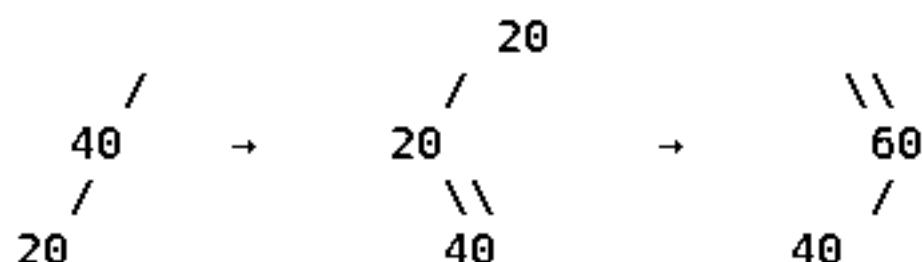
```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```



было: бамбук влево

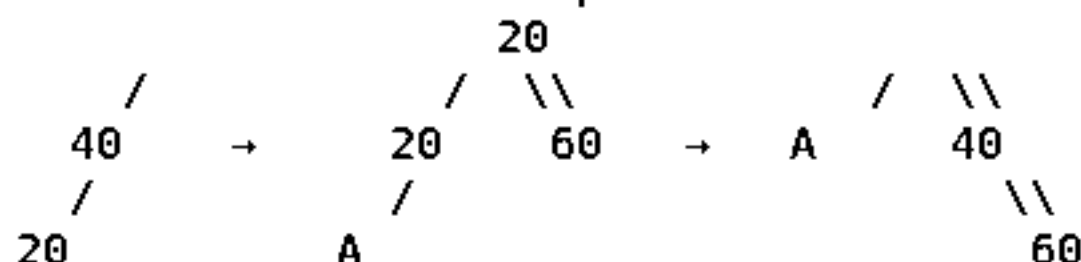
стало: бамбук вправо

глубина никого не сократилась!</pre>

```

      <p class="text-slate-400 text-xs mt-4">
        мортизации не получится.</p>
    </div>
    <div class="bg-slate-950 rounded-lg p-4">
      <p class="text-emerald-400 font-bold">
        <pre class="overflow-x-auto text-[12px] font-mono">

```



было: линия из 3 узлов

стало: кустик высоты 2

глубина ветки упала вдвое!</pre>

```

      <p class="text-slate-400 text-xs mt-4">
        ем.</p>
    </div>
    </div>
    <p class="text-slate-300 text-sm mt-4"><span>
      з, змейка (zig-zag) – крути СНИЗУ вверх.</span>
    </div>

```

```

<!-- 6. Полный пример -->
<div class="bg-slate-800/60 p-6 rounded-xl border-2">
  <h3 class="text-lg font-bold text-white mb-4">
    <p class="text-slate-300 text-sm mb-4">Дерево
      3 шага, а потом делаем Splay(10).</p>
    <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto">

```

```

    </div>
  </div>
</div>

```

```

сти запросов splay-дерево не хуже (с точностью
дерево. Оно как бы само подстраивается под
</div>

```

```

<!-- 8. Операции -->

```

```

<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
  <h3 class="text-lg font-bold text-white mb-3">Операции
  <p class="text-slate-300 text-sm mb-4">Красим
  лается в корне.</p>
  <div class="space-y-2 text-sm">
    <div class="bg-slate-900 rounded-lg p-3" style="border: 1px solid #333;">
      </span> <span class="text-slate-400">— спускаем
      маем последний узел на пути (соседний по значению)
      <div class="bg-slate-900 rounded-lg p-3" style="border: 1px solid #333;">
        )</span> <span class="text-slate-400">— Splay
        росто отрезаем ссылки.</span></div>
        <div class="bg-slate-900 rounded-lg p-3" style="border: 1px solid #333;">
          k)</span> <span class="text-slate-400">— splay
        /div>
        <div class="bg-slate-900 rounded-lg p-3" style="border: 1px solid #333;">
          )</span> <span class="text-slate-400">— Splay
          него не будет правого сына — туда и вешаем левый
        </div>
      </div>
    </div>
  </div>

```

```

<!-- 9. Код -->

```

```

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    open:border-b border-slate-700/50">
    Код: rotate + splay + find (нажми, чтобы увидеть)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <pre class="bg-slate-950 p-4 rounded overflow-x-auto">
      // узел: { key, left, right, parent }
    </pre>
  </div>

```

```

// один поворот: x встаёт на место своего родителя

```

```

function rotate(x) {
  const p = x.parent, g = p.parent;

```

```

        last = cur;
        if (key === cur.key) break;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return last ? splay(last) : null;    // splay делает
}

```

<p class="text-xs text-slate-400">Вся р
</span>
ate(x);</span> против font-monospace<span class="font-monospace">
т свою оценку.</p>

</div>

</details>

<!-- 10. Плюсы/минусы -->

<div class="grid grid-cols-1 md:grid-cols-2 gap-

<div class="bg-emerald-950/30 rounded-xl p-5 b

<p class="text-emerald-300 font-bold mb-

<ul class="list-disc list-inside text-s-

<li>Простой код: нет высот, цветов,

<li>Меньше памяти, чем у AVL и красн

<li>Сам подстраивается под «горячие

<li>split / merge получаются почти

</ul>

</div>

<div class="bg-rose-950/30 rounded-xl p-5 b

<p class="text-rose-300 font-bold mb-2"

<ul class="list-disc list-inside text-s-

<li>Нет гарантии на <em>отдельную</

<li>Не годится для реального времени

<li>Дерево меняется даже при чтении

<li>Постоянные записи в память при

</ul>

</div>

</div>

<!-- 11. Вопросы -->

<details class="bg-slate-900/40 rounded-lg bord

<summary class="p-4 font-bold text-slate-40

open:border-b border-slate-700/50">

```

        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1]
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            ≈ Аналогия 1: Вырезание прямоугольника
          </div>
          <p class="text-slate-300 text-sm mb-4">
            з него маленький целевой прямоугольник в центре
            ый верхний угол отрезается дважды! Поэтому
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
перед.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Учеба в вузе
        </div>
        <p class="text-slate-300 text-sm mb
еская сортировка – это расписание, которое
>
      </div>
      <div class="bg-slate-900 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
          Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выявл
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
    category: "Графы. Продвинутое",
    content: `

```

```

        <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400">
            <h3 class="text-xl font-bold text-rose-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-rose-300 italic">
                        <p class="text-xl font-mono text-white">
                            и  $fup[v] > tin[u]$ .</p>
                    </div>
                    <div class="grid grid-cols-1 gap-6">
                        <div class="bg-slate-800 p-6 rounded-lg">
                            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                                Аналогия 1: Единственный мост
                            </div>
                            <p class="text-slate-300 text-sm mb-4">
                                ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
                            </p>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает связность",
        category: "Графы. Продвинутое",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
        </div>
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border

```

```

</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Рисование не отрыв
    </div>
    <p class="text-slate-300 text-sm mb
е сходится нечетное число линий, значит, из
а везде должно быть четное число (зашел-выш
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "pathfinding-accel",
    title: "17. Графы. Ускорения поиска",
    type: "html",
    description: "",
    category: "Графы. Пути",
    content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-blue-400 r
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-blue-300 italic r
        <p class="text-xl font-mono text-white"
d-длина.</p>
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg

```

```

        </div>
        <p class="text-slate-300 text-sm mb
с самых дешевых дорог в стране". Он строит
единую сеть.</p>
    </div>
</div>
</div>
</div>
</section>`,
    },
    {
      id: "mst-prima",
      title: "19. Графы. Остовное дерево. Прима",
      type: "html",
      description: "",
      category: "Графы. Остовные",
      content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-emerald-4
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-emerald-300 ital
        <p class="text-xl font-mono text-white"
        которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              ✦ Аналогия: Заражение вирусом
            </div>
          <p class="text-slate-300 text-sm mb
ли вирус). Мы стартуем из одного города, и

```

```

        </div>
    </div>
</section>`,
    },
    {
        id: "string-kmp",
        title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
            i]), который одновременно является её суффиксом.
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <!-- Аналогия 1 -->
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border
                    rder border-emerald-500 shadow-md">
                        Аналогия 1: Бумеранг (Префикс и суффикс)
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        идишь "АБРА". Если ты ошибешься при поиске
                        концовка "АБРА" совпадает с началом "АБРА"
                        моя перепроверок!</p>
                    </div>
                </div>
            <details class="bg-slate-900/40 rounded-lg

```

```

    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
          Аналогия 1: Копипаста
        </div>
        <p class="text-slate-300 text-sm mb-2">
          в тексте кричит: "Эй! Начиная с этой буквы
          т "окно" [L, R] самой дальней найденной копии
        </div>
      </div>
      <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
        <summary class="p-4 font-bold text-slate-300">
          Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
          <pre class="bg-slate-950 p-4 rounded">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
          </div>
        </details>
      </div>
    </div>
  </section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `

```

```
    },  
  ];  
  
  -----
```

ФАЙЛ 18/82: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОВОК: 385 | РАЗМЕР: 10.5 КБ

---

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html";  
  content: string;  
}  
  
export const chapters: Chapter[] = [  
  {  
    id: "euler-path-vs-cycle",  
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",  
    type: "html",  
    content: `  
    <section id="euler-path-vs-cycle" class="text-white">  
      <div class="flex items-center mb-6">  
        <span class="bg-indigo-600 text-white px-4 py-1 text-sm font-medium">  
          <h2 class="text-3xl font-bold text-white">Не буй!  
        </div>  
  
        <div class="space-y-8">  
          <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">  
            <h3 class="text-xl font-bold text-blue-400">Путь ≠ Цикл  
  
            <div class="bg-slate-900 p-4 rounded-lg mb-4">  
              <p class="text-lg text-blue-300 italic">Эйлер проложил путь, который не повторяет  
              <div class="text-left font-mono text-sm">  
                <p><strong class="text-white">Эйлер проложил путь, который не повторяет  
                ин раз. Вершины повторять можно.</p>  
                <p><strong class="text-white">Эйлер проложил путь, который не повторяет  
                <div class="p-3 bg-slate-950 rounded">
```

```

        Главное – зайти и выйти, не заходя.
        Никто не знает, как решать быстро.
        <br><span class="text-xs text-rose-400">
    лноту).</span>
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-1
    <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
    Душный режим: Формальное доказательство
</summary>
<div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400">Теорема Эйлера
    <p>
        Связный граф содержит эйлеров цикл ⇔
        <br>Эйлеров путь (не цикл) ⇔ ровно 0 или 2 вершины
        <br>степени нечётности.
    </p>
    <p class="text-rose-400">Почему Гамильтон?
    <p>
        Задача коммивояжёра (TSP) – это взрыв сложности.
        Доказано, что любой полиномиальный алгоритм
        В общем, не парься. Просто запомни.
    </p>
    </div>
</details>
</div>
</section>`,
    },
    {
        id: "bridges-code",
        title: "Мосты в графах: код + визуализация",
        type: "html",
        content: `<section id="bridges-code" class="mb-20 space-y-4">
    <div class="flex items-center mb-6">
        <span class="bg-emerald-600 text-white px-4 py-2 rounded">
        <h2 class="text-3xl font-bold text-white">Мосты
```



```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
```

Аналогия: Кровеносный сосуд

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Граф — это кровеносная система. Ребра (вершины и терали) — это не мост, живём.

Если же перерезать единственную артерию, то...

Алгоритм DFS ищет такие «критические» ребра.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
  <summary class="p-4 font-bold text-slate-400 border-rose-500 shadow-md">
```

Полный код на C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p>Классическая реализация — <code class="text-slate-400 font-mono">
  оходов.</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg border-rose-500 shadow-md">
```

```
<pre class="text-xs font-mono text-slate-400">
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежности
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```

    },
    {
      id: "planarity-euler-formula",
      title: "Планарность: K5, K3,3 и формула Эйлера",
      type: "html",
      content: `<section id="planarity-euler-formula" class="planarity-euler-formula">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Планарность
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">Планарность

      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
          <p class="text-3xl font-mono text-white">
            <div class="text-sm text-slate-400 space-y-2">
              <p><span class="text-blue-400 font-mono">
                <p><span class="text-emerald-400 font-mono">
                <p><span class="text-rose-400 font-mono">
              </div>
            </div>

            <p class="text-slate-300 text-sm">
              <strong>Следствие:</strong> Для планарного графа:
              <math>V - E + F = 2</math>
              Если ребер больше — граф гарантированно не планарен.
            </p>
          </div>

          <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
            <div class="bg-slate-800 p-6 rounded-lg border">
              <div class="absolute -top-3 left-4 bg-slate-800 text-white font-weight-bold text-sm">
                order-green-500 shadow-md">
                  K5: Полный граф на 5 вершинах
                </div>
              </div>
            </div>
          </div>
        `
    }
  ]
}
```



то выше неё прыгать некуда). Поэтому для него у него **больше 1 независимого ребенка**

</p>

<div class="bg-slate-950 p-4 rounded-lg">

<pre class="text-xs font-mono text-white">

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">

<h4 class="text-lg font-bold text-indigo-400">

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориентацией

</p>

<ul class="list-disc list-inside text-sm text-white">

<li><strong class="text-rose-400">Точки

онг> и показывают физическую уязвимость свя

<li><strong class="text-emerald-400">SC

```
const dedup = new Map<string, Chapter>();
merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
  const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
  return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler",
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
  "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi",
  "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
  "string-z-func", "segment-trees", "sparse-table", "dynam
];
```

```

    },
    {
      id: "dfs",
      labels: ["DFS", "обход в глубину", "поиск в глубину"],
      title: "DFS — обход в глубину",
      short:
        "Прёшь вперёд до упора, упёрся — откатываешься на",
      formal:
        "Стек (или рекурсия). Из DFS растут: времена вход",
      complexity: "O(V + E)",
      demo: "dfs",
      simulator: "stack-dfs",
    },
    {
      id: "binsect",
      labels: ["binsect", "бинсект", "бинарный поиск", "д"],
      title: "Бинарный поиск (и его самозванец binsect)",
      short:
        "Бинарный поиск НАХОДИТ готовый элемент в уже отс",
        "о не сортирует. «binsect» из конспекта — шутилив",
        "quicksort.",
      formal:
        "Поиск одного элемента в отсортированном массиве -",
        "O(n log n). Бинарный поиск сортировку не выполн",
      complexity: "поиск: O(log n); сортировкой не являет",
    },
    {
      id: "heap",
      labels: ["куча", "кучу", "кучи", "куче", "кучей", "к"],
      title: "Куча (priority queue)",
      short:
        "Не отсортированный список, а «мешок, из которого",
      formal:
        "Полное бинарное дерево в массиве: дети узла i ле",
      complexity: "вставка/извлечение O(log n), «посмотре",
      demo: "heap",
      simulator: "heap-beam-search",
    },
  ],

```

```

    demo: "trie",
    simulator: "aho-corasick",
},
{
    id: "bfs-on-trie",
    labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
    ],
    title: "Обход бора в ширину (тот самый «какой-то обход»)",
    short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную очередь  

        о есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень).  

        Кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки"],
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла»",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},

```

```

        "разреженная таблица", "разреженная таблица", "sparse-table",
        "разреженная", "двоичные подьёмы", "двоичный подьём",
    ],
    title: "Разреженная таблица (метод двоичных подьёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1, 2, ..., m.",
    formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ ",
    demo: "sparse-table",
    simulator: "sparse-table",
},
{
    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дерево отрезков"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь массив, а остальные за куски.",
    formal: "Строится за  $O(n)$ , запрос и обновление —  $O(\log n)$ ",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается на две части.",
    formal: "Ребро (u,v) — мост, если  $low[v] > tin[u]$ , где  $low[v]$  — минимальный индекс вершины в поддереве, достижимом из v, не считая ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точки сочленения"],

```



```

        "класс Р", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (судно
        не в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полнота",
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов",
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
    formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — раск
        к путей сохраняется.",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты",
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждо
    formal: "Косарайю: DFS с сохранением порядка выхода",
    complexity: " $O(V + E)$ ",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функ
        "префикс", "префикса", "суффиксом", "суффикс", "суб
        "подстроки", "подстроку", "подстрока",
    ],

```

```

    },
    {
      id: "shortest-path",
      labels: ["кратчайший путь", "кратчайшие пути", "кра",
      title: "Кратчайший путь",
      short:
        "Самый дешёвый маршрут из А в В. «Дешёвый» – по с
      formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Форд
          ы,  $O(V^3)$ .",
      demo: "relax",
      simulator: "dijkstra",
    },
    {
      id: "negative-cycle",
      labels: ["отрицательный цикл", "отрицательного цикла",
      title: "Отрицательный цикл",
      short:
        "Круг, пройдя по которому вы становитесь «богаче»
      formal:
        "Форд–Беллман: если на  $V$ -й итерации ещё удастся с
      demo: "relax",
      simulator: "bellman-ford",
    },
    {
      id: "greedy",
      labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешевое ребро", "самое дешёвое ребро", "са
      ],
      title: "Жадный алгоритм",
      short:
        "На каждом шаге хватаем то, что лучше прямо сейчас
      formal:
        "Корректность обычно доказывается «леммой о разре
      demo: "mst",
      simulator: "mst-kruskal",
    },
  ],

```



```
{
  id: "kruskal",
  labels: ["Краскал", "Краскала", "Краскалу", "Kruska"],
  title: "Алгоритм Краскала",
  short:
    "Сортируем все рёбра по весу и берём подряд те, ч
  formal:
    "Сортировка  $O(E \log E) + E$  операций union/find. C
  complexity: " $O(E \log E)$ ",
  demo: "mst",
  simulator: "mst-kruskal",
},
{
  id: "components",
  labels: [
    "компонента связности", "компоненты связности", "
    "компоненте связности", "связности", "связный", "
  ],
  title: "Компоненты связности",
  short:
    "Граф-архипелаг: острова, между которыми нет мост
    бой.",
  formal:
    "Считаются одним проходом: пока есть непосещённая
  complexity: " $O(V + E)$ ",
  demo: "components",
  simulator: "graph-dfs-bfs",
},
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "гра
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Горо
  formal:
```

```

    complexity: "O(высоты): от  $O(\log n)$  до  $O(n)$ ",
    demo: "segment-tree",
},
{
    id: "inclusion-exclusion",
    labels: [
        "включений-исключений", "включения-исключения", "вырезание прямоугольников",
    ],
    title: "Включения-исключения на префиксных суммах",
    short:
        "Чтобы получить сумму прямоугольника, берём большой отрезали дважды.",
    formal:
        "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
    complexity: "запрос  $O(1)$  после  $O(nm)$  предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведё"],
    title: "Сведение задач",
    short:
        "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
    formal:
        "A ≤p B: есть полиномиальная функция, переводящая решение задачи A в решение задачи B.",
    demo: "np",
},
{
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива. Предподсчёты.",
    formal:

```

```

    demo: "zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zig",
    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
    title: "Zig-Zig – x и родитель с одной стороны",
    short:
      "Дерево вытянулось в «бамбук»: g → p → x идут в одну сторону (p, x). Именно это вдвое сокращает глубину всего дерева",
    formal:
      "Если крутить наивно (два раза поднимать x), бамбук превращается в AVL-дерево",
    demo: "zig-zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag – x и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»: p – левый сын g, а x – правый сын p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева",
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (чтобы было удобнее читать) */
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  GLOSSARY.map((e) => e.labels.map((label) => ({ label, id: e.id })))
    .sort((a, b) => b.label.length - a.label.length);

```

---

```
</div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
```

```
    <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
      <p class="text-lg text-emerald-300 italic">Аналогия 1: Позитивное планирование
```

```
      <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
```

```
    </div>
```

```
  <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
    <!-- Аналогия 1 -->
```

```
    <div class="bg-slate-800 p-6 rounded-lg">
```

```
      <div class="absolute -top-3 left-4 right-4 bottom-4 border-emerald-500 shadow-md">
```

```
        Аналогия 1: Позитивное планирование
```

```
      </div>
```

```
      <p class="text-slate-300 text-sm mb-4">Аналогия 1: Позитивное планирование  
(нельзя поехать и заработать). Он всегда в
```

```
      <div id="slot-chapter-image-dijkstra">
```

```
    </div>
```

```
    <!-- Аналогия 2 -->
```

```
    <div class="bg-slate-900 p-6 rounded-lg">
```

```
      <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
```

```
        Ограничения
```

```
      </div>
```

```
      <p class="text-slate-300 text-sm mb-4">Ограничения  
охождение улицы), Дейкстра сломается, так к
```

```
    </div>
```

```
  </div>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```
  <summary class="p-4 font-bold text-slate-300">Строгое доказательство (Скрыто)
```

```
  <div class="border-slate-700/50">
```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
        еряет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman" class="img-fluid">
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4
            border-rose-500 shadow-md">
                Отрицательный цикл
            </div>
            <p class="text-slate-300 text-sm mb-6">
            г станет ЕЩЕ короче – значит мы попали во в
            </div>
        </div>

        <div id="slot-bellman-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "floyd",
        title: "Билет 16. Флойд-Уоршелл",
        type: "html",
        category: "Графы",
        content: `<section id="floyd-content" class="mb-12">
        <div class="flex items-center mb-6">
            <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-3xl font-bold text-white">Флойд
        </div>

        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-indigo-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-6">

```



```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l">
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
      <p class="text-sm text-blue-300 italic mb-2">0с
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
  </div>
```

```
<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo">
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы движемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2">
  <div class="absolute -top-3 left-4 bg-rose-900
    -500 shadow-md">
    Аналогия 2: Матёшка (Терминальные)
  </div>
  <p class="text-slate-300 text-sm mb-4">Представ
    нашли и "he", потому что оно спрятано внутри
    терминальные ссылки</b> (term link) – прямые мос
```

```

        // Если суффиксная ссылка сама является словом
        // Иначе наследуем terminal_link
        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].link;
        } else {
            nodes[child].term_link = nodes[nodes[child].link].term_link;
        }

        q.push(child);
    }
}

</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-12">
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">Алгоритмы
    <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
        <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут
        содержать очень большие и маленькие числа (долгота и широта), мы часто
        имеем дело с числами, которые лежат в диапазоне от 0 до N, и уже по ним строим графы.</p>
    </div>
    </div>
</div>
</section>`

```

```

        ого критерия. Это NP-полная задача, которую не
    },
    {
        question: "Куда ведёт эйлеров путь, если нечётных
        options: [
            "Всё ещё можно пройти по всем рёбрам ровно раз",
            "Такого графа не существует",
            "Эйлерова пути нет — нужно удалять лишние рёбра",
        ],
        correctIndex: 2,
        explanation: "Если нечётных вершин больше 2, эйлер
            ровно 2 нечётные или 0."
    }
],
"bridges-code": [
    {
        question: "После DFS у нас low[u] = 4, а tin[v] =
        options: [
            "Да, так как low[u] (4) > tin[v] (2)",
            "Нет, low[u] должно быть меньше или равно",
            "Только если граф связный и неориентированный"
        ],
        correctIndex: 0,
        explanation: "Условие моста: low[u] > tin[v]. В н
    },
    {
        question: "Для чего нужен массив low[v]?",
        options: [
            "Для того чтобы было больше переменных в коде",
            "Чтобы знать минимальный tin предка, до которого
            "Чтобы хранить глубину рекурсии"
        ],
        correctIndex: 1,
        explanation: "low[v] хранит минимальное время вхо
            поиска мостов и точек сочленения."
    },
    {
        question: "Какова временная сложность алгоритма п

```

```
        correctIndex: 0,
        explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9
    }
}
};
```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 23/82: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../types';
```

```
export const tickets1to5: Chapter[] = [
```

```
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description: "Дерево отрезков – структура данных для",
    category: "Оптимизации и Продвинутое",
    content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div id="intro" class="bg-slate-700/50 p-6 rounded">
      <h3 class="text-xl font-bold text-blue-400">

      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
```

```

        lazy[2 * node + 1] += lazy[node];
        tree[2 * node + 1] += lazy[node];
        lazy[node] = 0;
    }
}
</pre>
</div>
</details>
</div>
</div>
</section>
<div id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

<span class="bg-indigo-600 text-white px-4 py-1">
  <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
    <h3 class="text-xl font-bold text-blue-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
        метода: Split и Merge.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Удар катаной (Split)
        </div>
        <p class="text-slate-300 text-sm mb-4">
          по значению X. Все, кто меньше X улетают вправо.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия 2: Слияние капель (Merge)
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вого). Мы просто смотрим на корни: у кого Y больше, тот становится левым, а кто меньше, тот становится правым.
            как потомок.</p>
          </div>
        </div>
      <details class="bg-slate-900/40 rounded-lg border border-slate-800">
        <summary class="p-4 font-bold text-slate-300">
          Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
          <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {

```

```

        <div class="bg-slate-800 p-6 rounded-lg"
          <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb
вится корнем дерева (VIP-статус). Если ты ч
ка почти не требуется времени!</p>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg"
          <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
            Аналогия 2: Балансировка чере
          </div>
          <p class="text-slate-300 text-sm mb
и становиться кривым как бамбук. Но как тол
</p>
        </div>
      </div>
      <details class="bg-slate-900/40 rounded-lg"
        <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
          Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300"
          <pre class="bg-slate-950 p-4 rounded
// Zig, Zag
function rotate(x) {
  let p = x.parent;
  if (p.left === x) { // Right rotate
    p.left = x.right;
    x.right = p;
  } else { // Left rotate
    p.right = x.left;
    x.left = p;
  }
}</pre>
      </div>
```

```

    type: "html",
    description: "Сжатие суммы подматрицы за  $O(1)$ ",
    category: "Алгоритмы матрицы",
    content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          ,  $y1...y2$ ) =  $P[x2][y2]$  -  $P[x1-1][y2]$  -  $P[x2][y1-1]$ 
        </p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ≈ Аналогия 1: Вырезание прямоугольника
          </div>
          <p class="text-slate-300 text-sm mb-4">
            з него маленький целевой прямоугольник в центре.
            ый верхний угол отрезается дважды! Поэтому
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`
  }
];

```



```

        </div>
        <p class="text-slate-300 text-sm mb-6">
            дный и не умеет пересматривать уже "зафиксирова
        </div>
    </div>
    <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
    id: "bellman-ford",
    title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-rose-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-6">
                    <p class="text-lg text-rose-300 italic">
                    <p class="text-xl font-mono text-white">
                </div>
                <div class="grid grid-cols-1 xl:grid-cols-2">
                    <div class="bg-slate-800 p-6 rounded-lg">
                        <div class="absolute -top-3 left-4 border
                        order border-emerald-500 shadow-md">
                            ♂ Аналогия 1: Параноик
                        </div>
                        <p class="text-slate-300 text-sm mb-6">
                            еряет ВСЕ возможные дороги V-1 раз подряд.
                        </div>
                        <div class="bg-slate-900 p-6 rounded-lg"

```

```

        Главный герой – <b>внешний цикл</b>
        шаг <code class="bg-slate-900 text-pink-400">шаг k?</code>
        шу себе проходить через вершину k?»</i>
    </p>
    <ul class="text-sm text-slate-300 space-y-4">
        <li><b>Шаг k=0:</b> Ты знаешь только k=0
        <li><b>Шаг k=1:</b> Разрешаем пересадки
        e> через путь <code class="text-indigo-300">шаг k?</code>
        <li><b>Шаг k=2:</b> Разрешаем пересадки
        нутри этих путей уже могут быть пересадки ч
        <li>...</li>
        <li><b>Шаг k=N:</b> Ты проверил все
    </li>
    </ul>
</div>
</div>
</div>
</section>`
    },
    {
        id: "johnson-algo",
        title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-amber-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-xl font-mono text-white">
            </div>
            <div class="grid grid-cols-1 gap-6 mb-6">
                <div class="bg-slate-800 p-6 rounded-lg

```

```

<h3 class="text-xl font-bold text-emerald-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
  <p class="text-xl font-mono text-white">
    (проверяем через DSU) - берем в ответ.</p>
</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
      Аналогия: Прокладка интернет
    </div>
    <p class="text-slate-300 text-sm mb-4">
      с самых дешевых дорог в стране". Он строит
      единую сеть.</p>
  </div>
</div>
</div>
</div>
</section>`
  },
  {
    id: "mst-prima",
    title: "19. Графы. Остовное дерево. Прима",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">

```

Универсальное пособие • полный исходный код

```
-----
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb
изкому племени для объединения. К вечеру пл
ства шлют гонцов к ближайшему чужому царств
        </div>
    </div>
</div>
</section>`
    }
};
```

-----  
ФАЙЛ 25/82: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```
-----
import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
    {
        id: "string-kmp",
        title: "21. Префикс-функция (π) – Взгляд назад (КМП)",
        type: "html",
        category: "Строки",
        content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-blue-400`
    }
];
```

```

        -b border-slate-700/50">
            Пример вычисления (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <p>Тот же пример: <b>abcabcd</b></p>
            <ul class="list-disc pl-5 space-y-2">
                <li><b>a</b>: Префиксов нет. <code>abc</code></li>
                <li><b>ab</b>: Из начала ("a") не получилось
            >
                <li><b>abc</b>: Опять мимо. <code>abca</code></li>
                <li><b>abca</b>: 0! Начало ("a") не получилось
            </code></li>
                <li><b>abcab</b>: Начало ("ab") не получилось
            </li>
                <li><b>abcabc</b>: Начало ("abc") не получилось
            <e></li>
                <li><b>abcabcd</b>: "d" всё испортило
            <code>"abca" vs "abcd"</code>}. Конец "abcd" не совпадает с "abca"
            </ul>
        </div>
    </details>
    <details class="bg-slate-900/40 rounded-lg p-5">
        <summary class="p-4 font-bold text-slate-300">
            -b border-slate-700/50">
                Код C++ (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
            </div>
        </details>
    </div>

```

```

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px; background-color: #1e293b; color: white;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md" style="border: 2px solid #f44336; border-radius: 10px; background-color: #1e293b; color: white;">
                Аналогия 2: LLM Самокопиаст
            </div>
            <p class="text-slate-300 text-sm mb-0" style="color: #a6b8c7; font-size: 0.9em; margin-bottom: 0;">
                В LLM Z-функция может применяться для поиска вхождений.
                <code>Z[i]</code> (где <i>i</i> указывает назад на начало строки)
                [i]</code> (где <i>i</i> указывает назад на начало строки)
                сама себя.
            </p>
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg" style="background-color: #1e293b; border: 1px solid #000000; border-radius: 10px; margin-top: 10px;">
        <summary class="p-4 font-bold text-slate-300" style="background-color: #1e293b; color: white; padding: 10px; border: 1px solid #000000; border-radius: 10px;">
            -b border-slate-700/50" style="background-color: #1e293b; color: white; padding: 10px; border: 1px solid #000000; border-radius: 10px;">
                Пример вычисления (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300" style="background-color: #1e293b; color: white; padding: 10px; border: 1px solid #000000; border-radius: 10px;">
                <p>Тот же пример: <b>abcabacd</b></p>
                <ul class="list-disc pl-5 space-y-2" style="list-style-type: none; padding-left: 20px;">
                    <li><code>Z[0]</code> – не считаем (начало строки)
                    <li><code>Z[1]</code> (<b>b</b> не совпадает с <b>a</b>)
                    <li><code>Z[2]</code> (<b>cab</b> совпадает с <b>abc</b>)
                    <li><code>Z[3]</code> (<b>ab</b> не совпадает с <b>ac</b>)
                    <li><code>Z[4]</code> (<b>acd</b> не совпадает с <b>abc</b>)
                    <li><code>Z[5]</code> (<b>acd</b> не совпадает с <b>abc</b>)
                    <li><code>Z[6]</code> (<b>acd</b> не совпадает с <b>abc</b>)
                </ul>
            </div>
        </details>
        <details class="bg-slate-900/40 rounded-lg" style="background-color: #1e293b; border: 1px solid #000000; border-radius: 10px; margin-top: 10px;">
            <summary class="p-4 font-bold text-slate-300" style="background-color: #1e293b; color: white; padding: 10px; border: 1px solid #000000; border-radius: 10px;">
                -b border-slate-700/50" style="background-color: #1e293b; color: white; padding: 10px; border: 1px solid #000000; border-radius: 10px;">
                    Код C++ (Скрыто)
                </summary>
            </div>
    </div>

```

```

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-700 p-2 text-white">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представим, что мы падаем по <b>суффиксной ссылке</b> ("he")
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-rose-900 p-2 text-white">
      Аналогия 2: Матёшка (Терминальные)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представим, что мы нашли и "he", потому что оно спрятано внутри
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-rose-500 shadow-md">
  <summary class="p-4 font-bold text-slate-400 outline-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <pre class="bg-slate-950 p-4 rounded overflow-x-auto">
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch);
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {

```

```

        енную сетку (ответ), он БЫСТРО (за класс Р)
      </div>
      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; box-shadow: 0 10px 30px #4b4b9b; position: relative; height: 150px; display: flex; align-items: center; justify-content: center;">
        <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
          Аналогия 2: Машина-переводчик
        </div>
        <p class="text-slate-300 text-sm mb-0">
          ы отличный переводчик на английский (Сведения о машине В, то В – это <b>NP-Полная</b> (сосредоточена на
        </div>
      </div>
    </div>
  </div>
</section>`
  }
];

```

ФАЙЛ 26/82: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
  {
    id: "graph-dfs-bfs",
    title: "6. Графы. Представление, DFS, BFS",
    type: "html",
    description: "Представление графов и базовые обходы",
    category: "Графы. База",
    content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы

```



```

        <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
            ♂ DFS (Поиск в глубину) = Жадный
        </div>
        <p class="text-slate-300 text-sm mb-2">
            <b>Что делает:</b> Жадный алгоритм находит ближайшую
            ую вершину (например, минимальную по номеру) и пробует другой путь.
        </p>
        <ul class="text-xs text-indigo-300 list-disc">
            <li><b>Где используется:</b></li>
            <li>Топологическая сортировка (и поиск мостов и точек сочленения)
            <li>Поиск мостов и точек сочленения
            <li>Сильно связанные компоненты Косараса
            <li>Поиск циклов и просто проверка связности
        </ul>
    </div>

    <!-- Аналогия BFS -->
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ♂ BFS (Поиск в ширину) = Волна
        </div>
        <p class="text-slate-300 text-sm mb-2">
            <b>Что делает:</b> Концентрические волны.
            Работает через Очередь (FIFO). Продвигается от источника.
        </p>
        <ul class="text-xs text-emerald-300 list-disc">
            <li><b>Где используется:</b></li>
            <li>Кратчайший путь в Невзвешенном графе
            <li>Алгоритм Ахо-Корасик (сборка всех подстрок)
            <li>Алгоритм Форда-Фалкерсона / Поиск в ширину
            <li>Двунаправленный поиск для ускорения
        </ul>
    </div>
</div>

```

```

{
  id: "graph-planar-colors",
  title: "7. Графы. Планарные. Покраска",
  type: "html",
  description: "Формула Эйлера и теорема о 4 красках.",
  category: "Графы. Теория",
  content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">7. Графы. Планарные. Покраска</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Покраска графов</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Теорема 7.1 (Формула Эйлера)</p>
        <p class="text-xl font-mono text-white">Если  $G$  — планарный граф, то  $V + F - E = 2$ , где  $V$  — количество вершин,  $F$  — количество граней,  $E$  — количество ребер.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">Границы стран на карте мира можно считать ребрами графа. Границы не пересекаются в одной точке по-прежнему не сливались цветами, Всегда можно всего 4 цветами раскрасить карту мира.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
},
{
  id: "graph-components",
  title: "8. Графы. Компоненты связности",

```

```
content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb">
еская сортировка – это расписание, которое
>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb">
получить опыт, нужна работа". Алгоритм выяв
        </div>
      </div>
    <div class="bg-slate-900/40 rounded-lg">
      <summary class="p-4 font-bold text-slate">
-b border-slate-700/50">
        Душный мод: Код на C++ (Скрыто)
```

```

content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">SCC
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">SCC
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-emerald-400">SCC
      <div class="text-slate-300 text-sm mb-4">
        <p>SCC (Strongly Connected Components) – это компоненты
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">Компоненты ориентированного графа, в котором
        </div>
      </div>

      <p class="text-slate-300 text-sm mb-4">
        <strong>«Это просто цикл?» – Нет!</strong>
      </p>
      <ul class="list-disc list-inside text-sm text-slate-300">
        <li>Если у тебя есть цикл <code>A → B → C → A</code>
        <li>Если ты добавишь вершину <strong>Г</strong> и четыре вершины – одна большая SCC</strong>.</li>
        <li>Правило: <b>Если два цикла имеют хотя бы одну общую вершину, то они образуют один цикл.</b>
      </ul>

      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия: Интриги в офисе
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Сильная связность – это "клуб слухов", где все связано
            ни в одной SCC. Добавить курьера Гошу, который

```

```

    },
    {
      id: "graph-bridges",
      title: "11. Графы. Мосты",
      type: "html",
      description: "Рёбра, удаление которых расхреначивает",
      category: "Графы. Продвинутое",
      content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
          и  $f_{up}[v] > t_{in}[u]$ .</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
            этот мост, экономикой обоих кусков придет
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "graph-articulation",
      title: "12. Графы. Точки сочленения",

```

Аналогия: Главный перекресток

</div>

<p class="text-slate-300 text-sm mb-4">

Представь город, где два района соединены перекрестком (удалят вершину) — районы будут хрупкостью системы.

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border">

<div class="absolute -top-3 left-4 bg-emerald-500 border-emerald-500 shadow-md">

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border">

<summary class="p-4 font-bold text-slate-400 order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space">

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил то выше неё прыгать некуда). Поэтому для него <strong>больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg">

<pre class="text-xs font-mono text-slate-300">

```
void dfs(int v, int p = -1) {
```

```
    visited[v] = true;
```

```
    tin[v] = low[v] = timer++;
```

```

    </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Эйлеров цикл
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Эйлеров цикл
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Эйлеров цикл — это замкнутый путь,
        <p class="text-xl font-mono text-white">который проходит по каждому ребру графа
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            order border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая
          </div>
          <p class="text-slate-300 text-sm mb-4">Если граф имеет
            е сходится нечетное число линий, значит, из каждой вершины
            а везде должно быть четное число (зашел-вышел)
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
];

```

```

    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 },
    { u: 3, v: 4 }, // 3 and 4 are articulation points (3
    { u: 4, v: 5 },
    { u: 5, v: 6 },
    { u: 6, v: 4 },
  ];

```

```

// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);
edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

```

```

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

```

```

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState(false);

```

```

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();

```



```

        `Возврат в ${v} из ${to}. Обновляем low[${v}
        v,
    );
    if (currentLow[to] >= currentTin[v] && p !==
        ap.add(v);
        recordStep(
            `Найдена точка сочленения: вершина ${v}
            v,
        );
    }
}
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это
        v,
    );
} else if (p === -1) {
    recordStep(
        `Корень дерева DFS ${v} имеет <=1 ребенка, он
        v,
    );
}
};

for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
        dfs(i);
    }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []);

```

```

        className="flex items-center gap-2 px-3 py-3 text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center w-10 h-10 text-white"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-3 gap-4" >
    <div className="lg:col-span-2 relative h-96 lg:h-128" >
      <svg className="w-full h-full" viewBox="0 0 800 800" >
        {/* Edges */}
        {edges.map((edge, i) => {
          const u = nodes.find((n) => n.id === edge.u)
          const v = nodes.find((n) => n.id === edge.v)

          const isBridge =
            (edge.u === 3 && edge.v === 4) ||
            (edge.u === 4 && edge.v === 3);

          return (
            <line
              key={"edge-" + i}
              x1={u.x}
              y1={u.y}
              x2={v.x}
              y2={v.y}
              stroke={isBridge ? "red" : "black"}
              stroke-width={isBridge ? 2 : 1}
            />
          )
        })}
      </svg>
    </div>
  </div>

```

```

        stroke={stroke}
        strokeWidth="3"
        className="transition-all duration-100"
      />
      <text
        x={node.x}
        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
        fontSize="14px"
        fontWeight="bold"
        className="select-none pointer-events-none"
      >
        {node.label}
      </text>

      {isVisited && (
        <>
          <text
            x={node.x}
            y={node.y - r - 15}
            textAnchor="middle"
            fill="#94a3b8"
            fontSize="10px"
          >
            tin:{" "}
            {currentStep.tin[node.id] >= 0
              ? currentStep.tin[node.id]
              : "-"}
          </text>
          <text
            x={node.x}
            y={node.y + r + 20}
            textAnchor="middle"
            fill="#94a3b8"
            fontSize="10px"
          >

```

```

        </div>
        <div className="flex items-center gap-2">
          <div className="w-8 h-1 bg-rose-500 border" style={{border: 1px solid black; height: 10px; width: 20px; margin: 0 auto; position: relative; border-radius: 4px;}}>
            Мост
          </div>
        </div>
      </div>

      <div>
        <h5 className="text-xs text-slate-500 font-medium">ЛОГ ДЕЙСТВИЙ:</h5>
        <div className="space-y-2">
          {steps
            .slice(0, currentStepIndex + 1)
            .reverse()
            .map((step, idx) => {
              <div
                key={idx}
                className={`text-xs p-2 rounded $
xt-slate-500`} `>
                >
                  {step.desc}
                </div>
              </div>
            })}
        </div>
      </div>
    </div>
    <div className="mt-4 shrink-0">
      <div className="w-full bg-slate-800 h-2 rounded" style={{background-color: '#2d3748', height: 10px; border-radius: 4px; margin: 0 auto; position: relative; width: 100%;}}>
        <div
          className="bg-rose-500 h-full transition" style={{
            width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`,
          }} />
      </div>
    </div>
  </div>
</div>

```

```

    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

const getEdges = (cycle: boolean): Edge[] => {
  const defaultEdges = [
    { u: 'S', v: 'A', w: 4 },
    { u: 'S', v: 'B', w: 5 },
    { u: 'B', v: 'C', w: -2 },
    { u: 'A', v: 'C', w: 1 },
    { u: 'C', v: 'D', w: 3 },
    { u: 'C', v: 'E', w: 4 },
    { u: 'D', v: 'F', w: 2 },
    { u: 'E', v: 'F', w: 1 },
  ];
  if (cycle) {
    return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
  } else {
    return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
  }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
  const list: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (list[n.id] = []));
  ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
  return list;
};

const adjList = getAdjList(edges);

const codeLines = [
  { line: 1, text: 'function bellman_ford(graph, start)' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0;' },
  { line: 3, text: '    for i from 1 to V - 1: // |V| - 1 iterations' },
  { line: 4, text: '        for (u, v, weight) in E:' },
  { line: 5, text: '            if dist[u] + weight < dist[v]' },
  { line: 6, text: '                dist[v] = dist[u] + weight;' },

```

```

        }`,
        activeCodeLine: updated ? 6 : 5,
    });
}

if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
        iteration: iter, checkingEdge: null, distances: { ...dist },
        log: ` Ранний выход после итерации ${iter}.`,
        activeCodeLine: 10
    });
    break;
}
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount - 1) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
            simulationSteps.push({
                iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
                distances: { ...dist }, previous: { ...prev },
                log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
                activeCodeLine: 9,
            });
            break;
        }
    }
}

if (!cycleFound) {
    simulationSteps.push({
        iteration: vCount, checkingEdge: null, distances: { ...dist }, previous: { ...prev },
        log: ` Проверка завершена. Ни одно ребро не попало в цикл.`,
        activeCodeLine: 10,
    });
}
}

```

```

        gap-1 ${hasCycle ? 'bg-rose-600 text-white' : ''}
    </div>
    <button onClick={() => { setIsPlaying(false);
        gap-slate-600 text-white px-3 py-2 rounded-lg
    <button onClick={() => setIsPlaying(!isPlaying)}
        position-colors text-white ${isPlaying ? 'bg-rose-600' : 'bg-slate-600'}
    <button onClick={() => { setIsPlaying(false);
        ; }} className="flex items-center gap-1 bg-slate-600 text-white px-3 py-2 rounded-lg"
    </button>
    </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 mt-6" >
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-blue-950/20 p-6 rounded-lg" >
        <div className="absolute top-3 left-3 bg-blue-950/20 p-3 rounded-lg shadow-sm" >
            Итерация {currentStep.iteration} (Шаг {currentStep.step})
        </div>

        {currentStep.hasNegativeCycle && (
            <div className="absolute top-3 right-3 bg-rose-600 text-white px-3 py-2 rounded-lg shadow-sm" >
                animate-pulse shadow-lg shadow-rose-600/50
                ⚠ Отрицательный цикл!
            </div>
        )}
    </div>

    <svg className="w-full h-auto max-h-[500px]" >
        <defs>
            <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10" viewBox="0 0 10 10" >
                <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
            <marker id="arrow-bf-active" markerWidth="10" markerHeight="10" viewBox="0 0 10 10" >
                <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
            <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10" viewBox="0 0 10 10" >
                <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
            </defs>

```

```

            ight="bold" textAnchor="middle" className="font-
                <text x={node.x} y={node.y + 35} fill="white">
split(' ')[0]</text>
            </g>
        });
    }}}}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
    <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
    <h4 className="text-lg font-bold text-emerald-400">
    <div className="space-y-2 text-sm font-mono text-white">
        {Object.keys(adjList).map(u => {
            <div key={u} className="flex flex-wrap gap-2">
                <span className="font-bold text-white">
                    {adjList[u].map((edge, idx) => {
                        const isChk = currentStep.checkingEdge(u, edge.v);
                        return (
                            <span key={idx} className={`${px-2} ${
                                edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
                            }`}>
                                {edge.v} ({edge.w})
                            </span>
                        )
                    })}
                </div>
            )
        })}
    </div>
    </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">

```



```
    });  
  }  
}
```

---

ФАЙЛ 29/82: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react';  
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [  
  { id: 0, label: '0', x: 300, y: 50 },  
  { id: 1, label: '1', x: 150, y: 150 },  
  { id: 2, label: '2', x: 450, y: 150 },  
  { id: 3, label: '3', x: 75, y: 250 },  
  { id: 4, label: '4', x: 225, y: 250 },  
  { id: 5, label: '5', x: 375, y: 250 },  
  { id: 6, label: '6', x: 525, y: 250 },  
];
```

```
const EDGES = [  
  { from: 0, to: 1 }, { from: 0, to: 2 },  
  { from: 1, to: 3 }, { from: 1, to: 4 },  
  { from: 2, to: 5 }, { from: 2, to: 6 }  
];
```

```
const ADJ: { [key: number]: number[] } = {  
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],  
  6: []  
};
```

```
interface Step {  
  activeLine: number;  
  queue: number[];  
  visited: number[];  
  currentNode: number | null;  
  logs: string;
```

```

        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Достаем из очереди (${curr}).`
    });

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
    for (const n of neighbors) {
        if (!visited.has(n)) {
            steps.push({
                activeLine: 7,
                queue: [...queue],
                visited: Array.from(visited),
                currentNode: curr,
                logs: `Рассматриваем соседа: ${n}. Он еще не
            });

            visited.add(n);
            queue.push(n);

            steps.push({
                activeLine: 9,
                queue: [...queue],
                visited: Array.from(visited),
                currentNode: curr,
                logs: `Помечаем ${n} как посещенную и добавляем
            });
        }
    }
} else {
    steps.push({
        activeLine: 6,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Соседей нет или все уже посещены.`
    });
}

```

```

    return () => clearTimeout(timer);
  }, [isPlaying, stepIdx, handleNext]);

return (
  <div className="bg-slate-900 border-2 border-slate-
    <div className="flex flex-col md:flex-row items-c
      <div>
        <h3 className="text-2xl font-bold text-white
          <span className="text-blue-400"> </span> Ал
        </h3>
        <p className="text-slate-400 text-sm mt-1">Ви
      </div>
      <div className="flex bg-slate-800 p-1.5 rounded
        <button
          onClick={() => { setIsPlaying(false); setSt
          className="p-2 text-slate-400 hover:text-wh
          title="Сброс"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
        <button
          onClick={() => setIsPlaying(p => !p)}
          className="p-2 text-blue-400 hover:text-blue
          title={isPlaying ? 'Пауза' : 'Воспроизведен
        >
          {isPlaying ? <Pause className="w-5 h-5" />
        </button>
        <button
          onClick={handleNext}
          disabled={stepIdx >= STEPS.length - 1}
          className="p-2 text-emerald-400 hover:text-
          ed:hover:bg-transparent"
          title="Шаг вперед"
        >
          <StepForward className="w-5 h-5" />
        </button>
      </div>
    </div>
  </div>

```

```

        stroke = '#34d399'; // emerald-400
        scale = 1.1;
    } else if (isVisited) {
        fill = '#0f172a'; // slate-900
        stroke = '#3b82f6'; // blue-500
    }

    return (
      <g key={n.id} className="transition-tran
        {isCurrent && {
          <circle cx={n.x} cy={n.y} r="30" fi
        }}
        <circle
          cx={n.x} cy={n.y}
          r="20"
          fill={fill}
          stroke={stroke}
          strokeWidth="3"
          className="transition-colors duration
        />
        <text
          x={n.x} y={n.y}
          textAnchor="middle"
          dy=".3em"
          className="fill-white font-bold tex
        >
          {n.label}
        </text>
      </g>
    );
  })))
</svg>
</div>

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-sl
    er">

```

```

        step.queue.map(({qId, i}) => {
          <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
            {qId}
          </span>
        ))
      )}
    </div>
    <div className="mt-2 text-sm text-amber-300
ms-center justify-center">
      "{step.logs}"
    </div>
  </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 30/82: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const imageMap: Record<string, { src: string; alt: string;
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: 'Добрый — только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {

```

```
interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
  if (compact) {
    return (
      <div className="flex items-center gap-2 text-xs">
        <button
          disabled={!prev}
          onClick={() => prev && onGo(prev.id)}
          title={prev ? prev.title : "Это первая тема"}
          className="flex items-center gap-1 px-2.5 py-1
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
        >
          <ChevronLeft className="w-4 h-4" />
          <span className="hidden sm:inline">Назад</span>
        </button>
        <span className="text-slate-500 font-mono tabular-nums">
          {index + 1} / {total}
        </span>
        <button
          disabled={!next}
          onClick={() => next && onGo(next.id)}
          title={next ? next.title : "Это последняя тема"}
          className="flex items-center gap-1 px-2.5 py-1
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
        >
```

```
    </nav>
  );
};
```

---

ФАЙЛ 32/82: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Check, Download, Loader2, Maximize2, MousePointer } from 'lucide-react';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from './hints/TermPopover';
import { getViz } from './vizRegistry';
import { ChapterNav } from './ChapterNav';
import { downloadChapterHtml } from '../utils/exportHtml';
```

```
interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator }) => {
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");

  useTermHints(contentRef, [chapter.id]);
```

```

const handlePointerOver = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-link');
  if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-link');
  if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-link');
  if (!el) return;
  e.preventDefault();
  if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-link');
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small">

```



```

        </span>
    </p>

    {viz && {
        <section id="chapter-viz" className="mt-8 scr
        <div className="flex items-center justify-b
        <h3 className="flex items-center gap-2 te
        <Sparkles className="w-4 h-4 text-indig
        Демонстрация: {viz.title}
        </h3>
        <button
            onClick={() => onOpenSimulator(chapter.
            className="flex items-center gap-1.5 te
            ate-300 hover:text-white hover:border-indig
            >
            <Maximize2 className="w-3.5 h-3.5" /> P
        </button>
        </div>
        {viz.hint && <p className="text-xs text-sla
        <viz.Component />
        </section>
    }}

    <ChapterNav prev={prev} next={next} index={inde
    </article>

    <TermPopover
        anchor={anchor}
        onClose={() => setAnchor(null)}
        onOpenSimulator={(id) => {
            setAnchor(null);
            onOpenSimulator(id);
        }}
        onCardEnter={cancelHide}
        onCardLeave={scheduleHide}
    />
    </>
    );

```

```

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],

```

```

    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
        activeEdge: [2,3], backEdge: null,
        log: "Идём C→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3},
        activeEdge: [3,4], backEdge: null,
        log: "D→E. tin[E]=6, low[E]=6.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
        activeEdge: [4,5], backEdge: null,
        log: "E→F. tin[F]=7, low[F]=7.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
        activeEdge: null, backEdge: [5,3],
        log: "Из F видим D (посещён). Обратное ребро (F,D).",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
        activeEdge: null, backEdge: null,
        log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],

```

```

const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
  if (dfsAuto) {
    dfsTimer.current = setInterval(() => {
      setDfsIdx(prev => {
        if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
        return prev + 1;
      });
    }, 2500);
  }
  return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">Step {step}
    </h4>
    <div className="flex items-center gap-1 bg-slate-800 p-1">
      <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
        disabled={dfsIdx === 0}
        className="p-1.5 hover:bg-slate-700 rounded text-white"
        <Undo className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => setDfsAuto(!dfsAuto)}
        className={`px-2.5 py-1 rounded text-[10px] ${
          dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
        }`}
        >{dfsAuto ? "<>" : "<"}
        <Pause className="h-3 w-3" />
      </button>
      <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1);
        disabled={dfsIdx === DFS_STEPS.length - 1}
        className="p-1.5 hover:bg-slate-700 rounded text-white"
        <SkipForward className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => { setDfsAuto(false); setDfsIdx(0);

```

```

    }}())
    {GRAPH_NODES.map(n => {
      const cur = step.currentNode === n.id;
      const vis = step.visitedIndices.includes(n.id);
      const st = step.stack.includes(n.id);
      return <g key={n.id}>
        <circle cx={n.x} cy={n.y} r={12}
          fill={cur ? "#10b981" : st ? "#064e3b" : "#f0f0f0"}
          stroke={cur ? "#fff" : st ? "#34d399" : "#ccc"}
          strokeWidth={2.5}
          className="transition-all duration-300" />
        <text x={n.x} y={n.y+3} textAnchor="middle">
          {n.label}</text>
        {vis && (
          <div>
            <rect x={n.x-18} y={n.y-28} width={36} height={10}
              className="transition-all duration-300" />
            <text x={n.x} y={n.y-20} textAnchor="center">
              {l:{step.low[n.id]||"?"}}</text>
            </div>
          )}
        </g>
      )}
    )}
  </svg>
  <div className="flex items-center justify-between gap-2">
    <span>Шаг {dfsIdx+1}/{DFS_STEPS.length}</span>
    <div className="flex-1 mx-2 bg-slate-950 h-10 rounded-lg border border-slate-800">
      <div className="bg-indigo-600 h-full transition-all duration-300">
      </div>
    </div>
  </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
  {/* Code panel */}
  <div className="bg-slate-950 rounded-lg border border-slate-800 p-3">
    <div className="bg-slate-900 px-3 py-1.5 text-slate-400">

```

```

        { /* Log */
        <div className="bg-indigo-950/10 border border-
          <p className="text-[11px] text-slate-300 le
          <div className="mt-2 pt-2 border-t border-s
            <span className="text-slate-400">Мосты:</
            <span className="font-mono font-bold text
              {step.bridges.length > 0 ? step.bridges
            </span>
          </div>
        </div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 34/82: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

export default function DijkstraViz() {

```

```

    { line: 8, text: '                if dist[u] + weight <
    { line: 9, text: '                dist[v] = dist[u]
    { line: 10, text: '                pq.push((dist[v]
    { line: 11, text: '    return dist // Все кратчайши
};

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999
  const visited: Record<string, boolean> = { A: false
  const prev: Record<string, string | null> = { A: nu

```

```

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пи
    activeCodeLine: 2,
  });

```

```

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }
}

```

```

  if (!u) break;

```

```

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,

```

```

    setSteps(simulationSteps);
  }, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border"
    /* Шапка симулятора */
    <div className="flex flex-wrap items-center justify-between"
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white"
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-indigo-300">Полный
          </p>
        </div>
      </div>

      <div className="flex items-center gap-2">
        <button
          onClick={() => { setIsPlaying(false); setCu
          className="flex items-center gap-1 bg-slate
          ion-colors"

```



```

    </marker>
    <marker id="arrow-dh-opt" markerWidth="6"
      <path d="M0,0 L6,3 L0,6 z" fill="#10b98"
    </marker>
  </defs>
  {/* Пёбра */}
  {edges.map((edge, idx) => {
    const uNode = nodes.find((n) => n.id === edge.u)
    const vNode = nodes.find((n) => n.id === edge.v)
    const isChecking =
      currentStep.checkingEdge &&
      (currentStep.checkingEdge.u === edge.u &&
        currentStep.checkingEdge.v === edge.v)

    const isOptimal = currentStep.previous[edge.id] === 'optimal'

    let marker = 'url(#arrow-dh-normal)';
    if (isChecking) marker = 'url(#arrow-dh-check)';
    else if (isOptimal) marker = 'url(#arrow-dh-opt)';

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecking ? '#f59e0b' : isOptimal ? '#1e293b' : '#f87192'}
          strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
          markerEnd={marker}
          className="transition-all duration-300"
          strokeDasharray={isChecking ? '4,4' : undefined}
        />
        <circle
          cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
          r="12" fill="#1e293b"
          stroke={isChecking ? '#f59e0b' : '#f87192'}
          strokeWidth={isChecking ? 2 : 1}
        />
        <text
          x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2) + 10}
          fill={isChecking ? '#f59e0b' : '#f87192'}
          stroke="none"
          fontSize="12"
          fontStyle="italic"
          fontWeight="bold"
        />
      </g>
    )
  })}
```

```

<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/60 p-4 rounded-2xl">
  <h4 className="text-lg font-bold text-emerald-400">
  <div className="space-y-2.5 flex-1 overflow-y-hidden">
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
        }`} >
          <div className="flex items-center gap-2">
            <span className={`px-2 py-0.5 rounded`}>
              {u}
            </span>
            <span className="text-slate-400">→</span>
          </div>
          <div className="flex flex-wrap gap-2">
            {adjList[u].map((edge, idx) => {
              const isCheckingThis = currentStep.currentNode === edge.v
              return (
                <span key={idx} className={`px-2 py-0.5 rounded ${
                  isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
                }`} >
                  {edge.v}
                </span>
                <span className="text-slate-400">→</span>
                <span className={isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'}>
                  {edge.w}
                </span>
              )
            })}
          </div>
        </div>
      )
    })}
  </div>
</div>

```

```

                {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-slate-600"
                {prev || '-'}
            </td>
        </tr>
    </tbody>
</table>
</div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60"
    <div>
        <h4 className="text-lg font-bold text-slate-400">Алгоритм
        <div className="bg-slate-950/80 rounded-lg p-4"
            {codeLines.map((item) => {
                const isActive = currentStep.activeCodeLine === item.line
                return (
                    <div key={item.line} className={`py-1 px-2
                        ${isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'}>
                        <span className="text-slate-600 select-none">{item.line}</span>
                        <span className="whitespace-pre font-family: monospace">{item.code}</span>
                    </div>
                )
            })}
        </div>
    </div>
</div>
</div>
</div>
</div>
);

```

```

const generatedSteps: DPStep[] = [];
let stepId = 0;
const memoMap: { [key: number]: number } = {};

function simulateFib(n: number): number {
    generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibМемо(${n}). Проверяем наличие в`,
        codeLine: 2,
        memoState: { ...memoMap }
    });

    if (n in memoMap) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'memo_hit',
            desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
            codeLine: 2,
            memoState: { ...memoMap }
        });
        return memoMap[n];
    }

    if (n <= 2) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'base_case',
            desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
            codeLine: 3,
            memoState: { ...memoMap }
        });
        return 1;
    }
}

```

```

    setIsPlaying(false);
  }, [targetN]));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800" >
    {/* Header & Controls */}
    <div className="flex flex-col lg:flex-row lg:items-center" >
      <div>
        <div className="flex items-center gap-3 mb-2" >
          <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-white" >
            Результаты
          </h3>
          <p className="text-sm text-slate-400" >
            Смотри, как таблица DP кэширует результаты
          </p>
        </div>
        <div>
          <div className="flex flex-wrap items-center gap-2" >
            <div className="flex items-center gap-2 bg-slate-800 p-2" >
              <span>N:</span>
              <select>

```

```

        </button>
        <button
          onClick={() => { setIsPlaying(false); set
          disabled={currentStepIndex === steps.length
          className="p-2 text-slate-400 hover:text-
          title="Шаг вперед"
        >
          <ChevronRight className="w-5 h-5" />
        </button>
      </div>
    </div>
  </div>

  {/* DP Table View */}
  <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded
    <h4 className="text-xs font-bold uppercase trac
    <div className="flex flex-wrap items-center gap
      {Array.from({ length: targetN }, (_, i) => i
      const isCached = num in memoState || num <=
      const val = num <= 2 ? 1 : memoState[num];
      const isCurrent = currentStep?.n === num;

      return (
        <div
          key={num}
          className={`flex flex-col items-center p
            isCurrent
              ? 'bg-emerald-950 border-emerald-500
              : isCached
              ? 'bg-slate-900 border-emerald-500/
              : 'bg-slate-900/50 border-slate-800
            `}
        >
          <span className="text-xs text-slate-400
          <span className={`text-xl md:text-2xl f
            {isCached ? val : '0'}
          </span>
          {num <= 2 && <span className="text-[9px

```

```

    >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between" >
  {activeTab === 'table' && (
    <div className="bg-slate-950 p-6 rounded-2xl border" >
      <h5 className="font-bold text-white text-sm" >
        <span className="text-emerald-400" > </span>
      </h5>
      <p className="text-sm text-slate-400" >
        В классической рекурсии для  $N=\{targetN\}$  по
        вычисленное значение в таблице, мы момента
      </p>
      <div className="p-4 bg-slate-900 rounded-xl border" >
        <span>Всего шагов симуляции: {steps.length}</span>
        <span className="text-emerald-400 font-bold" >
      </div>
    </div>
  )}

  {activeTab === 'code' && (
    <div className="bg-slate-950 rounded-2xl border" >
      <div className="bg-slate-900 px-6 py-3 border" >
        <span className="text-xs font-bold font-mono" >
        <span className="text-xs text-emerald-400" >
      </div>
      <pre className="p-6 font-mono text-sm space-pre" >
        {DP_CODE_LINES.map((line, idx) => {
          const lineNum = idx + 1;
          const isActive = currentStep?.codeLine === lineNum;
          return (
            <div
              key={lineNum}

```

ФАЙЛ 36/82: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину");
  const [c1Done, setC1Done] = useState(false);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кликни на вершину");
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы пройти все");
    }
  };
}
```



```

    old border border-slate-700 transition-all">
    <Undo className="h-3.5 w-3.5" /> Сброс
  </button>
</div>

<div className="bg-slate-950 rounded-xl p-4 border
  -hidden">
  <div className="absolute inset-0 bg-[radial-gra
  <svg className="w-full h-[260px] z-10 relative"
    {C1_EDGES.map((e, i) => {
      const u = C1_NODES.find(n => n.id === e.u)!
      const v = C1_NODES.find(n => n.id === e.v)!
      const key = `${Math.min(e.u,e.v)}-${Math.ma
      const used = c1VisitedEdges.includes(key);
      return <line key={i} x1={u.x} y1={u.y} x2={
        stroke={used ? "#6366f1" : "#475569"}
        strokeWidth={used ? 4 : 2}
        className="transition-all duration-300" />
    })}
    {C1_NODES.map(n => {
      const isLast = c1Path[c1Path.length-1] === n
      const visited = c1Path.includes(n.id);
      return <g key={n.id} className="cursor-poin
        {isLast && <circle cx={n.x} cy={n.y} r={1
        <circle cx={n.x} cy={n.y} r={11}
          fill={isLast ? "#6366f1" : visited ? "#
          stroke={isLast ? "#fff" : visited ? "#6
          strokeWidth={2.5}
          className="transition-all duration-200
        <text x={n.x} y={n.y+4} textAnchor="middl
        bel}</text>
      </g>;
    })}
  </svg>
  <div className="absolute top-2 left-2 bg-slate-
    Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.leng
  </div>
</div>

```

```

        'Мы перекрашиваем их в синий цвет и делим граф по
        'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе полу
        'Мы вызываем алгоритм Дейкстры для поиска нового
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане
        Это нарушит структуру дерева.'
},
{
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор
    options: [
        'Потому что он написан только для положительных ч
        'Он жадный и считает, что каждый следующий шаг де
            закрытые вершины.',
        'Отрицательные ребра создают гравитационный коллапс
        'Дейкстра просто не любил отрицательные балансы н
    ],
    correctAnswer: 1,
    explanation: 'Именно! Дейкстра уверен: если ты прие
        ля чего нужен алгоритм Беллмана-Форда.'
},
{
    id: 3,
    question: 'Какова ключевая фишка алгоритма Борувки
    options: [
        'Он использует меньше памяти благодаря коммунисти
        'Он умеет работать с отрицательными весами без ци
        'Он позволяет выполнять шаги слияния колхозов (ко
        'Он был разработан в Токио и поэтому самый быстры
    ],
    correctAnswer: 2,
    explanation: 'Абсолютно верно! В Борувке каждая ком
        лелить вычисления на GPU или многопроцессорных кл
},
{
    id: 4,
    question: 'В чем главная суть Динамического програм

```

```

        setScore(prev => prev + 1);
    }
};

const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
        setCurrentQIndex(prev => prev + 1);
        setSelectedOption(null);
        setIsAnswered(false);
    } else {
        setShowResults(true);
    }
};

const handleRestart = () => {
    setCurrentQIndex(0);
    setSelectedOption(null);
    setIsAnswered(false);
    setScore(0);
    setShowResults(false);
};

if (showResults) {
    return (
        <div className="bg-slate-900/90 border border-slate-800 p-12">
            <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
                <Award className="w-10 h-10 text-white" />
            </div>
            <h3 className="text-3xl font-bold text-white mb-4">Поздравляем!</h3>
            <p className="text-slate-400 text-sm mb-6">Вы ответили на все вопросы правильно!</p>

            <div className="bg-slate-950 border border-slate-800 p-12">
                <p className="text-slate-400 text-sm uppercase">Ваш результат</p>
                <p className="text-5xl font-black text-indigo-500">100%</p>
                <p className="text-slate-300 text-sm">
                    {score === quizQuestions.length ? "Поздравляем!" : "Вы ответили на все вопросы правильно!"}
                </p>
            </div>
        </div>
    );
}

```

```

<div className="space-y-4">
  {currentQ.options.map((option, idx) => {
    const isSelected = selectedOption === idx;
    const isCorrect = idx === currentQ.correctAnswer

    let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90"
    if (isAnswered) {
      if (isCorrect) {
        optionStyle = "bg-emerald-950/80 border-emerald-500"
      } else if (isSelected) {
        optionStyle = "bg-rose-950/80 border-rose-500"
      } else {
        optionStyle = "bg-slate-800/40 border-slate-700"
      }
    }

    return (
      <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4 rounded-lg"}
      >
        <div className="mt-0.5 shrink-0">
          {isAnswered && isCorrect ? (
            <CheckCircle2 className="w-6 h-6 text-emerald-500" />
          ) : isAnswered && isSelected && !isCorrect ? (
            <XCircled className="w-6 h-6 text-rose-500" />
          ) : (
            <div className={"w-6 h-6 rounded-full"}
              order-indigo-500 bg-indigo-500 text-white"
              {String.fromCharCode(65 + idx)}
            </div>
          )}
        </div>
        <div className="flex-1 text-sm md:text-base">
          {option}
        </div>
      </div>
    )
  })}
</div>

```

```
import { useState, useEffect } from 'react';

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

  const initialMatrix = [
    [0, 4, 999, 2, 999, 999, 999],
    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
    [999, 1, 999, 999, 999, 999, 0]
  ];
}
```

```

    for (let i = 0; i < N; i++) {
      for (let j = 0; j < N; j++) {
        if (i === j || i === k || j === k) continue;

        const oldVal = dist[i][j];
        const viaVal = dist[i][k] + dist[k][j];
        if (dist[i][k] !== 999 && dist[k][j] !== 999) {
          dist[i][j] = viaVal;
          simulationSteps.push({
            k, i, j, matrix: dist.map(r => [...r]), u
            log: `    Улучшение пути [${i}]→[${j}] чере
            ∞'.`,
            activeCodeLine: 7,
          });
        }
      }
    }
  }

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: '    Все пары отработаны. Алгоритм Флойда завер
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => setCurrentStepIndex((p)
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
})

```

```

<svg className="w-full h-auto max-h-[500px]"
  <defs>
    <marker id="arrow-fw-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
    <marker id="arrow-fw-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
    <marker id="arrow-fw-via" markerWidth="6"
    th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
  </defs>
  {Object.keys(adjList).flatMap((uStr) => {
    const u = parseInt(uStr);
    return adjList[u].map((edge) => {
      const uNode = nodesSvg.find((n) => n.id
      const vNode = nodesSvg.find((n) => n.id
      const isTarget = currentStep.i === u &&
      const isVia = (currentStep.i === u && c
      let strokeColor = '#475569'; let marker
      if (isTarget) { strokeColor = '#10b981'
      else if (isVia) { strokeColor = '#818cf8
      return (
        <g key={` ${u} - ${edge.v} `}>
          <line x1={uNode.x} y1={uNode.y} x2=
          markerEnd={marker} strokeDasharray={isTarg
          <circle cx={({uNode.x + vNode.x} / 2
          isVia ? '#818cf8' : '#64748b'} strokeWidth
          <text x={({uNode.x + vNode.x} / 2} y=
          8fafc'} fontSize="10" fontWeight="bold" tex
        </g>
      );
    });
  });
})}
  {nodesSvg.map((node) => {
    const isK = currentStep.k === node.id;
    const isI = currentStep.i === node.id;
    const isJ = currentStep.j === node.id;
    let fill = '#1e293b'; let stroke = '#64748b';
    if (isK) { fill = '#4f46e5'; stroke = '#a
    else if (isI) { fill = '#b45309'; stroke

```

```

        return (
            <span key={idx} className={`px-2
dow' : isVia ? 'bg-indigo-500 text-white bo
            {edge.v} ({edge.w})
            </span>
        )
    }}}}
</div>
);
}})
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
    /* Матрица расстояний */
    <div className="w-full lg:w-1/2 bg-rose-950/10">
        <div className="flex justify-between items-center">
            <h4 className="text-lg font-bold text-rose-950">
            </div>
            <div className="overflow-x-auto">
                <table className="w-full text-center border">
                    <thead><tr className="bg-rose-950/40 text-white">
                        <th className="p-2 border-r border-rose-950">
                        {nodeNames.map((n, idx) => <th key={idx}
-400 font-bold' : ''}`}>{n.split(' ')[0]} {
                        </tr></thead>
                        <tbody className="text-xs font-mono">
                            {currentStep.matrix.map((row, i) => (
                                <tr key={i} className="border-b borde
                                    <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`}>
                                    {nodeNames[i].split(' ')[0]} {i}
                                </td>
                                {row.map((val, j) => {
                                    const isUpd = currentStep.i === i
                                    const isVia = currentStep.k !== -
                                    tep.j === j});

```



```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};
```

```
type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
};
```

```

    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начало обхода из узла ${start}`,
      ray.from(vis)} });

    while (q.length > 0) {
      const v = q[0];
      steps.push({ type: 'process', node: v, desc: `Действие над узлом ${v}`,
        q, visitedNodes: [...Array.from(vis)] });
      q.shift();

      for (const u of adj[v]) {
        if (!vis.has(u)) {
          vis.add(u);
          q.push(u);
          steps.push({ type: 'visit', node: u, desc: `Посещение узла ${u}`,
            tack: [...q], visitedNodes: [...Array.from(vis)] });
        }
      }
    }

    steps.push({ type: 'backtrack', node: '', desc: `Очищение стека`,
      om(vis)} });

    return steps;
  }
}

```

```

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps() : generateBfsSteps(), [mode]);

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [mode]);
}

```

```

    <div className="bg-slate-900 border border-amber-500 min-h-[300px]">
      <svg className="w-full h-full max-w-[1000px] max-h-[1000px]">
        {/* Edges */}
        {edges.map((e, i) => {
          const u = nodes.find(n => n.id === e.u)
          const v = nodes.find(n => n.id === e.v)
          // Check if edge is part of the current path
          const edgeTraversed = isVisited(e.u, e.v, currentPath)

          return (
            <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
              className={edgeTraversed ? 'stroke-slate-500' : 'stroke-emerald-500'} />
          )
        })}

        {/* Nodes */}
        {nodes.map(n => {
          const visited = isVisited(n.id, currentPath)
          const current = isCurrent(n.id, currentPath)

          let fillColor = "fill-slate-900"
          let strokeColor = "stroke-slate-500"
          let textColor = "fill-slate-900"

          let radius = 18;

          if (visited) {
            fillColor = mode === 'dark' ? 'fill-slate-900' : 'fill-amber-500'
            strokeColor = mode === 'dark' ? 'stroke-slate-500' : 'stroke-amber-500'
            textColor = "fill-white"
          }

          if (current && step.type !== 'edge') {
            strokeColor = "stroke-amber-500"
            textColor = "fill-amber-500"
            radius = 22;
          }

          return (
            <div key={n.id} className="absolute" style={{
              left: n.x,
              top: n.y,
              width: 36,
              height: 36,
              border: `2px solid ${strokeColor}`,
              border-radius: radius,
              background: fillColor,
              color: textColor,
              display: flex,
              align-items: center,
              justify-content: center,
              font-size: 12,
              font-weight: bold,
            }}>
              {n.id}
            </div>
          )
        })}
      </svg>
    </div>
  )
}

```

```

        Узел: {item}
        {idx === (mode === 'dfs'
          <span className="ml-1"
        )}
      </div>
    )}
  {(!step.queueOrStack || step.queue
    <div className="text-slate-500"
  )}
</div>
</div>
</div>

<div className="flex flex-col sm:flex-row gap-4"
  {/* Controls */}
  <div className="flex bg-slate-900 border-1 border-slate-800 p-2 text-slate-400 hover:text-white hover:bg-slate-800"
    <Undo strokeWidth={3} size={16} />
    <button onClick={() => setAutoPlay(
      justify-center min-w-[80px] ${mode === 'dfs'
    }
    {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
    </button>
    <button onClick={() => {setAutoPlay(
      = steps.length - 1} className="p-2 text-slate-400 hover:bg-transparent">
      <SkipForward strokeWidth={3} size={16} />
    </button>
    <button onClick={() => {setAutoPlay(
      g-slate-800 rounded-lg ml-2 border-1 border-slate-800
      <RefreshCw strokeWidth={3} size={16} />
    </button>
  </div>

  {/* Description Log */}
  <div className="flex-1 bg-slate-900/50 border-1 border-slate-800 p-2 text-slate-400"

```

```

        idx = parent;
    } else break;
}
return a;
}

function siftDown(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    const n = a.length;
    let idx = 0;
    while (true) {
        let largest = idx;
        const l = 2 * idx + 1;
        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) largest = l;
        if (r < n && a[r].priority > a[largest].priority) largest = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
    return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
    const level = Math.floor(Math.log2(index + 1));
    const posInLevel = index - (Math.pow(2, level) - 1);
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
    return { x, y };
}

```

```
// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback((name: string, priority: number) => {
  if (busy || heap.length >= 15) {
    addLog('⚠ Все палаты переполнены! Дождитесь выписки');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);

  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
  const finalName = name || preset.name.split(' ')[0];
  const finalEmoji = preset.emoji;
  const finalSymptom = preset.symptom;

  const newPatient: Patient = {
    id: `p${uid++}`,
    name: finalName,
    emoji: finalEmoji,
    symptom: finalSymptom,
    priority,
    animState: 'in',
  };

  const nextHeap = heapInsert(heap, newPatient).map(x => x);
  // Mark specifically this new patient as 'in' in the heap
  const target = nextHeap.find(x => x.priority === priority);
  const marked = nextHeap.map(x => x.id === (target ? target.id : ''));

  setHeap(marked);
  addLog(`Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

  setTimeout(() => {
    setHeap(nextHeap);
    setBusy(false);
  }, 500);
}, [busy, heap]);
```

```

        setHeap(prev => {
          if (prev.length === 0) return [];
          const a = prev.map(x => ({ ...x }));
          a[0] = { ...a[a.length - 1] };
          a.pop();
          return siftDown(a).map(x => ({ ...x, animStat: 1 }));
        });

    // Patient gets healed in the bed
    setTimeout(() => {
      setHealing(prev => prev ? { ...prev, animStat: 1 } : null);
      addLog(` Успех! Пациент ${topPatient.name} поправился.`);
      setTimeout(() => {
        setHealing(null);
        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 1 })));
  setLog([' База данных скорой помощи перезагружена.']);
  setHealing(null);
  setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 1 }))), 100);
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < heap.length) {
    const lp = nodePos(l);
    res.push(

```

```

<div className="flex flex-col lg:flex-row items-s

    {/* Добавить случайного */}
    <button
      onClick={handleRandomInsert}
      disabled={busy || heap.length >= 15}
      className="flex-1 min-w-[150px] bg-gradient-to
        opacity-40 disabled:cursor-not-allowed text
        ve:scale-95 transition-all cursor-pointer f
    >
      <span> Привезти по Скорой (INSERT)</span>
    </button>

    {/* Добавить кастомного */}
    <form onSubmit={handleCustomInsert} className="
      <input
        type="text"
        required
        value={customName}
        onChange={e => setCustomName(e.target.value
        placeholder="Имя пациента"
        className="flex-1 bg-slate-800 border borde
        er-emerald-500 transition-colors placeholde
      />
    <div className="flex flex-col items-center mi
      <span className="text-[9px] font-mono text-
      <input
        type="range"
        min="10"
        max="99"
        value={customPriority}
        onChange={e => setCustomPriority(Number(e
        className="w-20 accent-emerald-500 cursor
      />
    </div>
    <button
      type="submit"
      disabled={busy || !customName.trim() || hea

```



```

<svg className="absolute inset-0 w-full h-full"
  {edges}
</svg>

```

```

{heap.map((patient, idx) => {
  const pos = nodePos(idx);
  const isRoot = idx === 0;
  return (
    <div
      key={patient.id}
      className={`
        absolute flex flex-col items-center
        -translate-x-1/2 -translate-y-1/2 t
        ${patient.animState === 'in' ? 'anim
        ${patient.animState === 'winner' ?
        ${patient.animState === 'out' ? 'an
        ${isRoot ? 'bg-rose-950/80 border-r
      `}
      style={{
        left: `${pos.x}%`,
        top: `${pos.y}%`,
        width: isRoot ? 60 : 52,
        height: isRoot ? 60 : 52,
      }}
    >
      <span className="text-2xl leading-non
      <span className="text-[10px] font-mono
        {patient.priority}%
      </span>

      {/* Tooltip */}
      <div className="absolute bottom-full r
        <div className="bg-slate-950 border
0 shadow-xl">
        <div className="font-bold text-wh
        <div className="text-emerald-400"
        </div>
      </div>
    </div>
  )
})
}

```

```

        <p className="text-xs font-bold font-mono">
        <p className="text-[10px] mt-1">Ждём са
      </div>
    })
  </div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => {
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-emerald-5
        <span>{entry}</span>
      </div>
    })}
  </div>
</div>
);
};

```

ФАЙЛ 41/82: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, use
```

```

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;

```

```

    bad: "#f43f5e",
    edge: "#475569",
  };

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
  <div className="w-full">
    <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
      {children}
    </svg>
    {caption && (
      <div className="text-[10px] text-slate-400 text-center">
        {caption}
      </div>
    )}
    <div className="text-[9px] text-slate-600 text-center">
      {paused ? "пауза — курсор на карточке" : "наведи"}
    </div>
  </div>
);
};

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </circle>
  </g>
);

```

```

return (
  <Svg
    caption={
      step === 0
        ? "k ещё запрещена: путь  $i \rightarrow j = 9$ "
        : step === 1
          ? "Разрешаем пересадку через k:  $3 + 4 = 7$ "
          : " $7 < 9 \rightarrow D[i][j] = 7$ "
    }
  >
    <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
    <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
  </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```
export const ComponentsDemo: React.FC = () => {
```

```
  const comps: [string, string][][] = [
    [{"A", "B"}, {"B", "C"}],
    [{"D", "E"}],
    [{"F", "G"}, {"G", "H"}, {"F", "H"}],
  ];

```

```
};
```

```

>
    {edges.map(([u, v, w], i) => {
      const a = pos[u];
      const b = pos[v];
      return (
        <g key={i}>
          <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
            {step >= 2 && (
              <text
                x={({a[0] + b[0]) / 2}
                y={({a[1] + b[1]) / 2 - 4}
                textAnchor="middle"
                fontSize={10}
                fontWeight="700"
                fill={C.warn}
              >
                {w}
              </text>
            )}
          </g>
        );
      )}
    );
  }
  {Object.entries(pos).map(([k, [x, y]]) => (
    <Node key={k} x={x} y={y} label={k} fill={C.idle} />
  ))}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (

```

```

    });
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#2ecc71";

  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => (
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      ))}
      {Object.entries(f.pos).map(([id, [x, y]]) => (
        <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
          <circle r={13} fill={color(id)} stroke={C.idl} />
          <text textAnchor="middle" dominantBaseline="middle">
            {id}
          </text>
        </g>
      ))}
    </Svg>
  );
};

```

```
);
```

ФАЙЛ 43/82: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}})
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
      }}})
    </Svg>
  );
};

export const DfsDemo: React.FC = () => {
```

```

        <rect x={n.x - 34} y={n.y - 12} width={68} h
        " }} />
        <text x={n.x} y={n.y} textAnchor="middle" d
        {idx >= 0 && <circle cx={n.x + 30} cy={n.y
        {idx >= 0 && {
            <text x={n.x + 30} y={n.y - 12} textAncho
            x + 1}</text>
        }}
    </g>
  );
  }}}
  <text x={224} y={62} textAnchor="middle" fontSize
  <text x={224} y={74} textAnchor="middle" fontSize
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пыт
    <Edge a={[60, 70]} b={[200, 70]} color={improved
    <text x={130} y={54} textAnchor="middle" fontSize
    <Node x={60} y={70} r={20} fill={C.active} label=
    <Node x={200} y={70} r={20} fill={improved ? C.do
    <text x={60} y={106} textAnchor="middle" fontSize
    <text x={200} y={106} textAnchor="middle" fontSiz
    <text x={130} y={22} textAnchor="middle" fontSize
    </Svg>
  );
};

```

```

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };

```



```

    });
};

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100],
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без ц`>
      {edges.map((e, i) => {
        const on = taken.has(`${e.u}-${e.v}`);
        return (
          <g key={i}>
            <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
            <text x={(pos[e.u][0] + pos[e.v][0]) / 2} y={pos[e.v][1]}
              textAnchor="middle" fontSize={9} fontWeight="bold" />
          </g>
        );
      })}
      {Object.entries(pos).map(([k, [x, y]]) => <Node
    </Svg>
  );
};

```

```

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45],
  };
  const edges: [string, string][] = [
    ["A", "B"], ["B", "C"], ["C", "D"], ["D", "A"],
  ];
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];

```

```
export const StackDemo: React.FC = () => {
  const states = [["A"], ["A", "B"], ["A", "B", "C"], [
  const step = useLoop(states.length, 800);
  const items = states[step];
  const popping = step >= 3;
  return (
    <Svg caption={popping ? "pop: забираем ВЕРХНИЮ таре
    <rect x={90} y={22} width={80} height={96} rx={6}
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={94} y={100 - i * 26} width={72} height={26}
          fill={i === items.length - 1 ? (popping ? C
        <text x={130} y={111 - i * 26} textAnchor="mi
      </g>
    ))}
    <text x={130} y={16} textAnchor="middle" fontSize=
  </Svg>
  );
};
```

```
export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к
    <defs>
      <marker id="mdArrow" markerWidth="6" markerHeight=
        <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
      </marker>
    </defs>
    <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХ
    <text x={206} y={44} fontSize={9} fill={C.dim}>ВХ
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={30 + i * 54} y={56} width={46} height={26}
        <text x={53 + i * 54} y={71} textAnchor="midd
```

```

const step = useLoop(TRIE_NODES.length + 1, 700);
return (
  <Svg caption="Путь от корня по буквам = слово; общие  

    {TRIE_NODES.filter((n) => n.parent !== null).map(
      const p = TRIE_NODES[n.parent as number];
      return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
    ]} }
    {TRIE_NODES.map((n) => {<Node key={n.id} x={n.x} :
  </Svg>
  );
};

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? levels[step - 1] : null;
  return (
    <Svg caption={`BFS по бору: уровень ${Math.max(0, M
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
      ]} }
      {TRIE_NODES.map((n) => {
        <Node key={n.id} x={n.x} y={n.y} label={n.ch}
          fill={wave.includes(n.id) ? C.active : doneId
        })}
      <text x={6} y={12} fontSize={9} fill={C.dim}>очер
    </Svg>
  );
};

export const SuffixLinkDemo: React.FC = () => {
  const links: [number, number][] = [[1, 0], [2, 0], [3
  const step = useLoop(links.length + 1, 900);
  return (
    <Svg caption="Суффиксная ссылка: «куда откатиться,
      {TRIE_NODES.filter((n) => n.parent !== null).map(

```

```

    </Svg>
  );
};

export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
      : `о префиксные суммы P`} >
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 39} y={24} width={34} height={12}
            fill={show && i >= l && i <= r ? C.active : C.dim} />
          <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
        </g>
      ))}
      <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
      {pref.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 33} y={78} width={29} height={12}
            fill={show && i === l ? C.bad : show && i > l ? C.dim : C.idle}
            stroke={C.idleStroke} style={{ transition: "fill 0.2s" }} />
          <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
        </g>
      ))}
    </Svg>
  );
};

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);

```

```

<Svg caption={step === 0 ? "Найти решение – долго ("
  <ellipse cx={130} cy={64} rx={112} ry={48} fill="
  <ellipse cx={92} cy={64} rx={52} ry={32} fill="#3
  <text x={92} y={64} textAnchor="middle" dominantB
  <text x={206} y={34} textAnchor="middle" fontSize
  <circle cx={196} cy={84} r={13} fill={step === 1
  <text x={196} y={84} textAnchor="middle" dominant
    ?"}</text>
  <text x={130} y={124} textAnchor="middle" fontSize
</Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} hei
            fill={step > 0 && (idx < len || (idx > i -
              style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="mi
            ext>
          <text x={32 + idx * 33} y={70} textAnchor="mi
            fill={idx <= i && step > 0 ? C.warn : C.idle
          </g>
        )}}
      <text x={4} y={40} fontSize={8} fill={C.dim}>s</t
      <text x={4} y={70} fontSize={8} fill={C.dim}>π</t
      <text x={130} y={106} textAnchor="middle" fontSize
        π[i] – длина самого длинного «префикс = суффикс
      </text>
    </Svg>
  )

```

```
-----
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo
import {
  AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
  SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo
} from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components"
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
```

```
import { getViz } from "../vizRegistry";

interface Props {
  vizId: string | null;
  onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);

  const viz = getViz(vizId ?? undefined);
  if (!vizId || !viz) return null;
  const Component = viz.Component;

  return createPortal(
    <div className="fixed inset-0 z-[110] flex items-en
      <div className="absolute inset-0 bg-slate-950/80
      <div className="relative w-full sm:max-w-5xl max-l
        ded-2xl shadow-2xl flex flex-col">
          <div className="flex items-center gap-3 px-4 py
            <div className="min-w-0 flex-1">
              <h3 className="font-bold text-white text-sm
                {viz.hint && <p className="text-[11px] text
              </div>
            <button
              onClick={onClose}
```

```
    onCardLeave: () => void;
  }

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor, ...props }) => {
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>({ top: 0, left: 0 });

  useEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });
  }, [anchor]);

  useEffect(() => {
    if (!anchor) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    window.addEventListener("scroll", onClose, true);
    return () => {
      window.removeEventListener("keydown", onKey);
    };
  }, [onClose]);
};
```



```

    {entry.demo && <MiniDemo kind={entry.demo} />

    {entry.formal && (
      <p className="text-[11.5px] leading-relaxed
        {entry.formal}
      </p>
    )}

    {entry.complexity && (
      <div className="text-[11px] font-mono text-
        Сложность: {entry.complexity}
      </div>
    )}

    {viz && simulatorId && (
      <button
        onClick={() => onOpenSimulator(simulatorId)}
        className="w-full flex items-center justify-
        unded-lg py-2 transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    )}
  </div>
</div>
</div>,
document.body
);
};

```

---

ФАЙЛ 48/82: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState } from 'react';
```

```

    if (e.u === 'S') {
        if (step === 0 || step === 7) return "opacity-0";
        if (step === 1) return "stroke-amber-500 stroke-width-2px";
        if (step === 2) return "stroke-pink-500 stroke-width-2px";
        return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
    }

    if (e.id === 'AB') {
        if (step === 4) return "stroke-amber-400 stroke-width-2px";
        if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
        if (step === 5) return "stroke-amber-400 stroke-width-2px";
        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-width-2px";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // отсюда и дальше
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";
};

```

```
};

const getDesc = () => {
  switch (step) {
    case 0: return (
      <div>
        <b>Исходный граф.</b> Имеется ребро
        Если мы запустим быстрый алгоритм Д
      </div>
    );
    case 1: return (
      <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соед
ми</span>.
        Это наша база для расчета потенциал
      </div>
    );
    case 2: return (
      <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгор
ных вершин:
        <span className="text-pink-300 ml-1
      </div>
    );
    case 3: return (
      <div>
        <b>Шаг 3: Магическая формула.</b><b>
        Всем ребрам графа назначаем новый в
        <div className="text-center font-mo
        Это повышает или понижает вес ребра
      </div>
    );
    case 4: return (
      <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-r
```

```

nter p-4">
    <svg width="400" height="300" class=
        <defs>
            <marker id="arrow-slate" vie
start-reverse">
                <path d="M 0 0 L 10 5 L
            </marker>
            <marker id="arrow-slate-dim
uto-start-reverse">
                <path d="M 0 0 L 10 5 L
            </marker>
            <marker id="arrow-amber" vie
start-reverse">
                <path d="M 0 0 L 10 5 L
            </marker>
            <marker id="arrow-pink" vie
tart-reverse">
                <path d="M 0 0 L 10 5 L
            </marker>
            <marker id="arrow-emerald"
o-start-reverse">
                <path d="M 0 0 L 10 5 L
            </marker>
        </defs>

        {/* Edges */}
        {edges.map((e, i) => {
            const u = nodes.find(n => n
            const v = nodes.find(n => n

            const isS = e.u === 'S';
            if (isS && (step === 0 || s

            const midX = (u.x + v.x) / 2;
            const midY = (u.y + v.y) / 2;

            const edgeColorClass = getE
            const textHigh = isEdgeText

```

```

                                />
                                <text x={n.x} y={n.y}
- white">                                {n.label}
                                </text>

                                {pot !== null && {
                                <g transform={`
fade-in zoom-in duration-500">                                <rect x="-1
k-500 stroke-1" />                                <text x="0"
font-bold fill-pink-400">                                {pot}
                                </text>
                                </g>
                                }}
                                </g>
                                )
                                }}}
                                </svg>
                                </div>

                                <div className="w-full md:w-[320px] flex
                                { /* Log Box */
                                <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
                                {getDesc()}
                                </div>

                                { /* Controls */
                                <div className="flex bg-slate-900 border
                                <button onClick={() => setStep(1)}
                                className="p-3 bg-slate-800 border
- slate-400 transition-colors" title="Сброс
                                <RotateCcw strokeWidth={3}
                                </button>
                                <button onClick={() => setStep(1)}

```

```

    Layers,
  } from "lucide-react";

interface GNode {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
}

```

```

    };

    recordStep(
        "init",
        "Инициализация алгоритма Косарайю. Начинаем первый шаг",
        null,
        false,
        -1,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    initialEdges.forEach((e) => adj[e.u].push(e.v));

    // DFS 1: post-order list
    const dfs1 = (u: number) => {
        visited.add(u);
        recordStep(
            "dfs1",
            `DFS 1: зашли в вершину ${u}, помечаем посещение`,
            u,
            false,
            -1,
        );

        for (const to of adj[u]) {
            if (!visited.has(to)) {
                dfs1(to);
            }
        }

        order.push(u);
        recordStep(
            "dfs1",
            `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
            u,
            false,
            -1,
        );
    };

```

```

        for (const to of adjT[u]) {
            if (!visited.has(to)) {
                dfs2(to, currentScc);
            }
        }
    };

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека`,
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в`,
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
    null,
    true,
    sccId,
);

```



```

    if (id === 2) return "fill-indigo-600 stroke-indigo-600";
    return "fill-slate-800 stroke-slate-500";
};

```

```

return (
  <div className="bg-slate-900 rounded-xl border border-slate-800" >
    <div className="bg-slate-800 p-4 border-b border-slate-700" >
      <h3 className="font-bold text-white flex items-center" >
        <Layers className="w-5 h-5 text-emerald-400" />
        Визуализатор Алгоритма Косарайю (Поиск SCC)
      </h3>
    </div>

```

```

    <div className="flex items-center gap-2 bg-slate-800" >
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-2 border border-slate-700 text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

```

```

      <button
        onClick={reset}
        className="flex items-center justify-center gap-2 px-3 py-2 border border-slate-700 text-white"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

```

```

  <div className="grid grid-cols-1 lg:grid-cols-12" >
    { /* Playfield SVG */ }
  </div>

```



```

        className={`transition-all duration
strokeWidth="3"
/>
<text
  x={node.x}
  y={node.y}
  dy=".3em"
  textAnchor="middle"
  fill="white"
  fontSize="13px"
  fontWeight="bold"
  className="select-none"
>
  {node.label}
</text>

{scssId && {
  <text
    x={node.x}
    y={node.y - 28}
    textAnchor="middle"
    fill="#10b981"
    fontSize="10px"
    fontWeight="bold"
  >
    SCC #{scssId}
  </text>
}}
</g>
);
}}
</svg>

```

```

{step.transposed && {
  <div className="absolute top-4 left-4 bg-in
1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>

```

```

    {/* List of actions log */}
    <div>
      <span className="text-[10px] text-slate-500">
        ЛОГ ОПЕРАЦИЙ:
      </span>
      <div className="space-y-1.5 max-h-[150px]">
        {steps
          .slice(0, currentStepIdx + 1)
          .reverse()
          .slice(0, 5)
          .map(({s, idx}) => {
            <div
              key={idx}
              className={`text-[11px] p-1.5 rounded`
            >
              {s.desc}
            </div>
          )}}
        </div>
      </div>
    </div>

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        № {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1.5">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
        >
          Назад
        </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
        >
          Далее
        </button>
      </div>
    </div>
  </div>
)

```

```
interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'
```

```

    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
      description: 'Оно соединяет две другие бесцветные вершины',
      type: 'info'
    }, ...prev]));
  },
  // Step 4: Include D-E
  () => {
    setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро D-E в осто́ве',
      description: 'Вершины D и E окрашены в синий цвет',
      type: 'success'
    }, ...prev]));
  },
  // Step 5: Inspect B-C (4)
  () => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
      title: 'Шаг 3: Берем ребро B-C (вес 4)',
      description: 'Оно соединяет красную вершину B и синюю вершину C',
      type: 'info'
    }, ...prev]));
  },
  // Step 6: Include B-C
  () => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро B-C в осто́ве',
      description: 'Вершина C перекрашена в красный цвет',
      type: 'success'
    }, ...prev]));
  },
  // Step 7: Inspect A-D (5)
  () => {

```

```

    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
      title: 'Шаг 6: Берем ребро E-F (вес 7)',
      description: 'Оно соединяет фиолетовую вершину
      type: 'info'
    }, ...prev]));
  },
// Step 12: Include E-F
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро E-F в осто́ве',
    description: 'Вершина F окрашена в фиолетовый ц
    type: 'success'
  }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Берем ребро D-G (вес 8)',
    description: 'Оно соединяет фиолетовую вершину
    type: 'info'
  }, ...prev]));
},
// Step 14: Include D-G
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро D-G в осто́ве',
    description: 'Вершина G окрашена в фиолетовый.'
    type: 'success'
  }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {

```

```

        setLogs(prev => [{
            title: 'Шаг 10: Берем ребро E-H (вес 11)',
            description: 'Оба конца E и H уже фиолетовые. С',
            type: 'warning'
        }, ...prev]));
    },
    // Step 20: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Отказ: Цикл обнаружен!',
            description: 'Выбрасываем ребро E-H.',
            type: 'error'
        }, ...prev]));
    },
    // Step 21: Inspect H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 11: Берем ребро H-I (вес 12)',
            description: 'Соединяет фиолетовую H и последнюю',
            type: 'info'
        }, ...prev]));
    },
    // Step 22: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Успех: MST построено Краскалом!',
            description: 'Все 9 вершин окрашены в единый фи
                + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
            type: 'success'
        }, ...prev]));
    },
    ];

    // --- PRIM STEPS ---

```



```

        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону верш
        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в куч
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к верш
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-
        type: 'success'
    }, ...prev]);
},

```

```

    }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
        description: 'Плесень из центра Токио (E) тянет',
        type: 'info'
    }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
        eap' } : e));
    setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
    setLogs(prev => [{
        title: 'Успех: Вершина F захвачена',
        description: 'Ребро F-I(13) добавлено в кучу. P
        type: 'success'
    }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
        description: 'Плесень расширяется вниз к G.',
        type: 'info'
    }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
        eap' } : e));
    setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H

```

```

        ear' } : e));
    setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)']);
    setLogs(prev => [{
        title: 'Успех: Вершина H захвачена',
        description: 'В кучу добавлено H-I(12).',
        type: 'success'
    }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Куча выдает E-H (вес 11)',
        description: 'Обе вершины E и H уже в плесени.',
        type: 'warning'
    }, ...prev]));
},
// Step 21: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setHeapQueue(['H-I (12)', 'F-I (13)']);
    setLogs(prev => [{
        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 12: Куча выдает H-I (вес 12)',
        description: 'Плесень тянется к последней чистой вершине.',
        type: 'info'
    }, ...prev]));
},
// Step 23: Include H-I
() => {

```

```

() => {
  // Concurrently build chosen roads. Merge into:
  // Component 1: {A, B, C} (colored red)
  // Component 2: {D, E, F, G, H, I} (colored gold)
  setVertices(prev => prev.map(v =>
    [0, 1, 2].includes(v.id) ? { ...v, color: 'red' }
  ));
  setEdges(prev => prev.map(e => {
    if (['0-1', '1-2'].includes(e.id)) {
      return { ...e, status: 'included', color: 'red' }
    }
    if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
      return { ...e, status: 'included', color: 'blue' }
    }
    return e;
  }));
  setLogs(prev => [{
    title: 'Слияние: Деревни объединились в колхозы',
    description: 'Все выбранные дороги построены радом с колхозами {D, E, F, G, H, I}.',
    type: 'success'
  }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    status: 'included', color: 'red'
  } : e));
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост',
    description: 'Теперь Красный и Синий колхозы одобрены мостом A-D с весом 5. Остальные мосты B-E(6) и C-F(9) не выбраны.',
    type: 'warning'
  }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz

```

```

    setEdges(initialEdges);
    setStepIndex(0);
    setHeapQueue([]);
    if (algo === 'kruskal') {
        setLogs([
            {
                title: 'Введение в раскраску (Краскал)',
                description: 'Представь, что все 9 вершин графа
                type: 'info',
            },
        ]);
    } else if (algo === 'prim') {
        setLogs([
            {
                title: 'Введение в Плесень (Прима)',
                description: 'Логика «Плесени»: выбираем стар
                едь (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else {
        setLogs([
            {
                title: 'Введение в Коллективизацию (Борувка)',
                description: 'Логика «Борувки»: Каждая из 9 д
                чтобы объединиться в колхоз.',
                type: 'info',
            },
        ]);
    }
};

const getColorClass = (color: string, id: number) =>
    switch (color) {
        case 'red': return 'bg-red-500 border-red-300 tex
        case 'blue': return 'bg-blue-500 border-blue-300
        case 'purple': return 'bg-purple-600 border-purple
        case 'mold': return 'bg-emerald-500 border-emerald

```

```
</div>
```

```
{/* Algorithm Tabs */}
```

```
<div className="flex flex-wrap items-center gap-2">
```

```
  <div className="flex items-center bg-slate-100 p-2">
```

```
    <button
```

```
      onClick={() => handleReset('kruskal')}>
```

```
      className={
```

```
        "flex items-center gap-1.5 px-3 py-2"
```

```
      +
```

```
        {activeAlgo === 'kruskal' ?
```

```
          'bg-indigo-600 text-white shadow-md' :
```

```
          'text-slate-400 hover:text-slate-500'}
```

```
    </button>
```

```
  >
```

```
    <GitGraph className="w-3.5 h-3.5" />
```

```
    <span>Краскал (Билет 18)</span>
```

```
  </button>
```

```
  <button
```

```
    onClick={() => handleReset('prim')}>
```

```
    className={
```

```
      "flex items-center gap-1.5 px-3 py-2"
```

```
    +
```

```
      {activeAlgo === 'prim' ?
```

```
        'bg-emerald-600 text-white shadow-md' :
```

```
        'text-slate-400 hover:text-slate-500'}
```

```
    </button>
```

```
  >
```

```
    <Layers className="w-3.5 h-3.5" />
```

```
    <span>Прима (Билет 19)</span>
```

```
  </button>
```

```
  <button
```

```
    onClick={() => handleReset('boruvka')}>
```

```
    className={
```

```
      "flex items-center gap-1.5 px-3 py-2"
```

```
    +
```

```
      {activeAlgo === 'boruvka' ?
```

```
        'bg-rose-600 text-white shadow-md' :
```

```

<div className="grid grid-cols-1 lg:grid-cols-3
  {/* Graph Canvas */}
  <div className="lg:col-span-2 bg-slate-950 ro
    ative min-h-[480px] overflow-hidden">

    {/* Legend */}
    <div className="absolute top-4 left-4 bg-sl
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5
        <div className="w-3 h-3 rounded-full bg
      </div>
      {activeAlgo === 'kruskal' ? (
        <
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
        </>
      ) : activeAlgo === 'prim' ? (
        <
          <div className="flex items-center gap
            <div className="w-3 h-1 bg-sky-600
          </div>
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
        </>
      ) : (
        <
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
          <div className="flex items-center gap

```

```

        x={(u.x + v.x) / 2 + "%"}
        y={(u.y + v.y) / 2 + "%"}
        fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
        fontSize="4.5"
        fontFamily="monospace"
        fontWeight="bold"
        textAnchor="middle"
        dominantBaseline="middle"
        className="select-none filter drop
dy="-1.5"
    >
        {edge.weight}
    </text>
</g>
    );
  })}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: v
      className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'p
        <span className="text-[8px] block -
      }}
    </div>
  })}
</div>

{stepIndex >= currentSteps.length && (
```



```

        </div>
      )}

      <div className="flex-1 p-4 overflow-y-auto"
        {logs.map((log, idx) => {
          <div
            key={idx}
            className={
              "p-4 rounded-xl border transition-a
              (log.type === 'success'
                ? 'bg-emerald-950/40 border-emera
                : log.type === 'warning'
                ? 'bg-amber-950/40 border-amber-5
                : log.type === 'error'
                ? 'bg-rose-950/40 border-rose-500
                : 'bg-slate-900/60 border-slate-7
              )
            }
          >
            <div className="flex items-center gap
              {log.type === 'success' && <CheckCi
              {log.type === 'warning' && <HelpCir
              {log.type === 'error' && <XCircle c
              {log.type === 'info' && <Play class
              <h5 className="font-bold text-sm le
            </div>
            <p className="text-xs text-slate-300
          </div>
        )}}
      </div>
    </div>
  </div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
  <div className="flex flex-col sm:flex-row items
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-40

```

```

<div className="bg-slate-950 p-4 rounded-1
  <p className="text-slate-400 text-xs for
    <Award className="w-3.5 h-3.5 text-in
  </p>
  <p className="text-2xl font-black text-r
  <p className="text-slate-400 text-xs mt
</div>
<div className="bg-slate-950 p-4 rounded-1
  <p className="text-slate-400 text-xs for
ожении:</p>
  <p className="text-xs text-slate-300 le
ди и красим вершины в один цвет, бракуя цик
</div>
</div>
<div className="lg:col-span-2">
  <div className="bg-slate-950 p-4 rounded-1
    <p className="text-slate-400 text-xs for
      <Code className="w-3.5 h-3.5 text-ind
    </p>
    <pre className="text-xs text-emerald-400
      80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:

```

```

            80 flex-1">
`import heapq

def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}
    </pre>
  </div>
</div>
</div>
}}

{activeCheatTab === 'boruvka' && {
  <div className="grid grid-cols-1 lg:grid-cols-2" >
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-rose-500" />
        </p>
        <p className="text-2xl font-black text-white">Борувка</p>
        <p className="text-slate-400 text-xs font-medium">
          Метод Борувки — это алгоритм нахождения минимального остовного
          дерева (MST) для неориентированного взвешенного графа. Он был
          предложен в 1926 году Яном Борувкой. Алгоритм работает за
          время O(E log V), где E — количество ребер, а V — количество
          вершин.
        </p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          Алгоритм начинается с выбора произвольной вершины и
          построения минимального остовного дерева для каждой
          компоненты связности. Затем эти деревья соединяются в
          одно целое, пока не останется одно дерево.
        </p>
      </div>
    </div>
  </div>
}

```

```
        </div>

    </div>
  );
};
```

---

ФАЙЛ 51/82: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
  },
];
```

```
    </div>
  );
}
```

---

ФАЙЛ 52/82: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
  const [open, setOpen] = useState(false);

  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : "";
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  useEffect(() => {
```

```
        </button>
      </div>
      <div className="flex-1 min-h-0">
        <Sidebar
          selectedChapterId={selectedChapterId}
          setSelectedChapterId={setSelectedChapterId}
          onNavigate={() => setOpen(false)}
        />
      </div>
    </div>
  </div>
)}
</>
);
};
```

---

ФАЙЛ 53/82: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cloud } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
```

```

</button>
<button
  id="tab-simulator-btn"
  onClick={() => setActiveTab('simulator')}
  className={`flex-1 lg:flex-none flex items-
    uration-200 whitespace-nowrap ${
      activeTab === 'simulator'
        ? 'bg-indigo-600 text-white shadow-lg s
        : 'text-slate-400 hover:text-white'
    }`}
>
  <Cpu className="w-4 h-4" /> Тренажёры
</button>
<a
  id="download-pdf-btn"
  href="/export/full_code_all_pages.pdf"
  download="full_code_all_pages.pdf"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
    over:bg-emerald-600/20 border border-emeral
  title="Скачать полный PDF сборник всех стра
>
  <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
  id="download-txt-btn"
  href="/export/full_code_all_pages.txt"
  download="full_code_all_pages.txt"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
    :bg-sky-600/20 border border-sky-500/30 tra
  title="Скачать полный TXT файл со всем кодо
>
  <FileText className="w-4 h-4" /> TXT
</a>
<button

```

```

        {id:3,x:90,y:230},
        {id:4,x:50,y:100}
    ];
    const edges = [
        [0,1],[0,2],[0,3],[0,4],
        [1,2],[1,3],[1,4],
        [2,3],[2,4],
        [3,4]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
        if (a===c||a===d||b===c||b===d) return
        const p1=findPt(a),p2=findPt(b),p3=find
        function ccw(ax:number,ay:number,bx:num
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
        for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1]
                if (!crosses.includes(i)) crosses.p
                if (!crosses.includes(j)) crosses.p
            }
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(
        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x
            stroke={xing ? "#f43f5e" : "#4f46e5"}
        }
    }
    const labels = ["A","B","C","D","E"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>

```



```

    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
            if (!crosses.includes(i)) crosses.push(i);
            if (!crosses.includes(j)) crosses.push(j);
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
            stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
    }
    const labels = ["\u04141","\u04142","\u04143"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
            <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
        </g>);
    }
    return <>{lines}{circles}</>;
  })()}
</svg>
<div className="absolute bottom-2 left-2 right-2 top-2 z-20">
  V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

```

<div className="bg-amber-950/40 border border-amber-950/40 p-10">
  ▲ Запомни: K5 и K3,3 – два кита непланарности.
  гомеоморфный K5 или K3,3 – он непланарен. Теорема Кёрши.
</div>
</div>

```

```

);

```

```

const [queue, setQueue] = useState<Customer[]>([
  { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red', col: 'red',
    голода после матана!', animState: 'idle' },
  { id: 'q2', name: 'Кодер Семён', emoji: '🍝', color: 'blue', col: 'blue',
    ишу ИИ за порцию борща!', animState: 'idle' },
  { id: 'q3', name: 'Кот Борис', emoji: '🍦', color: 'green', col: 'green',
    ез сметаны, плиз.', animState: 'idle' },
]);

const [servedHistory, setServedHistory] = useState<string[]>([]);
const [isServing, setIsServing] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяжёло!');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
}, [isServing, queue]);

const preset = PRESETS[counter % PRESETS.length];
counter++;

const newGuest: Customer = {
  id: Date.now().toString(),
  ...preset,
  animState: 'in',
};

setQueue(prev => [...prev, newGuest]);
addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

```

```

counter = 3;
setQueue([
  { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', col: 1, row: 1, text: 'Я съел суп и теперь не чувствую голода после матана!', animState: 'in' },
  { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'blue', col: 2, row: 1, text: 'Я пишу ИИ за порцию борща!', animState: 'in' },
  { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'orange', col: 3, row: 1, text: 'Я сижу без сметаны, плиз.', animState: 'in' },
]);
setServedHistory([]);
setIsServing(false);
setLog(['Столовая сброшена. Снова голодная толпа!']);
setTimeout(() => {
  setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
});

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-indigo-950/40 border border-indigo-500 p-6">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
        <p className="text-xs text-slate-300 mt-1 leading-tight">
          В очереди за супом всё честно. Кто первый встал, тот и съел.
          Новые гости встают строго <b>в конец</b> очереди.
          Здесь нет обхода и блата — только строгий порядок.
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex items-center gap-3 flex-wrap">
      <button
        onClick={enqueue}
        disabled={isServing}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-medium rounded-md">
        Добавить в очередь
      </button>
    </div>
  </div>
);

```

```

    { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
    <div className="flex flex-col items-center">
      <div className="relative">
        <div className="absolute -top-10 left-10">
          <div className="w-1.5 h-6 bg-slate-400">
            <div className="w-2 h-4 bg-slate-400">
            </div>
          </div>
          <span className="text-5xl block animate-pulse">
          </span>
        </div>
        <div className="text-center">
          <span className="text-3xl block">
          </span>
          <span className="text-[10px] font-bold font-mono tracking-wider">
            ПОВАР
          </span>
        </div>
      </div>
    </div>

    { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
    <div className="flex flex-col items-center">
      <span className="text-2xl animate-pulse">
      </span>
      <span className="text-[8px] font-mono tracking-wider">
      </span>
    </div>

    { /* СПИСОК ОЧЕРЕДИ */ }
    <div className="flex-1 flex items-end gap-4">
      { queue.length === 0 ? (
        <div className="flex-1 flex flex-col">
          <span className="text-4xl mb-1">
          </span>
          <p className="text-xs font-bold font-mono">
          </p>
          <p className="text-[10px]">Все сыты
          </p>
        </div>
      ) : (
        queue.map((customer, idx) => {
          const isHead = idx === 0;
          const isTail = idx === queue.length - 1;
          return (
            <div

```

```

        </div>
      </div>
    );
  })
  })
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 round
  <div className="absolute top-4 left-4 text-[1
    Сытые гости (Архив FIFO)
  </div>

  <div className="absolute top-4 right-4 flex i
    order-emerald-800/80 text-[9px] font-mono">
    <Sparkles className="w-3.5 h-3.5 animate-pu
    <span>СЫТЫ </span>
  </div>

  <div className="flex-1 flex flex-col justify-
    {servedHistory.length === 0 ? (
      <div className="flex-1 flex flex-col item
        <span className="text-5xl mb-2"> </span>
        <p className="text-xs font-bold font-mo
        <p className="text-[10px] mt-1">Здесь 6
      </div>
    ) : (
      <div className="flex flex-col gap-1.5 w-fr
        {servedHistory.map((item, idx) => (
          <div
            key={idx}

```

```

-----
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR', 'FISH', 'SALMON']);
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: TrieNode }>({});
  const [simulationStep, setSimulationStep] = useState<number>(0);
  const [bfsQueue, setBfsQueue] = useState<number[]>([]);
  const [currentNode, setCurrentNode] = useState<number>(0);
  const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и попробуй!');
  const [isTalking, setIsTalking] = useState<boolean>(false);
  const [isBlinking, setIsBlinking] = useState<boolean>(false);

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);
};

```

```

        newNodes[curr].words.push(word);
        newNodes[curr].is_terminal = true;
    }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
            if (leftX === -1) leftX = newNodes[v].x!;
            rightX = newNodes[v].x!;
        });
        newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX / 2;
        if (leftX === -1) currentX++;
    }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
    `Отлично! Я построил базовое префиксное дерево (БПД) для слова "S", чтобы расставить fail-ссылки!`
);

```

```

    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на графике  

        уже суффикса просто нет. Жми «Следующий шаг»!');
    });
};

// Один шаг BFS
const stepBFS = () => {
    if (bfsQueue.length === 0) {
        setSimulationStep(2);
        setCurrentNode(null);
        setSpeechText(
            'Ура! Очередь пуста! Мы успешно построили полный  

            граф и полюбоваться таблицей переходов!');
    };
    return;
}

const r = bfsQueue[0];
const newQueue = bfsQueue.slice(1);
const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.children})`;

for (const [char, u] of Object.entries(rNode.children)) {
    newQueue.push(u);
    let v = rNode.fail;
    while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
        v = updatedNodes[v].fail;
    }
}

```



```

<div className="flex items-center gap-2">
  <button
    onClick={() => buildInitialTrie(words)}
    className="flex items-center gap-1 bg-slate
      transition-colors"
    title="Сбросить автомат"
  >
    <RotateCcw className="w-4 h-4" /> Сбросить
  </button>
</div>
</div>

```

```

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center just
    <div className="w-40 h-40 relative flex items
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
    <svg
      viewBox="0 0 200 200"
      className={`w-36 h-36 transition-transform
        isTalking ? 'scale-105 drop-shadow-[0_0
        ]`}
    >
      {/* Тело лосося */}
      <ellipse cx="100" cy="100" rx="70" ry="45"
      <path d="М 40 85 Q 100 55 160 85 Q 100 115
        40 85"
        fill="#be123c"
        className="origin-[35px_100px] animate-
      />

      {/* Плавники */}
      <path d="М 90 55 L 70 30 L 110 55 Z" fill=

```

```

    {/* Пузырь с текстом (Баббл) */}
    <div className="md:col-span-2 bg-slate-800 border-2 border-transparent
      tween min-h-[140px]">
      {/* Треугольник баббла */}
      <div className="absolute -left-3 top-1/2 -transform rotate(180deg)
        slate-800 border-b-[12px] border-b-transparent" />

      <div className="flex items-center space-x-2 text-white">
        <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
        <span>Прямое вещание из аквариума:</span>
      </div>

      <p className="text-slate-200 text-base leading-relaxed">
        {speechText}
      </p>

      <div className="mt-4 pt-3 border-t border-slate-700">
        <div className="text-xs text-slate-400 flex justify-between">
          <Info className="w-3.5 h-3.5 text-blue-400" />
          Статус: {simulationStep === 0 ? 'Бор готов к запуску!' : 'Запущено!'}
        </div>

        <div className="flex gap-2">
          {simulationStep === 0 && (
            <button
              onClick={startBFS}
              className="bg-emerald-600 hover:bg-emerald-500 text-white px-3 py-1 shadow-lg shadow-emerald-900/40 transition" />
            <Play className="w-4 h-4" /> Запустить
          )}
          {simulationStep === 1 && (
            <button
              onClick={stepBFS}
              className="bg-blue-600 hover:bg-blue-500 text-white px-3 py-1 shadow-lg shadow-blue-900/40 transition" />
            Шаг
          )}
        </div>
      </div>
    </div>
  )}
  </div>
</div>

```

```

<defs>
  <marker id="arrowhead" markerWidth="6"
    <polygon points="0 0, 6 3, 0 6" fill=
  </marker>
  <marker id="fail-arrow" markerWidth="6"
    <polygon points="0 0, 6 3, 0 6" fill=
  </marker>
</defs>

{/* Draw Edges */}
{Object.values(nodes).map(node => {
  if (node.id === 0) return null;
  const parent = nodes[node.parent];
  if (!parent.x || !parent.y || !node.x || !node.y) return null;

  const isCurrent = currentNode === node.id;

  return (
    <g key={`edge-${node.id}`}>
      <line
        x1={parent.x}
        y1={parent.y + 16}
        x2={node.x}
        y2={node.y - 16}
        stroke={isCurrent ? '#60a5fa' : '#ccc'}
        strokeWidth="2"
        markerEnd="url(#arrowhead)"
      />
      <text
        x={({parent.x + node.x} / 2 - 10)}
        y={({parent.y + node.y} / 2)}
        fill="#94a3b8"
        fontSize="12"
        fontWeight="bold"
      >
        {node.char}
      </text>
    </g>
  )
})}

```

```

    let stroke = '#475569';

    if (isCurrent) {
      fill = '#2563eb';
      stroke = '#60a5fa';
    } else if (isInQueue) {
      fill = '#b45309';
      stroke = '#fbbf24';
    } else if (node.is_terminal) {
      fill = '#059669';
      stroke = '#34d399';
    }

    return (
      <g key={`node-${node.id}`}>
        <circle
          cx={node.x}
          cy={node.y}
          r="16"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
        />
        <text
          x={node.x}
          y={node.y + 4}
          fill="white"
          fontSize={isRoot ? "10" : "12"}
          fontWeight="bold"
          textAnchor="middle"
        >
          {isRoot ? 'R' : node.char}
        </text>
        {!isRoot && (
          <text
            x={node.x + 12}
            y={node.y - 12}
            fill="#94a3b8"

```

```
</div>
```

```
<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600 transition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>
```

```
{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
  <h5 className="text-white font-bold mb-3 flex items-center gap-1.5" >
    <span> </span> Таблица переходов и fail-ссылок
    <span className="text-xs text-slate-400 font-normal" >
      Всего вершин: {Object.keys(nodes).length}
    </span>
  </h5>
```

```
<div className="max-h-[220px] overflow-y-auto" >
  <table className="w-full text-left border-collapse" >
    <thead>
      <tr className="border-b border-slate-700" >
        <th className="py-2 px-3" >ID Вершины<
```

```

</td>
<td className="py-2.5 px-3">
  <span className="bg-slate-800 b
    {node.char}
  </span>
</td>
<td className="py-2.5 px-3 text-s
  {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
  {node.id === 0 ? {
    <span className="text-slate-5
  } : {
    <span className="bg-indigo-95
      #{node.fail}
    </span>
  }}
</td>
<td className="py-2.5 px-3">
  {node.id === 0 || node.term_lin
    <span className="text-slate-5
  } : {
    <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Cк
      #{node.term_link}
    </span>
  }}
</td>
<td className="py-2.5 px-3 text-s
  {Object.keys(node.children).len
    ? '0'
    : Object.entries(node.children
      .map([c, id]) => `${c}→#
      .join(', '))}
</td>
<td className="py-2.5 px-3">
  {node.words.length === 0 ? {
    <span className="text-slate-6

```

```
// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
    const n = arr.length;
    const tree: Record<number, number> = {};
    function go(v: number, l: number, r: number) {
        if (l === r) { tree[v] = arr[l]; return; }
        const m = (l + r) >> 1;
        go(2 * v, l, m);
        go(2 * v + 1, m + 1, r);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    }
    go(1, 0, n - 1);
    return tree;
}
```

// ===== STEP GENERATORS =====

```
// 1. Build steps (recursive top-down)
function genBuildSteps(arr: number[]): Step[] {
    const n = arr.length;
    const steps: Step[] = [];
    let id = 0;
    const tree: Record<number, number> = {};

    const CODE = [
        "build(v, l, r) {", // 1
        "    if (l === r) {", // 2
        "        tree[v] = A[l]; return", // 3
        "    }", // 4
        "    mid = (l + r) >> 1", // 5
        "    build(2v, l, mid)", // 6
        "    build(2v+1, mid+1, r)", // 7
        "    tree[v] = tree[2v]+tree[2v+1]", // 8
        "}", // 9
    ];
    void CODE;
}
```

```

    }
    if (ql <= l && r <= qr) {
        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
        return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 6, treeState: tree, highlightNodes: [v]
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`, codeLine: 8, treeState: tree, highlightNodes: [v],
    rgeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1], result: res
    return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}] = ${ans}`, codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number): Step[] {
    const n = arr.length;
    // Build iterative tree: leaves at [n..2n-1]
    const tree: Record<number, number> = {};
    for (let i = 0; i < n; i++) tree[n + i] = arr[i];
    for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

    const steps: Step[] = [];
    let id = 0;
    let l = qL + n;
    let r = qR + n + 1; // half-open [l, r)
    let res = 0;

    steps.push({ id: id++, desc: `Справа: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}, codeLine: 3, treeState: tree, highlightNodes: [l, r]
    return steps;
}

```



// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
];

const QUERY_TD_CODE = [
    "query(v, l, r, ql, qr) {",
    "    if (ql > r || qr < l) return 0;",
    "    // -----",
    "    if (ql <= l && r <= qr) return tree[v];",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    // спускаемся в обоих детей",
    "    return query(left) + query(right);",
    "}"
];

const QUERY_BU_CODE = [
    "queryBU(ql, qr) {",
    "    res = 0;",
    "    l = ql + n; r = qr + n + 1;",
    "    // -----",
    "    while (l < r) {",
    "        if (l & 1) res += tree[l++];",
    "        // -----",
    "        if (r & 1) res += tree[--r];",
    "        l >>= 1; r >>= 1;",
    "    }",
    "    return res; }"
];
```

// ===== PLAYER COMPONENT =====

```

const isHigh = step.highlightNodes.includes(nd.v);
const isChild = step.mergeChildren?.includes(nd.v);
const has = nd.val !== null;

let cls = 'bg-slate-800 border-slate-700 text-slate-200';
if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-2xl`;
else if (isChild) cls = `${clr.childRing} text-amber-500`;
else if (has) cls = `${clr.filled} text-slate-200`;

return (
  <div key={nd.v} className="flex flex-col items-center justify-center">
    <div className={`flex flex-col items-center justify-center ${cls}`}>
      <span className="text-[7px] opacity-60 font-mono">{nd.val}</span>
      <span className="text-[9px] font-bold">[{nd.left, nd.right}]</span>
      <span className="text-xs font-extrabold font-mono">{nd.val}</span>
    </div>
    {(nd.left || nd.right) && (
      <div className="flex flex-col items-center mt-2">
        <div className="w-px h-2 bg-slate-700" />
        <div className="flex justify-around w-full">
          {nd.left && renderNode(nd.left)}
          {nd.right && renderNode(nd.right)}
        </div>
      </div>
    )}
  </div>
);
}, [step, clr]));

```

```

return (
  <div className={`rounded-2xl border overflow-hidden ${
    step.isHigh ? 'border-emerald-500/30' : 'border-amber-500/30'
  } /* Header */`} >
    <div className={`px-4 py-2.5 flex items-center justify-center ${
      step.isHigh ? 'bg-emerald-950/50' : 'bg-amber-950/50'
    } border-b-2 border-slate-700`} >
      <h4 className="font-bold text-sm text-white flex-grow-1">
        {step.isHigh ? 'High' : 'Child'}
      </h4>
      <span className="text-[10px] font-mono text-slate-200">{step.val}</span>
    </div>
  </div>
);

```

```

{ /* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 border
  px]">
  <span className={`mt-0.5 inline-block w-2 h-2 r
  <span className="leading-relaxed">{step.desc}</
  {step.result !== undefined && {
    <span className={`ml-auto shrink-0 font-mono
      ' : color === 'emerald' ? 'text-emerald-300
      = {step.result}
    </span>
  }}
</div>

```

```

{ /* Controls */ }
<div className="px-3 py-2.5 bg-slate-950 border-t-2 border-l-2 border-r-2 border-b-2 border-slate-900" >
  <div className="flex items-center bg-slate-900" >
    <button onClick={() => { setPlaying(false); setSpeed(1200); }}
      className="p-2 rounded" title="C6poc"><RotateCcw className="w-4 h-4" />
    <button onClick={() => { setPlaying(false); setSpeed(700); }}
      className="p-2 rounded" title="Hopa"><RotateCcw className="w-4 h-4" />
    <button onClick={() => setPlaying(!playing)}
      className="p-2 rounded" title="Play/Pause">
      {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
    </button>
    <button onClick={() => { setPlaying(false); setSpeed(120); }}
      className="p-2 rounded" title="Typ6o"><RotateCcw className="w-4 h-4" />
    </button>
  </div>
  <select value={speed} onChange={e => setSpeed(Number(e.target.value))}
    className="w-40 text-[9px] font-bold rounded px-1.5 py-1 outline-1 outline-slate-400" >
    <option value={1200}>Медл.</option>
    <option value={700}>Норма</option>
    <option value={350}>Быстро</option>
    <option value={120}>Тип6o</option>
  </select>
</div>

```

```
{/* Progress bar */}
```

```
const queryTDSteps = useMemo(() => genQueryTopDownSteps)
const queryBUSteps = useMemo(() => genQueryBottomUpSteps)
```

```
const totalSum = arr.reduce((a, b) => a + b, 0);
```

```
return (
  <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
    {/** ---- TOP BAR: array editor + query range ---- */}
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">
        {/** Array input */}
        <form onSubmit={handleApply} className="flex flex-col">
          <div className="flex items-center justify-between">
            <label className="text-[10px] font-bold uppercase">
              Array input
            </label>
            <button type="button" onClick={randomize} className="text-white">
              Refresh
            </button>
          </div>
          <div className="flex gap-2">
            <input
              type="text"
              value={customInput}
              onChange={e => setCustomInput(e.target.value)}
              className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
              placeholder="5, 8, 3, 12, 7, 2"
            />
            <button type="submit" className="bg-indigo-600 text-white px-4 py-2">
              Применить
            </button>
          </div>
          <div className="flex flex-wrap gap-1.5 pt-1">
            {arr.map((v, i) => (
              <span key={i} className="bg-slate-900 border border-indigo-500 padding-2px">
                {v}
              </span>
            ))}
            <span className="text-xs text-slate-500 font-medium">
              A[0] = 5, A[1] = 8, A[2] = 3, A[3] = 12, A[4] = 7, A[5] = 2
            </span>
          </div>
        </form>
      </div>
    </div>
  )
```

```

<div className="flex flex-wrap items-center gap-1"
  <button onClick={() => setView('build')} className=
    -colors ${view === 'build' ? 'border-indigo-500' : 'border-slate-200'}`}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} class=
    on-colors ${view === 'queries' ? 'border-emerald-500' : 'border-slate-200'}`}>
    <Search className="w-3.5 h-3.5" /> Запрос: св
  </button>
  <button onClick={() => setView('theory')} class=
    n-colors ${view === 'theory' ? 'border-blue-500' : 'border-slate-200'}`}>
    <Info className="w-3.5 h-3.5" /> Почему / Сра
  </button>
</div>

```

```

{/* ---- TAB CONTENT ---- */}

```

```

{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=""
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
)}

```

```

{view === 'queries' && (
  <div className="grid grid-cols-1 xl:grid-cols-2"
    <SimPanel
      key={`qtd-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху
      icon=""
      steps={queryTDSteps}

```



```

const groups = useMemo(() => {
  const q = query.trim().toLowerCase();
  const filtered = q ? chapters.filter((c) => c.title
  const map = new Map<string, typeof chapters>();
  for (const c of filtered) {
    const key = c.category?.trim() || "Прочие темы";
    if (!map.has(key)) map.set(key, []);
    (map.get(key) as typeof chapters).push(c);
  }
  return Array.from(map.entries());
}, [query]);

return (
  <aside className="w-full bg-slate-900/80 lg:bg-tran
    <div className="p-4 pb-3 shrink-0">
      <h3 className="text-xs font-bold text-slate-400
        <Compass className="w-4 h-4 text-indigo-400"
      </h3>
      <div className="relative">
        <Search className="w-4 h-4 text-slate-500 abs
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Поиск по темам..."
          className="w-full bg-slate-800/70 border bo
          500 focus:outline-none focus:border-indigo-!
        />
        {query && (
          <button
            onClick={() => setQuery("")}
            className="absolute right-2 top-1/2 -tran
            aria-label="Очистить поиск"
          >
            <X className="w-3.5 h-3.5" />
          </button>
        )}
      </div>

```

```

        </button>
      );
    }}}
  </div>
</div>
)}}
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-300 to-indigo-400
] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex items-center">
    <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по наведению.
  </p>
  <p className="text-slate-400 italic">Стрелки ← и → переключают вид.
  </div>
</aside>
);
};

```

---

ФАЙЛ 59/82: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  icon: string;
}

```



```

const [newCustomProb, setNewCustomProb] = useState(0.);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: '🍳' },
    { name: 'Кастрюля от супа', icon: '🍲' },
    { name: 'Чашка от кофе', icon: '☕' },
    { name: 'Миска с остатками салата', icon: '🥗' },
    { name: 'Тарелка от тако', icon: '🌮' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
};

const resetStack = () => {
  setStack([
    { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
    { id: '3', name: 'Блюдец от торта', icon: '🍰' },
  ]);
};

```

```

    });

    // Heap handlers
    const addHypothesis = (e: React.FormEvent) => {
      e.preventDefault();
      if (!newCustomText.trim()) return;
      const item: Hypothesis = {
        id: Date.now().toString(),
        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
      };
      setHeap(prev => [...prev, item]);
      setNewCustomText('');
      setHeapMessage(`Добавлена гипотеза с вероятностью`);
    };

    const popHeap = () => {
      if (heap.length === 0) {
        setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
        return;
      }
      // Find highest probability
      let maxIdx = 0;
      for (let i = 1; i < heap.length; i++) {
        if (heap[i].probability > heap[maxIdx].probability) {
          maxIdx = i;
        }
      }
      const best = heap[maxIdx];
      setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
      setHeapMessage(`Выбран самый "тяжелый" кандидат: ${best.text}`);
    };

    const resetHeap = () => {
      setHeap([
        { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
        { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
        { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
      ]);
    };
  
```

```

    </span>
  </button>

  <button
    onClick={() => setActiveTab('queue')}
    className={`flex items-center justify-between
      activeTab === 'queue'
        ? 'bg-indigo-600/20 border-indigo-500 text-white'
        : 'bg-slate-800/60 border-slate-700/60 text-white'}
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-400'}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">Queue
          <div className="text-xs opacity-80">FIFO
        </div>
      </div>
      <span className="text-xs bg-indigo-500/20 text-white">
        BFS
      </span>
    </button>

    <button
      onClick={() => setActiveTab('heap')}
      className={`flex items-center justify-between
        activeTab === 'heap'
          ? 'bg-emerald-600/20 border-emerald-500 text-white'
          : 'bg-slate-800/60 border-slate-700/60 text-white'}
    >
      <div className="flex items-center gap-3">
        <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-400'}>
          <div className="text-left">
            <div className="font-bold text-sm text-white">Heap
            <div className="text-xs opacity-80">Приоритетная
          </div>
        </div>
      </div>
    >
  </div>
</div>

```

```

        className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
      >
        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  </div>

  {popMessage && (
    <div className="bg-blue-950/60 border borde
imate-fade-in">
      <span className="inline-block w-2 h-2 roun
        {popMessage}
      </div>
    )}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
    >
    {stack.length === 0 ? (
      <div className="text-center py-12 text-sl
        <div className="text-5xl mb-3">    </div>
        <p className="text-base font-bold">Стор
        <p className="text-xs mt-1">Добавьте гр
      </div>
    ) : (
      <div className="w-full max-w-sm flex flex
        <div className="absolute top-0 text-xs
full flex items-center gap-1">
        <span>ВЕРШИНА СТЕКА (TOP)</span>
        <ArrowDown className="w-3.5 h-3.5 anim
      </div>

      {/* Reversed map so top of stack is vis
      {stack.slice().reverse().map((plate, ind
        const isTop = index === 0;
        return (
          <div

```

```

        <span> Симуляция очереди за супом</span>
        <span className="text-xs font-mono bg-indigo-500/40 border border-indigo-500/60 p-1" />
      /span>
    </h3>
    <p className="text-slate-400 text-xs mt-1">
      Кто первым пришел, тот первым получил суп
    </p>
  </div>
  <div className="flex flex-wrap items-center gap-2">
    <button
      onClick={addPerson}
      className="flex-1 md:flex-none flex items-center justify-center gap-2 rounded-xl text-sm font-bold shadow-lg"
    >
      <Plus className="w-4 h-4" /> Встать в очередь
    </button>
    <button
      onClick={servePerson}
      className="flex-1 md:flex-none flex items-center justify-center gap-2 rounded-xl border-indigo-500/40 px-4 py-2.5 rounded"
    >
      Налить суп первому
    </button>
    <button
      onClick={resetQueue}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white transition-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{dequeueMessage && (
  <div className="bg-indigo-950/60 border border-indigo-500/60 p-3 animate-fade-in">
    <span className="inline-block w-2 h-2 rounded" />

```



```

        className="w-full bg-slate-900 border border-emerald-500 transition-colors"
      />
    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emerald-500">
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e.target.value))}
        className="w-full accent-emerald-500 cursor-pointer"
      />
    </div>
    <div className="md:col-span-3">
      <button
        type="submit"
        disabled={!newCustomText.trim()}
        className="w-full flex items-center justify-center gap-2 border border-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed rounded-md p-2"
      >
        <Plus className="w-4 h-4" /> Добавить в список
      </button>
    </div>
  </form>

  {heapMessage && (
    <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">
      <span className="inline-block w-2 h-2 rounded-full bg-emerald-500" />
      {heapMessage}
    </div>
  )}

```

```

        {item.text}
      </div>
      {isTop && (
        <span className="text-[10px]
-0.5 rounded inline-block mt-1">
          Вершина Кучи (Root) • Буд
        </span>
      )}
    </div>
  </div>

  <div className="flex items-center
  { /* Custom progress bar */}
  <div className="hidden sm:block
    <div
      className={`h-full rounded-
      style={{ width: `${percent}%
    ></div>
  </div>
  <span className={`font-mono font
    isTop ? 'text-emerald-400 tex
  }`}>
    {percent}%
  </span>
  </div>
</div>
  );
  })))
</div>
)}
<div className="mt-6 text-xs text-slate-500
  НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
  </div>
</div>
</div>
)}
</div>
);

```



```

    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q) ||
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

return (
  <div>
    <div className="mb-6">
      <h2 className="text-xl sm:text-2xl font-extrabold">
        <p className="text-sm text-slate-400 leading-relaxed">
          Все интерактивные демонстрации в одном месте.
          здесь их можно открыть напрямую.
        </p>
      </div>

      <div className="relative mb-6 max-w-md">
        <Search className="w-4 h-4 text-slate-500 absolute">
          <input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Поиск тренажёра..."
            className="w-full bg-slate-900 border border-1
              focus:outline-none focus:border-indigo-500"
          />
          {query && (
            <button
              onClick={() => setQuery("")}
              className="absolute right-2 top-1/2 -translate-y-1/2"
              aria-label="Очистить поиск"
            >
              <X className="w-4 h-4" />
            </button>
          )}
        </div>
      </div>

```

---

ФАЙЛ 61/82: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево,
 * поднять отмеченный узел в корень. Клик по узлу = один поворот.
 * Порядок поворотов выбираешь сам; правильный ли он по твоему?
 * только после того, как узел окажется наверху (или по твоему мнению).
 */
```

```
export interface TNode {
  key: number;
  left: TNode | null;
  right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просеивание
export const PRESETS: { name: string; hintCase: string; target: number; build: () => TNode } = {
```

```
  {
    name: "Линия влево",
    hintCase: "Zig-Zig",
    target: 1,
    build: () => {
      const n1 = leaf(1);
```

```

    const path: TNode[] = [];
    let cur = root;
    while (cur) {
        path.push(cur);
        if (key === cur.key) return path;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];

```

```

const rowH = 62;
return {
  nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2, size: 1000, color: '#ccc' })),
  edges,
  width: cols * colW,
  height: rows * rowH + 20,
};
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.build());
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; color: string }[]>([]);
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
    [presetIdx]
  );

  const { nodes, edges, width, height } = useMemo(() => preset.build(), [presetIdx]);
  const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
  const expected = useMemo(() => (solved ? null : canonicalPath(target, nodes)), [target, nodes]);

```

```

    </button>
  </div>

  <div className="flex gap-2 mb-3 overflow-x-auto" >
    {PRESETS.map((p, i) => {
      <button
        key={p.name}
        onClick={() => reset(i)}
        className={`px-3 py-1.5 rounded-lg text-[11pt]
          i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"
        }`}
      >
        {p.name}
      </button>
    })}
  </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800" >
    <svg
      viewBox={`0 0 ${width} ${height}`}
      className="h-auto block mx-auto"
      style={{ minWidth: Math.min(width * 1.5, 420) }}
      role="img"
    >
      {edges.map(([a, b], i) => {
        const na = nodes.find((n) => n.key === a);
        const nb = nodes.find((n) => n.key === b);
        if (!na || !nb) return null;
        const onPath = path.has(a) && path.has(b);
        return (
          <line
            key={i}
            x1={na.x}
            y1={na.y}
            x2={nb.x}
            y2={nb.y}
            stroke={onPath ? "#6366f1" : "#475569"}
            strokeWidth={onPath ? 3 : 1.8}
          />
        )
      })}
    </svg>
  </div>

```

```

        onClick={undo}
        disabled={history.length === 0}
        className="flex items-center gap-1.5 px-3 py-1
          -white disabled:opacity-40 text-xs font-bold"
      >
        <Undo2 className="w-3.5 h-3.5" /> Отменить
    </button>
    <button
      onClick={() => reset()}
      className="flex items-center gap-1.5 px-3 py-1
        ext-xs font-bold transition-colors"
    >
      <RotateCcw className="w-3.5 h-3.5" /> Заново
    </button>
    <button
      onClick={() => setShowAnswer((s) => !s)}
      className={`flex items-center gap-1.5 px-3 py-1
        showAnswer
          ? "bg-amber-600/20 border-amber-500 text-amber-500"
          : "bg-slate-800 border-slate-700 text-slate-200"
        }`
    >
      {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
    </button>

    <span className="text-[11px] text-slate-500 ml-1" >
      поворотов: {moves.length} • минимум: {minimalMoves}
    </span>
  </div>

  {showAnswer && !solved && expected && (
    <div className="mt-3 text-[12px] bg-amber-950/40 p-3" >
      Сейчас splay крутил бы узел{" "}
      <span className="font-mono font-bold">{expected}
    </div>
  )}
  )}

```

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
 * * сразу видно, что дерево остаётся деревом поиска;
 * * под деревом печатается обход слева направо: он ОДНОЗНАЧЕН;
 * * это и есть доказательство, что поворот ничего не ломает;
 * * кадр «прицел» ничего не двигает, только подсвечивает;
 * * на кадре «результат» остаются призраки – пунктирные узлы
 *   местах и стрелки «откуда → куда»;
 * * по умолчанию анимация НЕ крутится сама: пользователь
 *   идёт в своём темпе. Автопрокрутка включается кнопкой.
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
  moved?: NodeId[];
  ms: number;
}
```

```

        text: "Нам нужен ключ 20, а он на глубине 1. Деду
ms: RESULT_MS,
    },
    {
        ...ZIG_BEFORE,
        kind: "aim",
        title: "Прицел: крутим ребро 40 – 20",
        text: "Ничего пока не двигается. Смотри на подсве
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [160, 50], A: [80, 130], p: [245, 130],
edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 – корень",
text: "40 стало правым сыном двадцатки. Поддерев
        ева от 40 – это одно и то же место.",
        moved: ["x", "p", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

```

/* ----- Zig-Zig -----
const ZZ_S0 = {
    pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZZ_S1 = {
    pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZIG: Case = {
    key: "zig-zig",
    label: "Zig-Zig",

```



```

    },
    {
      pos: { x: [100, 45], A: [45, 118], p: [188, 118],
      edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
      kind: "result",
      title: "Стало: 20 в корне, бамбук стал кустом",
      text: "Сравни с первым кадром: А и В были на глуб
        дерево, а не только достаёт ключ.",
      moved: ["x", "p", "A", "B"],
      ms: RESULT_MS,
    },
  ],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
  pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
  edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
  pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
  edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
  key: "zig-zag",
  label: "Zig-Zag",
  when: "x и p – дети с РАЗНЫХ сторон (змейка)",
  rotations: "2 поворота, первым – нижний",
  keys: { x: 40, p: 20, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZG_S0,
      kind: "start",
      title: "Было: 60 → влево на 20 → вправо на 40",
    }
  ]
}

```

```

    ],
  };

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

const COLOR: Record<string, { fill: string; stroke: string }> = {
  x: { fill: "#6366f1", stroke: "#a5b4fc" },
  p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
  g: { fill: "#f59e0b", stroke: "#fcd34d" },
  sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id === "B";

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C }> = ({
  frame,
  prev,
  keys,
}) => {
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
  const ghosts = frame.kind === "result" ? (moved ?? []) : [];

  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
      <defs>
        <marker id="splay-arrow" viewBox="0 0 10 10" refX="10" refY="0">
          <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
        </marker>
      </defs>

      {/* призраки: где узел стоял до поворота */}
      {ghosts.map((id) => {
        const from = prev[id]!;
        const to = pos[id]!;
        const r = isSubtree(id) ? 15 : 20;

```

```

        style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)"}
      />
    );
  }}}

  {(Object.keys(pos) as NodeId[]).map((id) => {
    const p = pos[id];
    if (!p) return null;
    const sub = isSubtree(id);
    const c = COLOR[sub ? "sub" : id];
    const r = sub ? 15 : 20;
    const hot = Boolean(focus && focus.includes(id));
    return (
      <g
        key={id}
        style={{
          transform: `translate(${p[0]}px, ${p[1]}px)`,
          transition: MOVE,
          opacity: dim(id) ? 0.28 : 1,
        }}
      >
        {hot && <circle r={r + 7} fill="none" stroke="red" />}
        <circle r={r} fill={c.fill} stroke={c.stroke} />
        <text
          textAnchor="middle"
          dominantBaseline="central"
          fontSize={sub ? 12 : 15}
          fontWeight="700"
          fill={sub ? "#94a3b8" : "#fff"}
        >
          {sub ? id : keys[id]}
        </text>
        {!sub && (
          <text textAnchor="middle" y={-r - 7} font-size={10}>
            {id}
          </text>
        )}
      </g>
    );
  })}

```

```

        i === caseIdx ? "bg-indigo-600 text-white
      }` }
    >
    {c.label}
  </button>
  )))
</div>

```

```

<div className="mb-3 text-xs sm:text-[13px] text-
  <span className="font-bold text-indigo-300">Кор,
  <span className="text-slate-500">({active.rotat
</div>

```

{/\* шаг + пояснение стоят НАД картинкой: сначала

```

<div className="mb-3 rounded-xl border border-sla
  <div className="flex items-center gap-2 mb-1 fl
    <span
      className={`text-[10px] font-bold uppercase
        current.kind === "aim"
          ? "bg-amber-500/20 text-amber-300"
          : current.kind === "start"
            ? "bg-slate-700 text-slate-300"
            : "bg-indigo-500/20 text-indigo-300"
        }` }
    >
    {current.kind === "aim" ? "прицел" : current
  </span>
  <span className="text-sm font-bold text-white
    Шаг {idx + 1} из {total}. {current.title}
  </span>
</div>
<p className="text-[13px] text-slate-400 leadin
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
  <Tree frame={current} prev={prevPos} keys={acti

```

{/\* обход слева направо – одинаковый на всех ка

```

        disabled={idx === 0}
        className="px-3 py-2 rounded-lg bg-slate-800 text-slate-300
        d: hover:text-slate-300 text-xs font-bold transition-colors"
    >
        ← Назад
    </button>
    <button
        onClick={() => {
            setPlaying(false);
            setFrame((f) => (f + 1) % total);
        }}
        className="px-4 py-2 rounded-lg bg-indigo-600 text-white
        w-indigo-900/40"
    >
        {last ? "⏮ Сначала" : "Дальше →"}
    </button>
    <button
        onClick={() => setPlaying((p) => !p)}
        className="flex items-center gap-1.5 px-3 py-2 text-white
        text-xs font-bold transition-colors"
    >
        {playing ? <Pause className="w-3.5 h-3.5" /> : <Play />}
        {playing ? "Стоп" : "Авто"}
    </button>
    <button
        onClick={() => setSlow((s) => !s)}
        title="Автопрокрутка ещё медленнее"
        className={`flex items-center gap-1.5 px-3 py-2 text-white
        slow
            ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
            : "bg-slate-800 border-slate-700 text-slate-300"}
    >
        <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1x" : "1x"}
    </button>
    <button
        onClick={() => {
            setFrame(0);
        }}
    >
        ⏮ Сбросить
    </button>
    </div>
</script>

```

```

interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
  return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
  const y = x.left;
  if (!y) return x;
  x.left = y.right;
  y.right = x;
  return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
  const y = x.right;
  if (!y) return x;
  x.right = y.left;
  y.left = x;
  return y;
};

```

```
// We implement the classic top-down or bottom-up splay
// For visualization logs, we do a bottom-up explanation
let path: { node: SplayNode; dir: "left" | "right" } = { node: root, dir: null };
let current: SplayNode | null = root;

// Find the element or the element where search failed
let lastNode: SplayNode = root;
while (current) {
  lastNode = current;
  if (key === current.key) {
    path.push({ node: current, dir: null });
    break;
  } else if (key < current.key) {
    path.push({ node: current, dir: "left" });
    current = current.left;
  } else {
    path.push({ node: current, dir: "right" });
    current = current.right;
  }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
  steps.push(`Поиск ${key}: элемент отсутствует в дереве`);
  steps.push(
    `Поиск споткнулся на узле [${lastNode.key}]. Инициализируем подъем`
  );
} else {
  steps.push(
    `Поиск ${key}: элемент найден! Запускаем подъем`
  );
}

// Now let's implement standard Splay logic recursively
const splayAction = (
```

```

        }
      } else if (k > node.right.key) {
        // Zag-Zag (Right-Right)
        node.right.right = splayAction(node.right.right, target);
        node = leftRotate(node);
        steps.push(
          `Вращение Zag-Zag (Левый поворот родителя для правого ребенка)`
        );
      }

      if (!node.right) return node;
      steps.push(
        `Финальный поворот Zag (Левое вращение корня)`
      );
      return leftRotate(node);
    }
  };

  const finalRoot = splayAction(cloneTree(root), target);
  return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
  setHighlightedNode(key);
  const { newRoot, steps } = splayStepByStep(tree, key);
  if (newRoot) {
    setTree(newRoot);
  }
  setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;

  // Standard BST insert, then splay!

```



```

        if (node.right) {
            computeCoordinates(node.right, x + levelWidth, y + levelHeight);
        }
    };

    const drawTree = cloneTree(tree);
    computeCoordinates(drawTree, 300, 40, 140);

    // Render lines and nodes
    const renderLines = (node: SplayNode | null): React.ReactNode[] => {
        if (!node) return [];
        let lines: React.ReactNode[] = [];
        if (node.left && node.left.x && node.left.y) {
            lines.push(
                <line
                    key={`l-${node.key}-${node.left.key}`}
                    x1={node.x}
                    y1={node.y}
                    x2={node.left.x}
                    y2={node.left.y}
                    stroke="#475569"
                    strokeWidth="2"
                />,
            );
            lines = lines.concat(renderLines(node.left));
        }
        if (node.right && node.right.x && node.right.y) {
            lines.push(
                <line
                    key={`r-${node.key}-${node.right.key}`}
                    x1={node.x}
                    y1={node.y}
                    x2={node.right.x}
                    y2={node.right.y}
                    stroke="#475569"
                    strokeWidth="2"
                />,
            );
        }
    };

```

```

        </g>,
    );
    circles = circles.concat(renderNodes(node.left));
    circles = circles.concat(renderNodes(node.right));
    return circles;
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    { /* Header */}
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <BookOpen className="w-5 h-5 text-indigo-400"
        Интерактивное Splay-дерево
      </h3>
      <p className="text-xs text-slate-400">
        Кликните на вершины дерева, чтобы «вытащить»
        Splay.
      </p>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-12
      { /* Playfield */}
      <div className="lg:col-span-7 bg-[#0f172a] p-4
        <svg
          className="w-full h-full max-w-[600px] max-h-[320px]
          viewBox="0 0 600 320"
        >
          {renderLines(drawTree)}
          {renderNodes(drawTree)}
        </svg>

        { /* Quick interactive controls */}
        <div className="absolute bottom-4 left-4 right-4
          <form onSubmit={handleFind} className="flex
            <input
              type="number"
              placeholder="Поиск..."

```

```

        <RotateCcw className="w-3.5 h-3.5" /> Сброс
      </button>
    </div>
  </div>

```

```

  { /* Logs */ }
  <div className="lg:col-span-5 bg-slate-950 border-1
    to overflow-y-auto custom-scrollbar">
    <h4 className="text-xs font-bold text-slate-400">
      Ход вращения (Splay-логи)
    </h4>
    <div className="flex-1 space-y-2 mb-4 overflow-y-auto">
      {log.map((step, idx) => (
        <div
          key={idx}
          className={`text-xs p-2.5 rounded transition
            idx === log.length - 1
              ? "border-l-2 border-indigo-500 bg-slate-900/50"
              : "text-slate-400 bg-slate-900/50"
            }`}
        >
          {step}
        </div>
      ))}
    </div>

```

```

  <div className="border-t border-slate-800 pt-2">
    <p>
      <span className="font-semibold text-slate-400">
        у самого корня. Делаем 1 вращение.
      </span>
    </p>
    <p>
      <span className="font-semibold text-slate-400">
        оба левые или оба правые. Вращаем деда, поворачиваем
      </span>
    </p>
    <p>
      <span className="font-semibold text-slate-400">
        колено. Вращаем узел в разные стороны.
    </span>

```

```

<div className="bg-slate-900/60 p-4 rounded-l
  <div>
    <p className="font-bold text-slate-300 mb
      2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-rela
      Если агент запрашивает по очереди{" "}
      <code className="text-rose-400">[10]</code>
      <code className="text-rose-400">[60]</code>
      углы), дерево будет превращаться в палки
      сторону. Первая операция Splay(60) стои
      <strong className="text-white">O(n)</st
      глубину n. Вторая Splay(10) заставляет
      Постоянный свинг убивает производительн
      сложности <strong className="text-white
    </p>
  </div>
  <div className="mt-3 text-[10px] text-rose-
    Кэшировать свингующие пары на концах — фа
  </div>
</div>

```

```

<div className="bg-slate-900/60 p-4 rounded-l
  <div>
    <p className="font-bold text-slate-300 mb
      3. Амортизация O(log n)
    </p>
    <p className="text-slate-400 leading-rela
      Парадокс: один запрос за{" "}
      <code className="text-emerald-400">O(n)</code>
      всю структуру вдвое более плоской за сч
      при двойных вращениях {
      <code className="text-emerald-400 font-
      }. Дорогая операция инвестирует в дешев
      операций. Поэтому амортизированное врем
      <strong className="text-white">O(log n)</st
    </p>
  </div>

```

```

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', state: 'idle' },
    ]
  );

  const [cleanRack, setCleanRack] = useState<Plate[]>(
    []
  );
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>(
    []
  );

  const addLog = (msg: string) => setLog(prev => [...prev, msg]);

  // PUSH: Положить грязную тарелку сверху стопки
  const pushPlate = useCallback(() => {
    if (isWashing) return;
    if (dirtyStack.length >= 7) {
      addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
      setShake(true);
      setTimeout(() => setShake(false), 400);
      return;
    }
  }, [isWashing]);

  const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };
};

```

```

        setShowSoap(false);
        setIsWashing(false);
        addLog(` Готово! Чистая тарелка ${topPlate.emoji}
    }, 850);
    }, [dirtyStack, isWashing]));

```

```

const reset = () => {
    counter = 0;
    setDirtyStack([
        { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝',
          State: 'in' },
        { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕',
          imState: 'in' },
        { id: 'r3', name: 'Блюдец от торта', emoji: '🍰',
          mState: 'in' },
    ]);
    setCleanRack([]);
    setIsWashing(false);
    setShowSoap(false);
    setLog([' Стол сброшен. Посуда снова грязная!']);
};

```

```

const topIndex = dirtyStack.length - 1;

```

```

return (
    <div className="flex flex-col gap-6">
        {/* МНЕМОНИКА / ПРАВИЛО */}
        <div className="bg-blue-950/40 border border-blue-950">
            <div className="text-3xl"> </div>
            <div>
                <h3 className="font-black text-blue-400 text-2xl">
                    Ты складываешь грязные тарелки друг на друга.
                    А когда приходит время мыть, ты берёшь именно эту.
                    Попытаешься вытащить нижнюю – всё рухнет!
                </h3>
            </div>
        </div>
    </div>
)

```



```

<div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500" />
</div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4" />
  <div className="absolute top-4 left-4 text-[18px] font-mono text-white" />
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-center gap-2" />
    <Sparkles className="w-3.5 h-3.5 animate-pulse text-white" />
    <span>ЧИСТО </span>
  </div>

  <div className="flex-1 flex flex-col justify-between p-4" />
    {cleanRack.length === 0 ? (
      <div className="flex-1 flex flex-col items-center justify-center gap-2" />
        <span className="text-5xl mb-2" />
        <p className="text-xs font-bold font-mono text-white" />
        <p className="text-[10px] mt-1" />Здесь 6 тарелок
      </div>
    ) : (
      <div className="flex flex-col gap-2 w-full" />
        {cleanRack.map(plate => (
          <div
            key={plate.id}
            className="w-full h-8 rounded-full bg-white" />
            <div className="flex items-center justify-center gap-2 opacity-50" />
            <span className="text-sm" /> {plate.name}
            <span className="text-[10px] text-white" /> {plate.status}
          </div>
        ))}
      </div>
    )}
  </div>

```



```

steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало

for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i,
        ;

    while (j > 0 && s[i] !== s[j]) {
        steps.push({ i, j, pi: [...pi], highlight:
            яд назад: откат j = pi[${j - 1}] = ${pi[j -
            j = pi[j - 1];
    }

    if (s[i] === s[j]) {
        steps.push({ i, j, pi: [...pi], highlight:
            ска увеличивается.` });
        j++;
    } else {
        steps.push({ i, j, pi: [...pi], highlight:
            овпадения нет.` });
    }

    pi[i] = j;
    steps.push({ i, j, pi: [...pi], desc: `Записывае

    if (j === pattern.length) {
        steps.push({ i, j, pi: [...pi], found: true
    }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);

```

```

        if (z[i] === pattern.length) {
            steps.push({ i, l, r, z: [...z], found: true });
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
    const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
    const [pattern, setPattern] = useState("aba");
    const [text, setText] = useState("abacaba");

```

```

    // Ensure inputs are valid dynamically
    const safePattern = pattern || " ";
    const safeText = text || " ";

```

```

    const { s, steps, patternLength } = useMemo(() => {
        if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
        else return generateZSteps(safePattern, safeText);
    }, [mode, safePattern, safeText]);

```

```

    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);
    const timer = useRef<NodeJS.Timeout | null>(null);

```

```

    // Reset when inputs or mode change
    useEffect(() => {
        setStepIdx(0);
        setAutoPlay(false);
    }, [s, mode]);

```

```

    // Timer loop for autoPlay
    useEffect(() => {
        if (autoPlay) {
            timer.current = setInterval(() => {
                setStepIdx(prev => {
                    if (prev >= steps.length - 1) { setAutoPlay(false); }
                    return prev + 1;
                });
            }, 1000);
        }
    }, [steps, autoPlay]);

```

```

te text-xs font-bold font-mono px-2 py-1 ou
    </div>
  </div>
</div>

<div className="bg-slate-950 rounded-xl border-2 border-slate-900 px] flex items-center justify-start">
  <div className="flex gap-1.5 px-4 h-full">
    {s.split("").map((char, index) => {
      const isPattern = index < pattern.length;
      const isSeparator = index === pattern.length;

      let borderColor = "border-slate-900";
      let bgColor = "bg-slate-900";
      let textColor = isSeparator ? "text-slate-50" : "text-slate-400";

      // KMP Logic
      if (mode === "kmp") {
        if (step.highlight?.includePattern) {
          borderColor = "border-emerald-900";
          bgColor = step.match === true ? "bg-emerald-900" : "bg-slate-900";
        }
      }

      // Z Logic
      if (mode === "z") {
        if (index >= step.l && index <= step.r) {
          bgColor = "bg-emerald-900";
          borderColor = "border-emerald-900";
        }
        if (step.zTarget === index) {
          borderColor = "border-emerald-900";
          bgColor = step.match === true ? "bg-emerald-900" : "bg-slate-900";
        }
      }
    })}
  </div>
</div>

```

```

                <div className="abs
20">
                    <span className
                </div>
            }}

            {/* Z fingers */}
            {mode == "z" && step.i
                <div className="abs
e z-20">
                    <span className
                </div>
            }}
            {mode == "z" && step.z
                <div className="abs
                    <span className
                </div>
            }}
            {mode == "z" && step.z
                <div className="abs
                    <span className
                </div>
            }}

            {/* Found flash effect
            {step.found && mode ==
                <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
            }}
            {step.found && mode ==
                <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
            }}
        </div>
    )
    }}}
</div>
</div>

```

```

<span className="text-2xl mt-1"> </span>
<p className="text-xs text-slate-400 leading-relaxed">
    Обрати внимание на хитрость с решёткой.
    Мы пишем <b>Шаблон + # + Текст</b>.
    Когда значение {mode === 'kmp' ? "применяется" : "не применяется"}
    о было найдено!
    Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z).
</p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800 px-4 py-2 border-radius-4">
    <div className="bg-slate-800 px-4 py-2 border-radius-4 text-slate-400 uppercase">
        <span>Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})</span>
    </div>
    <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
    }
}`}

    </pre>
    </div>
</div>

);
}

```

```

    { u: 0, v: 2 },
    { u: 1, v: 3 },
    { u: 2, v: 3 },
    { u: 3, v: 4 },
  ];

```

```

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

```

```

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
      });
    };
  });
}

```

```

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    recordStep(
      `Запускаем новый обход DFS от нетронутой верши
      null,
    );
    dfs(i);
  }
}

recordStep(
  "Топологическая сортировка завершена! Полученный
  null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
  desc: "Запуск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1600);
  } else {
    setIsPlaying(false);
  }
}

```

```

        <button
            onClick={reset}
            className="flex items-center justify-center
            ors"
        >
            <RotateCcw className="w-4 h-4" />
        </button>
    </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12
    { /* Playfield SVG */ }
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"
            <defs>
                <marker
                    id="dag-arrow"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
                </marker>
                <marker
                    id="dag-arrow-active"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
                </marker>
            </defs>

```



```

        stroke = "#818cf8";
        textColor = "text-white font-bold";
    } else if (isProcessed) {
        fill = "#022c22";
        stroke = "#059669";
        textColor = "text-emerald-300";
    } else if (isVisited) {
        fill = "#3730a3";
        stroke = "#6366f1";
        textColor = "text-indigo-200";
    }

    return (
      <g key={node.id}>
        <rect
          x={node.x - 55}
          y={node.y - 22}
          width="110"
          height="44"
          rx="8"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
          className="transition-all duration-!
        />
        <text
          x={node.x}
          y={node.y - 4}
          textAnchor="middle"
          fill="white"
          fontSize="11px"
          fontWeight="bold"
          className="pointer-events-none"
        >
          ({node.label}) {node.name.split(" "
        </text>
        <text
          x={node.x}

```

```

        </span>
      ) : {
        getTopoListString()
      }
    </div>
  </div>

  { /* Color key */ }
  <div className="space-y-1">
    <span className="text-[10px] text-slate-500">
      ВЕРШИНЫ В DFS:
    </span>
    <div className="flex items-center gap-2">
      <span className="w-3.5 h-3.5 rounded bg-slate-400">
        <span className="text-slate-400 text-[10px]">
          Серые (зашли)
        </span>
      </span>
    </div>
    <div className="flex items-center gap-2">
      <span className="w-3.5 h-3.5 rounded bg-slate-900">
        <span className="text-slate-400 text-[10px]">
          Черные (готово)
        </span>
      </span>
    </div>
  </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev + 1)}
      className="bg-slate-900 border border-slate-700 text-white"
    >

```

## Универсальное пособие • полный исходный код

-----  
\* Канонический набор: все x и все y различны  $\Rightarrow$  дерево  
\* Узел идентифицируется парой  $id = "x:y"$ .  
\*/

```
export interface Pair {  
  key: number;  
  grade: number;  
}  
type NodeId = string;  
const pairId = (key: number, grade: number): NodeId =>  
  
interface TNode {  
  id: NodeId;  
  key: number;  
  grade: number;  
  left: TNode | null;  
  right: TNode | null;  
}  
  
export const POINTS: Pair[] = [  
  { key: 45, grade: 8 },  
  { key: 3, grade: 6 },  
  { key: 6, grade: 5 },  
  { key: 2, grade: 4 },  
  { key: 9, grade: 3 },  
  { key: 7, grade: 2 },  
  { key: 12, grade: 0 },  
  { key: 1, grade: -4 },  
];  
  
/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован  
export const INPUT_ORDER: Pair[] = [  
  { key: 3, grade: 5 },  
  { key: 45, grade: 8 },  
  { key: 3, grade: 6 },  
  { key: 1, grade: -4 },  
  { key: 7, grade: 2 },  
  { key: 6, grade: 5 },
```

```

        return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ aId: t.id, goLeft: false, note: `(${t.key}
        о левое поддереву.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
};
const res = go(root);
return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
    root: TNode | null;
    trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; note: string; };
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
    const trace: MergeResult["trace"] = [];
    const go = (a: TNode | null, b: TNode | null): TNode | null {
        if (!a || !b) {
            trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: `одно из деревьев целиком.` });
            return a ?? b;
        }
        if (a.grade > b.grade) {
            trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `a (${a.key}, ${a.grade}) имеет его правое поддереву с (${b.key}, ${b.grade})` });
            a.right = go(a.right, b);
            return a;
        }
        trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `b (${b.key}, ${b.grade}) имеет его левое поддереву с (${a.key}, ${a.grade})` });
        b.left = go(a, b.left);
        return b;
    };
    return { root: go(a, b), trace };
}

```

```

    prev = n.key;
    walk(n.right, n.key, max, n.grade);
  };
  walk(root, -Infinity, Infinity, null);
  return { bst, heap };
}

/* ----- раскладка сцены дерева -----

const layoutTree = (root: TNode | null) => {
  const pos = new Map<NodeId, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 44 });
    slot += 1;
    walk(n.right, depth + 1);
  };
  walk(root, 0);
  return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
  const es: { from: NodeId; to: NodeId }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ from: n.id, to: n.left.id }); }
    if (n.right) { es.push({ from: n.id, to: n.right.id }); }
  };
  walk(root);
  return es;
};

interface TreeViewProps {
  root: TNode | null;
  width?: number;
  height?: number;
}

```

```

        return (
          <g key={`tn-${n.id}`} transform={`translate
            <circle r={pulse ? 24 : 20} fill={nodeColo
          } strokeWidth={pulse ? 4 : 2} className="tr
            <text y={-2} fill="#fff" fontSize="13" fo
            <text y={13} fill="#cbd5e1" fontSize="10"
          </g>
        );
      }}}
    </svg>
    {subtitle && <p className="text-xs text-slate-400
  </div>
  );
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
  points: Pair[];
  drawnEdges: { a: Pair; b: Pair }[];
  pending: Pair | null;
  interactive?: boolean;
  selected?: Pair | null;
  cursor?: { x: number; y: number } | null;
  badEdge?: { a: Pair; b: Pair } | null;
  onPointDown?: (p: Pair) => void;
  onPointUp?: (p: Pair) => void;
  onMove?: (e: React.PointerEvent<SVGSVGElement>) => vo

```

```

    { /* временная линия игры */
    {interactive && selected && cursor && {
      <line
        x1={PX(selected.key)} y1={PY(selected.grade)}
        stroke="#818cf8" strokeWidth={2.5} strokeDash=[5,5]
      />
    }}

    {points.map((p) => {
      const done = drawnEdges.some((e) => (e.a.key === p.key))
      const isSel = selected && selected.key === p.key
      return (
        <g
          key={`pp-${p.key}:${p.grade}`}
          className="transition-all duration-500"
          onPointerDown={interactive && onPointDown ? () => {
            selected = p
          }}
          onPointerUp={interactive && onPointUp ? () => {
            selected = null
          }}
          style={interactive ? { cursor: "pointer" } : {}}
        >
          <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
            stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
            strokeDash={isSel ? [5,5] : done ? [5,5] : [1,0]}
          />
          <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12} font-family="monospace">{p.key}</text>
        </g>
      )
    })}

    {pending && {
      <g className="transition-all duration-500">
        <circle cx={PX(pending.key)} cy={PY(pending.grade)} r={10}
          stroke="#ccc" strokeDash=[1,0]
        />
        <text x={PX(pending.key)} y={PY(pending.grade) + 3.5} font-size={12} font-family="monospace">{pending.key}</text>
      </g>
    }}
  </svg>
);
};

```

```

for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e.a]);
const hasParent = new Set(edges.map((e) => gpid(e.b)));
const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
const ok = (n: Pair | undefined, lo: number, hi: number) => {
  if (!n) return true;
  if (n.key <= lo || n.key > hi) return false;
  const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
  const left = ch.filter((c) => c.key < n.key);
  const right = ch.filter((c) => c.key > n.key);
  if (left.length > 1 || right.length > 1) return false;
  return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
};
return roots.every((r) => ok(r, -Infinity, Infinity));
}

```

/\*\* Полный вердикт по текущим линиям. \*/

```

export function gameVerdict(edges: GEdge[], all: Pair[]) {
  const heap = edges.every((e) => e.a.grade > e.b.grade);
  const oneParent = new Set(edges.map((e) => gpid(e.b)));
  const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
  const bst = forestIsBst(edges, all);
  const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.a));
  const complete = edges.length === all.length - 1 && roots.length === 1;
  return { heap, oneParent, acyclic, bst, complete, win: true };
}

```

/\*\* Попытка хода: почему нельзя – или ОК. \*/

```

export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
  if (sameP(a, b)) return { ok: false, reason: "Это одна линия." };
  if (a.grade === b.grade) {
    return { ok: false, reason: "«!» равны – «кто выше?»." };
  }
  const parent = a.grade > b.grade ? a : b;
  const child = a.grade > b.grade ? b : a;
  if (parentOf(edges, gpid(child))) {
    return { ok: false, reason: `У точки ${child.key} уже есть родительская линия.` };
  }
  if (reaches(edges, parent, child)) {
    return { ok: false, reason: "Цикл: линия замкнёт саму себя." };
  }
  return { ok: true, reason: "ОК." };
}

```



-----  
 /\* ----- игра «соедини мышкой»: чистая логика

/\*\* Ребро игры: a = родитель (выше по y), b = ребёнок.

export interface GameEdge {

  a: Pair;

  b: Pair;

}  
 const pid = (p: Pair) => `\${p.key}:\${p.grade}`;

export function canConnect(  
   edges: GameEdge[],  
   x: Pair,  
   y: Pair,  
 ): { ok: boolean; edge?: GameEdge; reason?: string } {

  if (x.key === y.key && x.grade === y.grade) return {

    ok: false,

    reason: "такая линия уже проведена";

  for (const e of edges) {

    if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||

        (pid(e.a) === pid(y) && pid(e.b) === pid(x)))

      return { ok: false, reason: "такая линия уже проведена";

    }

  if (x.grade === y.grade) return { ok: false, reason: "такая линия уже проведена";

  const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];

  if (edges.some((e) => pid(e.b) === pid(child)))

    return { ok: false, reason: `у точки \${child.key} уже есть родитель`;

  // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)

  const childComp = new Set<string>([pid(child)]);

  let grew = true;

  while (grew) {

    grew = false;

    for (const e of edges) {

      if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a)))

        childComp.add(pid(e.a));

      grew = true;

    }

  }

  if (childComp.has(pid(parent))) return { ok: false, reason: "такая линия уже проведена";

  return { ok: true, edge: e; }

  return { ok: false, reason: "такая линия уже проведена";

```

let bstOk = roots.length === 1;
if (bstOk) {
  const seen = new Set<string>();
  const inorder: number[] = [];
  const walk = (p: Pair): void => {
    if (seen.has(pid(p))) {
      bstOk = false; // цикл
      return;
    }
    seen.add(pid(p));
    const kids = children.get(pid(p)) ?? [];
    if (kids.length > 2) {
      bstOk = false;
      return;
    }
    const sorted = [...kids].sort((u, v) => u.key - v.key);
    let left: Pair | null = null;
    let right: Pair | null = null;
    if (sorted.length === 2) {
      if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) {
        bstOk = false; // двое детей с равным x или с
        return;
      }
      left = sorted[0];
      right = sorted[1];
    } else if (sorted.length === 1) {
      // одиночный ребёнок: сторона важна – определяем
      if (sorted[0].key <= p.key) left = sorted[0];
      else right = sorted[0];
    }
    if (left) walk(left);
    inorder.push(p.key);
    if (right) walk(right);
  };
  walk(roots[0]);
  if (seen.size !== n) bstOk = false;
  for (let i = 1; i < inorder.length; i++)
    if (inorder[i] < inorder[i - 1]) bstOk = false;
}

```

```

const [phase, setPhase] = useState<BuildPhase>("sort")
const [algo, setAlgo] = useState<SortAlgo>("none");
const [order, setOrder] = useState<ConnectOrder>("y-d");
const [sortIdx, setSortIdx] = useState(0);
const [connIdx, setConnIdx] = useState(0);
const [playing, setPlaying] = useState(false);
const [sortCost, setSortCost] = useState<string | null>();

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
  const es: { a: Pair; b: Pair }[] = [];
  const walk = (node: TNode | null) => {
    if (!node) return;
    if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left }, b: node.left }); }
    if (node.right) { es.push({ a: { key: node.key, group: node.group, right: node.right }, b: node.right }); }
  };
  walk(userTree);
  return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].filter(e => e.a.group !== e.b.group));

useEffect(() => {
  if (!playing || phase === "game") return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return; }
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.length : canonEdges.length - 1;
      if (connIdx < total) setConnIdx(connIdx + 1);
    }
  }, 1000);
}, [playing, phase, algo, sortIdx, connIdx, order]);

```

```

    });

    const verdict = useMemo(() => checkGameState(userPair));
    const gameWon = phase === "game" && verdict.won;
    const gameTree = useMemo(() => (gameWon ? edgesToTree : null));

    const attemptConnect = (x: Pair, y: Pair) => {
        const r = canConnect(gameEdges, x, y);
        if (r.ok && r.edge) {
            setGameEdges((prev) => [...prev, r.edge!]);
            setGameMsg(r.reason ?? null);
            setBadEdge(null);
        } else {
            setGameMsg(r.reason ?? "так нельзя");
            setBadEdge({ a: x, b: y });
            window.setTimeout(() => setBadEdge(null), 900);
        }
    };

    const pointDown = (p: Pair) => {
        if (phase !== "game") return;
        if (!sel) {
            setSel(p);
            setGameMsg(`Выбрана (${p.key}; ${p.grade}) — теперь...`);
        } else if (sel.key === p.key && sel.grade === p.grade) {
            setSel(null);
            setGameMsg(null);
        } else {
            attemptConnect(sel, p);
            setSel(null);
        }
    };

    const pointUp = (p: Pair) => {
        if (phase !== "game" || !sel) return;
        if (sel.key === p.key && sel.grade === p.grade) return;
        attemptConnect(sel, p);
        setSel(null);
        setCursor(null);
    };

```

```

        {parsed.errors.map((e, i) => <div key={i}>x
        </div>
    )}
    {parsed.errors.length === 0 && (parsed.equalX |
    <div className="mt-2 text-xs text-amber-300 b
    {parsed.equalY && <div>✂ Есть <b>равные y</b>
    кто <b>раньше в списке</b> (сортировка ста
    {parsed.equalX && <div>✂ Есть <b>равные x</b>
    de.key → left).</div>}
    </div>
    )}
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4 l
    <div className="flex flex-wrap items-center jus
    <h4 className="text-base font-bold text-emera
    <div className="flex items-center gap-2">
        {phase === "sort" ? (
            <>
                <span className="text-xs text-slate-400
                {[ "quicksort", "counting" ] as const).m
                <button
                    key={a}
                    onClick={() => { setAlgo(a); setSor
                    Math.round(n * Math.log2(n)) : 0} сравнений
                    disabled={n < 2}
                    className={`px-3 py-1.5 rounded-lg
                    600 text-white" : "bg-slate-700 text-slate-
                >
                    {a === "quicksort" ? "quicksort (сп
                </button>
            )}}
        </>
    ) : (
        <>
            <span className="text-xs text-slate-400
            <button

```

```

    {phase === "sort" ? (
      algo === "none" ? (
        <>Ввод приходит <b>как попало</b> ({n} па
        ть</b>.</>
      ) : sortIdx < 3 ? (
        algo === "quicksort" ? (
          <>✂ quicksort: счётчик сравнений: <b cl
          {comparisons}</b> ( $n \cdot \log_2 n$ ).</>
        ) : (
          <> counting: точки в корзины по значени
          , потому что у – маленькие целые.</>
        )
      ) : (
        <>Отсортировано ({sortCost ?? "способ не
        <b>Игру    </b></>
      )
    ) : phase === "game" ? (
      gameWon ? (
        <> Готово! {hintUsed ? "Решение подсказан
        ево: линии вниз по у, обход по х отсортирован
      ) : gameMsg ? (
        <>Δ {gameMsg}</>
      ) : (
        <>Клики точку-родителя (выше по «!»), за
      )
    ) : connDone ? (
      <> Проведено {connEdges.length} линий в по
      </b>: точки прибиты, дерево одно. А теперь
    ) : (
      <>Линий проведено: {connIdx} из {connEdgesA
    )}
  </div>
</div>

/* Чек-лист инвариантов */
<div className={`rounded-xl p-4 border transition
  er-slate-700 bg-slate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300

```



```

    </div>
  );
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const { l, r } = useMemo(() => splitTree(buildTree(P0), buildTree(P1)), [P0, P1]);
  const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
  const steps = merged.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;
  const cur = steps[idx];

  return (
    <div className="space-y-4">
      <div className="flex items-center gap-3">
        <button onClick={() => setStepIdx(Math.max(0, idx - 1))} className="bg-slate-700 text-white disabled:opacity-30">←
        <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))} className="font-bold bg-indigo-600 text-white disabled:opacity-30">→
        <span className="text-xs text-slate-400">маркер {idx}
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
          <h5 className="text-sm font-bold text-sky-300">Левая часть
          <TreeView root={l} width={480} height={220} nodeRenderer={nodeRenderer} />
        </div>
        <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
          <h5 className="text-sm font-bold text-rose-300">Правая часть
          <TreeView root={r} width={480} height={220} nodeRenderer={nodeRenderer} />
        </div>
      </div>

      <div className="text-xs text-slate-300 bg-slate-900 p-3">
        {cur?.chosenId ? (

```



```

const pathSet = new Set(pathIds);

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить
      <select value={key} onChange={(e) => { setKey(N
        te-700 rounded-lg px-3 py-1.5 text-sm font-mo
        {POINTS.map((p) => <option key={p.key} value=
      </select>
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">шаг {i
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только ве
        />
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <h4 className="text-sm font-bold text-slate-30
        <TreeView root={res.root} subtitle="Валидность
        <ul className="mt-2 space-y-1 text-sm">
          <li> обход слева-направо по-прежнему сортир
          <li> каждое ребро вниз по y (куча целая) –
        </ul>
      </div>
    ))
  </div>
  <div className="text-xs text-slate-300 bg-slate-90

```

```

      text-white" : "bg-slate-700 text-slate-300"}
    </div>

    {pick === "heap" ? {
      <div className="bg-slate-900/40 rounded-xl p-4" >
        <h4 className="text-sm font-bold text-slate-300" >
          4>
        <div className="flex gap-2 overflow-x-auto mb-2" >
          {heapArr.slice(1).map((v, i) => {
            <div key={i} className="px-3 py-2 rounded" >
              <div className="text-slate-500 text-[10px]" >
                <div className="text-white font-bold">{v}</div>
                <div className="text-emerald-400 text-[10px]" >
                  {i}
                </div>
              </div>
            </div>
          </div>
        <p className="text-xs text-slate-400">Полное  $2i/2i+1$ 
          →  $2i/2i+1$  попадает в узел.</p>
      </div>
    ) : {
      <div className="bg-slate-900/40 rounded-xl p-4" >
        <h4 className="text-sm font-bold text-rose-300" >
          5>
        <TreeView root={full} subtitle="Попробуй «посмотреть»" />
        <div className="flex gap-2 overflow-x-auto mt-2" >
          {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => {
            const id = slots[i];
            return (
              <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-950" >
                <div className="text-slate-500 text-[10px]" >
                  {v}
                </div>
                {id ? <div className="text-white font-[10px]" >
                  {id}
                </div> : null}
              </div>
            );
          </div>
        <p className="text-xs text-rose-300 mt-2">Смо

```

```

    const mid = Math.floor((lo + hi) / 2);
    if (sortedByKey[mid].key === target) { steps.push(
      нашли за ${steps.length + 1} сравнение(й).` });
    if (sortedByKey[mid].key < target) { steps.push(
      } — вся левая половина выброшена.` }); lo = mid
    else { steps.push({ lo, hi, mid, found: false, no
      ` }); hi = mid - 1; }
  }
  return steps;
}, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      {stage < 2 && (
        <button onClick={() => { setStage(stage === 0
          digo-600 text-white">
            {stage === 0 ? "Найти y для x=7 бинарным по
          </button>
        )}
      {stage === 2 && (
        <button onClick={() => setBidx(Math.min(bsStep
          -1.5 rounded-lg text-xs font-bold bg-indigo
          Шаг поиска →
        </button>
      )}
      <button onClick={() => { setStage(0); setBidx(0
        C6рос</button>
    </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
  {(stage === 2 ? sortedByKey : shuffled).map((p,
    const highlight = stage === 1 && i === wrongMid
    const inBs = stage === 2 && (() => { const s :
    const isMid = stage === 2 && (() => { const s

```

```

const [sel, setSel] = useState<Pair | null>(null);
const [hover, setHover] = useState<Pair | null>(null);
const [msg, setMsg] = useState<string>("Кликни точку-");

const v = gameVerdict(edges, POINTS);

const click = (p: Pair) => {
  if (!sel) {
    setSel(p);
    setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кликай!`);
    return;
  }
  if (sameP(sel, p)) {
    setSel(null);
    return;
  }
  const res = tryConnect(edges, sel, p);
  if (res.ok) {
    const parent = sel.grade > p.grade ? sel : p;
    const child = sel.grade > p.grade ? p : sel;
    setEdges([...edges, { a: parent, b: child }]);
    setMsg(`Линия (${parent.key}; ${parent.grade})`);
    setSel(null);
  } else {
    setMsg("x " + res.reason);
    setSel(null);
  }
};

const win = v.win;

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-2">
      <button onClick={() => { setEdges(edges.slice(0, edges.length - 1)); setSel(null); }}>
        Удалить последнюю точку
      <button onClick={() => { setEdges([]); setSel(null); }}>
        Сброс
      </div>
    </div>
  );

```

```

        <text x={PX(p.key)} y={PY(p.grade) + 3.5}
space">{p.key}</text>
        <text x={PX(p.key)} y={PY(p.grade) + 24}
    }</text>
    </g>
  );
  }}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
ate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    <li>{v.hear ? " " : " "} каждое ребро – вниз
    <li>{v.oneParent ? " " : " "} у каждой точки
    <li>{v.acyclic ? " " : " "} циклов нет</li>
    <li>{v.bst ? " " : " "} обход слева-направо –
    <li>{v.complete ? " " : " "} линий  $n-1 = \{POI$ 
  </ul>
  {win && {
    <p className="mt-3 text-sm font-bold text-eme
      ения просто не существует.</p>
  }}
</div>

  <div className="text-xs text-slate-300 bg-slate-9
</div>
  );
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "lay

const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },

```

```
        {tab === "build" && <BuildMode />}
        {tab === "split" && <SplitMode />}
        {tab === "merge" && <MergeMode />}
        {tab === "erase" && <EraseMode />}
        {tab === "layers" && <LayersMode />}
        {tab === "search" && <SearchMode />}
        {tab === "game" && <GameMode />}
    </div>
  );
};
```

---

ФАЙЛ 68/82: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import { SplayTreeViz } from "./SplayTreeViz";
```

```

    Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветитс
    Component: SparseTableViz,
  },

```

```

"planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: () => (
  <>
    <ChapterImage vizType="planarity-euler-formula" />
    <PlanarityEulerFormViz />
  </>
),
},
"dijkstra": {
  title: "Дейкстра",
  hint: "Жадно забираем ближайшую вершину и релаксируем",
  Component: () => (
    <>
      <ChapterImage vizType="dijkstra" />
      <DijkstraViz />
    </>
  ),
},
"bellman-ford": {
  title: "Форд-Беллман",
  Component: () => (
    <>
      <ChapterImage vizType="bellman-ford" />
      <BellmanFordViz />
    </>
  ),
},
"floyd": {
  title: "Флойд-Уоршелл",
  Component: () => (
    <>
      <ChapterImage vizType="floyd" />
      <FloydViz />
    </>
  ),
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
  <>
    <ChapterImage vizType="johnson-algo" />
    <JohnsonAlgoViz />
  </>
),
},
"mst-kruskal": { title: "Краскал и DSU", Component: () => (
  <>
    <ChapterImage vizType="mst-kruskal" />
    <KruskalViz />
  </>
),
},
"mst-prima": { title: "Прим", Component: () => (
  <>
    <ChapterImage vizType="mst-prima" />
    <PrimsViz />
  </>
),
},
"mst-boruvka": { title: "Борувка", Component: () => (
  <>
    <ChapterImage vizType="mst-boruvka" />
    <KruskalSimul />
  </>
),
},
"string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
  <>
    <ChapterImage vizType="string-kmp" />
    <KmpViz />
  </>
),
},
"string-z-func": { title: "Z-функция", Component: () => (
  <>
    <ChapterImage vizType="string-z-func" />
    <ZFuncViz />
  </>
),
},
"aho-corasick": {
  title: "Ахо-Корасик",
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
  Component: () => (
    <>
      <ChapterImage vizType="aho-corasick" />
      <AhoCorasickViz />
    </>
  ),
},

```



```

    words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
  0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, d: 100, r: 100,
    // Ветка H -> E -> R -> S
    1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, d: 100, r: 100,
      2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, d: 100, r: 100,
        3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, d: 100, r: 100,
          4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, d: 100, r: 100,
            // Ветка H -> I -> S
            5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d: 100, r: 100,
              6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, d: 100, r: 100,
                // Ветка S -> H -> E
                7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, d: 100, r: 100,
                  8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d: 100, r: 100,
                    9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, d: 100, r: 100,
                      = 2 (HE)
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  };

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training'>('training');

```

```

    }

    const char = textToScan[currentIndex];
    let curr = currentNode;
    let logMsg = `[Символ '${char}']`;

    let jumpType: 'normal' | 'fail' | null = null;
    void jumpType;
    let originalCurr = curr;

    if (TRIE_TRANSITIONS[curr][char] !== undefined) {
        const nextNode = TRIE_TRANSITIONS[curr][char];
        logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
        setJumpPath({ from: curr, to: nextNode, type: 'normal' });
        curr = nextNode;
        setActiveSearchCodeLine(3);
    } else {
        setActiveSearchCodeLine(2);
        let jumps = 0;
        while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
            curr = WATERFALL_NODES[curr].fail;
            jumps++;
        }
        const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
        if (originalCurr === 0) {
            logMsg += `В корне нет перехода по '${char}'. Лист`;
            setJumpPath(null);
        } else {
            logMsg += `Поток по '${char}' у вершины #${originalCurr}`;
            logMsg += `рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label})`;
            setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
            curr = nextNode;
        }
    }

    const currentMatches: { index: number; word: string } = [];

```

```

    },
    {
        targetNodeId: 1,
        title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
        desc: 'Мы обрабатываем первую вершину из очереди -  
        просто не существует. fail[H] автоматически нап
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'H',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 7,
        title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
        desc: 'Обрабатываем дочернюю вершину #7 (S) на Dep
        ,
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 2,
        title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
        desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
        а-разведчик плавает в ROOT и ищет ветку по "E".
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'E',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 5,
        title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
        desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель
    
```

```

        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 9,
        title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С...
        desc: '    МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
            ба плывет в #1 (H) и ищет ветку "E". У вершины :
        codeLine: 4,
        scoutPos: 1,
        inspectedParentFail: 1,
        checkTargetChar: 'E',
        checkingBranchesFrom: 1,
    },
    {
        targetNodeId: 4,
        title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
        desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро...
            #7 (S). fail[HERS] = #7 (S). Автомат полностью
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
};

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTra

```

```

return (
    <div className="bg-slate-800/95 rounded-2xl p-6 border-1" >
        {/* Шапка и переключатель режимов */}
        <div className="flex flex-wrap items-center justify-center" >
            <div className="flex items-center space-x-3" >
                <span className="text-3xl"> </span>
                <div>

```

```

/* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
<div className="grid grid-cols-1 lg:grid-cols-3"
  {/* Управление шагами обучения */}
  <div className="bg-slate-900/90 p-5 rounded-x
    <div>
      <h5 className="text-indigo-300 font-bold"
        <span> </span> Полный обход в ширину (В
      </h5>
      <p className="text-xs text-slate-300 mb-3"
        Показаны все шаги очереди BFS (начиная с
        ар 8 (SHE → HE)</strong>, где наглядно виден
      </p>
      <div className="space-y-1.5 mb-6 overflow
        {TRAINING_STEPS_INFO.map((step, idx) =>
          <button
            key={idx}
            onClick={() => setTrainStep(idx)}
            className={`w-full text-left px-3 p
              ${
                idx === trainStep
                  ? 'bg-indigo-600 text-white fon
                  : 'bg-slate-800 text-slate-400 l
              }`}
            >
              <span className="line-clamp-1">{step
                {idx === trainStep && <span classNa
              </button>
            )}}
          </div>
        </div>

      <div className="flex gap-2">
        <button
          onClick={() => setTrainStep((prev) => M
          disabled={trainStep === 0}
          className="flex-1 bg-slate-700 hover:bg
          ion-all"
        >

```

```

        </div>
        <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
        <span>3. while (v !== root && trie[v]
        {currentTrainInfo.codeLine === 3 && <
т, возврат в корень</span>}
        </div>
        <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
        <span>4. if (trie[v][char] !== undefin
/span>
        {currentTrainInfo.codeLine === 4 && <
ан IF!</span>}
        </div>
        <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
        <span>5. else fail[u] = root; // Bero
        {currentTrainInfo.codeLine === 5 && <
root</span>}
        </div>
    </div>
</div>

<div>
    <h5 className="text-slate-200 font-bold m
    <Lightbulb className="w-4 h-4 text-ambe
    </h5>
    <div className="bg-slate-800 p-4 rounded-:
w-inner min-h-[72px] flex items-center">
        {currentTrainInfo.desc}
    </div>
</div>
</div>
</div>
) : (
/* ПАНЕЛЬ РЕЖИМА ПОИСКА */
<div className="grid grid-cols-1 lg:grid-cols-3
    <div className="bg-slate-900/60 p-5 rounded-x

```

```

        </span>
      })
    </div>

    <button
      onClick={stepSearchSimulation}
      disabled={isSearchDone || textToScan.length === 0}
      className={`w-full py-2.5 rounded-lg font-medium transition-colors
        ${isSearchDone || textToScan.length === 0
          ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
          : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
      `}
    >
      <span>{currentIndex === 0 ? 'Начать план' : 'Продолжить'}</span>
      <ArrowRight className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
  <div>
    <h5 className="text-white font-bold mb-3">
      <span>✂</span> Текущее действие в коде
    </h5>
    <div className="bg-slate-950 p-4 rounded-2xl">
      <div className={`p-1.5 rounded flex items-center justify-between border border-blue-500 text-blue-300 font-bold`} >
        <span>1. curr = state; char = text[i]
        {activeSearchCodeLine === 1 && <span>инициализация</span>}
      </div>
      <div className={`p-1.5 rounded flex items-center justify-between border border-rose-500 text-rose-300 font-bold`} >
        <span>2. while (curr !== root && !trie[curr].next)
        {activeSearchCodeLine === 2 && <span>прыжок по fail!</span>}
      </div>

```

```
</div>
```

```
<div className="flex items-center justify-between">
  <h5 className="text-white font-bold text-base">
    <span> </span> Ветвящийся синий водопад (Борис)
  </h5>
```

```
<div className="flex flex-wrap gap-4 text-xs">
  <span className="flex items-center gap-1 text-white">
    <span className="w-3 h-0.5 bg-blue-500 inline-block">
    </span>
    <span className="flex items-center gap-1 text-white">
      <span className="w-3 h-0.5 border-t border-blue-500">
      </span>
    </span>
  </div>
</div>
```

```
{/* Само SVG дерево водопада */}
```

```
<div className="w-full overflow-x-auto custom-scrollbar">
  <svg viewBox="0 0 800 520" className="w-full h-full">
```

```
    {/* Горизонтальные линии и подписи уровней */}
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
      { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
      { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
      { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
      { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
    ].map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        >
```



```

        <text
            x={{fromNode.x + toNode.x} / 2}
            y={{fromNode.y + toNode.y} / 2 +
            fill={isTrainingExamined ? (isMatch
            fontSize="12"
            fontWeight="bold"
            textAnchor="middle"
        >
            {char}
        </text>

        {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
        {isTrainingExamined && (
            <g transform={`translate(${(fromNode
.y) / 2)}`}>
                <rect x="-15" y="-12" width="30
strokeWidth="2" />
                <text x="0" y="5" fontSize="14"
                    {isMatchBranch ? ' ' : ' '}
                </text>
            </g>
        )}
        </g>
    );
    });
    })}

```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
    if (node.id === 0) return null;
    const targetNode = WATERFALL_NODES[node.f

```

```

// В режиме обучения показываем fail-ссыл
// Найдем индекс шага, на котором обработ
const stepIdxForNode = TRAINING_STEPS_INF
if (activeMode === 'training' && trainStep

```

```

let isHighlighted = activeMode === 'search

```

```

<g key={`node-${node.id}`} className="с
  {/* Внешнее свечение для активной вер
  {isCurrent && (
    <circle cx={node.x} cy={node.y} r="
  )}
  {isTargetChild && (
    <circle cx={node.x} cy={node.y} r="
  )}

  {/* Основной круг вершины */}
  <circle
    cx={node.x}
    cy={node.y}
    r={isCurrent || isTargetChild ? '24'
    fill={isCurrent ? '#2563eb' : isTar
    stroke={isCurrent ? '#ffffff' : isTa
    strokeWidth={isCurrent || isTargetCh
    className="transition-all duration-
  />

  {/* Текст внутри вершины */}
  <text
    x={node.x}
    y={node.y + 5}
    fill={isCurrent || isTargetChild ?
    fontSize={isCurrent || isTargetChil
    fontWeight="bold"
    textAnchor="middle"
  >
    {node.label}
  </text>

  {/* Метка искомой вершины в режиме об
  {isTargetChild && (
    <g transform={`translate(${node.x},
      <rect x="-35" y="-12" width="70"
      <text x="0" y="2" fontSize="10" f

```

ИЩЕМ FAIL

```

        >
        {activeMode === 'training' ? ' ♂ ' :
        </text>
        </g>

        </svg>
        </div>
    </div>

    /* Список найденных совпадений (только в режиме
    {activeMode === 'search' && (
        <div className="mt-6 bg-slate-900/50 p-5 rounded
            <h5 className="text-white font-bold mb-3 text
                <span> </span> Улов лосося (Найденные слова
            </h5>
            {foundMatches.length === 0 ? (
                <p className="text-sm text-slate-400 italic
                    Пока ничего не поймали. Запустите шаги си
                </p>
            ) : (
                <div className="flex flex-wrap gap-2">
                    {foundMatches.map((match, idx) => (
                        <div
                            key={idx}
                            className="bg-emerald-950 border bord
                                -sm font-bold shadow animate-[scaleUp_0.3s_
                            >
                                <CheckCircle2 className="w-4 h-4 text
                                <span>{match.word}</span>
                                <span className="text-[10px] bg-emera
                                    Индекс {match.index}
                                </span>
                            </div>
                        </div>
                    )}}
                </div>
            )}
        </div>
    )}

```

```
export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  return (
    <div className="w-full bg-slate-950 rounded-2xl border-2 border-indigo-500">
      <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
        {(
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        ).map(([key, label]) => (
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm transition-colors
              ${mode === key ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300"}
            `}
          >
            {label}
          </button>
        ))}
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};
```

```
/* ----- 1D ----- */
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[acc.length - 1] + v],
    []);
};
```

```
/** Что сейчас под курсором: префикс P[i] или элемент A[i] */
const [hover, setHover] = useState<{ kind: "pref" | "elem", i: number}>({ kind: "pref", i: 0 });
const [range, setRange] = useState<{ l: number; r: number}>({ l: 0, r: 0 });
```

```

        : inRange
        ? "bg-indigo-900/70 text-indigo
        : "bg-slate-800 text-slate-300
    }`}
    title={`a[${i}] = ${v}`}
  >
    {v}
  </div>
);
}}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right"
  {pref.map(({v, i}) => {
    const active = hover?.kind === "pref" &&
    const isL = i === range.l;
    const isR = i === range.r + 1;
    return (
      <div
        key={i}
        onMouseEnter={() => setHover({ kind:
        onMouseLeave={() => setHover(null)}
        className={`${cellBase} cursor-help $
          active
            ? "bg-emerald-500 text-white ring
            : isR
              ? "bg-emerald-700 text-white"
            : isL
              ? "bg-rose-700 text-white"
            : "bg-slate-900 text-slate-300
          }`}
        title={`P[${i}] = cymma a[0..${i} - 1]
      >
        {v}
      </div>
    );
  })}

```

```

    })
  </div>

  {/* Запрос */}
  <div className="bg-slate-900 border border-indigo
    <div className="text-xs text-slate-400 mb-2">
      Запрос суммы на отрезке (кликайте по массиву
    </div>
    <p className="font-mono text-sm sm:text-base te
      sum(a[{range.l}..{range.r}]) = P[{range.r + 1
      <span className="text-emerald-400">{pref[range
      <span className="text-rose-400">{pref[range.l
      <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
      Один вычитание вместо цикла: запрос за  $O(1)$  п
    </p>
  </div>
</div>
  );
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
  const n = A2.length;
  const m = A2[0].length;

  const P = useMemo(() => {
    const p = Array.from({ length: n + 1 }, () => Array
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i
      }
    }
    return p;
  }, [n, m]);

  const [hover, setHover] = useState<{ i: number; j: num

```

```

        >
          {v}
        </div>
      );
    }}}
  </div>
)}}
</div>
</div>

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
  <div className="text-xs text-slate-400 mb-2">
    <div className="inline-block">
      {P.map((row, i) => {
        <div key={i} className="flex gap-1.5 mb-1">
          {row.map((v, j) => {
            const corner = cornerOf(i, j);
            const isHover = hover?.i === i && hover?.j === j;
            const cornerColor =
              corner === "A"
                ? "bg-emerald-600 text-white"
                : corner === "D"
                ? "bg-sky-600 text-white"
                : corner === "C"
                ? "bg-rose-600 text-white"
                : "bg-slate-900 border border-slate-500";
            return (
              <div
                key={j}
                onMouseEnter={() => setHover({ i, j })}
                onMouseLeave={() => setHover(null)}
                className={` ${cellBase} cursor-help ${
                  isHover ? "bg-amber-500 text-white" : ""
                }`}
              >
                {v}
              </div>
            );
          })}
        >
          {v}
        </div>
      )}
    </div>
  </div>

```

```
    });  
};
```

---

ФАЙЛ 71/82: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРӨК: 709 | РАЗМ

---

```
import React, { useState } from "react";  
import { motion, AnimatePresence } from "motion/react";  
import {  
  Play,  
  RotateCcw,  
  ChevronRight,  
  ChevronLeft,  
  Lock,  
  ChevronDown,  
} from "lucide-react";
```

```
// Data for 1D Array  
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];  
const N = arr1D.length;  
const LOG = 4;
```

```
const st1D: number[][] = Array.from({ length: N }, () => []);  
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];  
for (let j = 1; j < LOG; j++) {  
  for (let i = 0; i + (1 << j) <= N; i++) {  
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);  
  }  
}
```

```
// Data for 2D Matrix (Sparse Table 2D)  
const val2D = [  
  [45, 12, 88, 34, 11, 76, 23, 90],  
  [67, 19, 44, 55, 33, 21, 65, 87],  
  [14, 51, 99, 13, 22, 64, 43, 76],  
];
```



```

    }
  }
}

```

```

export const SparseTableViz: React.FC = () => {
  const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d")

  return (
    <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
      <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
        <button
          onClick={() => setTab("1d")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : `bg-slate-800 text-slate-400`}
        >
          1. Сворачивание 1D
        </button>
        <button
          onClick={() => setTab("2d_build")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : `bg-slate-800 text-slate-400`}
        >
          2. Сворачивание 2D (Построение)
        </button>
        <button
          onClick={() => setTab("2d_query")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : `bg-slate-800 text-slate-400`}
        >
          3. Запрос 2D (Формула)
        </button>
      </div>

      <AnimatePresence mode="wait">
        {tab === "1d" ? (
          <Viz1D key="1d" />
        ) : tab === "2d_build" ? (
          <Viz2DBuild key="2d_build" />
        ) : tab === "2d_query" ? (
          <Viz2DQuery key="2d_query" />
        ) : null}
      </AnimatePresence>
    </div>
  )
}

```

```

        disabled={step === 0}
      >
        <ChevronLeft size={20} />
      </button>
      <button
        onClick={() => setStep(Math.min(maxStep, step + 1))}
        className="p-2 bg-indigo-600 hover:bg-indigo-700"
        disabled={step === maxStep}
      >
        <ChevronRight size={20} />
      </button>
      <button
        onClick={() => setStep(0)}
        className="p-2 bg-slate-800 hover:bg-slate-900"
      >
        <RotateCcw size={20} />
      </button>
    </div>
  </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800" >
    <div className="text-[11px] uppercase tracking-wider" >
      <div className="flex gap-1 sm:gap-1.5 overflow-x-auto" >
        {arr1D.map((v, idx) => {
          const inCover = !!covered && idx >= covered - 1;
          return (
            <div key={idx} className="flex flex-col items-center justify-center" >
              <div
                className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(9px,1.2em)]}
                ${inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-400"}
              >
                {v}
              </div>
              <span className="text-[9px] text-slate-400" >
                {v}
              </span>
            </div>
          );
        })}
      </div>
    </div>
  </div>

```



```

                                : "bg-transparent to
                                }` }
                                >
                                {stlD[i][j]}
                                </motion.div>
                                }}

                                { /* Arrows visualization */ }
                                {(isSrc1 || isSrc2) && (
                                <motion.svg
                                initial={{ opacity: 0 }}
                                animate={{ opacity: 1 }}
                                className="absolute pointer-e
                                style={{
                                width: 100,
                                height: isSrc2 ? 40 + (1 <<
                                right: -50,
                                top: 24,
                                overflow: "visible",
                                }}
                                >
                                <path
                                d={`M 0 0 C 30 0, 30 ${isSrc
                                fill="none"
                                stroke={isSrc1 ? "#10b981"
                                strokeWidth="3"
                                />
                                </motion.svg>
                                }}
                                </div>
                                );
                                }}}
                                </React.Fragment>
                                )))
                                </div>
                                </div>
                                </div>

```



```

        if (isTop && isLeft) {
            highlightClass = 'bg-rose';
        } else if (isTop && !isLeft) {
            highlightClass = 'bg-blue';
        } else if (!isTop && isLeft) {
            highlightClass = 'bg-emerald';
        } else {
            highlightClass = 'bg-amber';
        }
        md:scale-[1.08]';
        zIndex = 'z-10';
    }
}

return (
    <div
        key={idx}
        onMouseEnter={() => { if(isCurr) setCurr(idx); }}
        onMouseLeave={() => { if(isCurr) setCurr(-1); }}
        onClick={() => {
            if(isCurr) {
                if(locked && locked) return;
                else setLocked({r, c});
            }
        }}
        className={`flex items-center justify-center gap-2`}
        style={{width: clamp(24px, 6.5vw, 38px), height: clamp(20px, 5.2vw, 30px), text-align: center}}
    >
        {ST2D[r][c][step][step]}
    </div>
);
}}}
</div>

```

```
="text-amber-300">1</span>][k-<span className=
span><br/>
```

```
    &nbsp;&nbsp;&nbsp;<span className="text-amber-300 text-white">c + pL</span>][k-<span className="text-amber-300 text-white">c + pL</span>][k-<span className="text-slate-500">// Правый Низ</span><br/>
    {'}'}));
</div>
```

```
    {activeCell ? (
      <div className="bg-slate-950 p-5 rounded-2xl mt-2 transition-all">
        <p className="text-slate-300 mb-3">
          <span>Пример для ячейки:</span>
          {locked && <span className="text-slate-500 border border-rose-500/20"><Lock size={
            </p>
            <p className="text-indigo-300 border border-rose-500/20">
              <span className="font-bold text-white">
                }}[k]][k]][k]</span>
              <span className="text-slate-400">
                </p>

```

```
      <div className="space-y-2 pt-3 pl-2">
        {(() => {
          const prevL = 1 << (k - 1);
          return (
            <>
              <div className="flex items-center justify-between shadow-[0_0_8px_#f43f5e]"></div> ST[{activeCell.r}[activeCell.c][k-1][k-1]}</span>
              </div>
              <div className="flex items-center justify-between shadow-[0_0_8px_#3b82f6]"></div> ST[{activeCell.r}[activeCell.c + prevL][k-1][k-1]}</span>
            </>
          )
        }
      )
    )
  }
}

```

```
const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);

  const H = r2 - r1 + 1;
  const W = c2 - c1 + 1;
  const kx = Math.floor(Math.log2(H));
  const ky = Math.floor(Math.log2(W));
  const Lx = 1 << kx;
  const Ly = 1 << ky;

  const matRows = 8 - Lx + 1;
  const matCols = 8 - Ly + 1;

  return (
    <div className="flex flex-col xl:flex-row gap-6">
      <div className="flex-1 min-w-0 bg-slate-900">
        <h3 className="text-xl font-bold text-rose-500">
          <p className="text-slate-400 text-[13px] font-normal">
            r1, c1 - левый верх, r2, c2 - правый низ).</p>
        </h3>
      </div>
      <div className="flex flex-col items-center justify-center">
        <div className="grid grid-cols-[repeat(8, 1fr)] gap-2.4vw rounded-2xl shadow-inner w-max max-w-full">
          {Array.from({length: 64}).map((_, idx) => {
            const r = Math.floor(idx / 8);
            const c = idx % 8;
            const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;

            return (
              <div key={idx} className={`flex flex-col items-center justify-center rounded text-[clamp(9px,2.4vw,12px)] ${isInQuery ? 'text-rose-500' : 'text-slate-400'} border-rose-500/50 border scale-105 z-10`}>
                {val2D[r][c]}
              </div>
            )
          })}
        </div>
      </div>
    </div>
  )
}
```



```

        <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
        </div>
      </label>
      <label className="flex items-cen
        <span className="text-slate-
        <div className="flex gap-2">
          <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
          <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
          </div>
        </label>
      </div>
    </div>
  </div>
</div>

<div className="flex-1 bg-slate-900 p-6 rou
  <h3 className="text-xl font-bold text-em
  <p className="text-sm font-sans text-sla
    Мы знаем, что максимальная степень дв
    rounded text-white">{Lx}</span>. Аналогично
    te">{Ly}</span>.
  </p>
  <p className="text-sm font-sans text-sla
    Чтобы покрыть весь прямоугольник, мы
    Name="font-mono bg-[#0a0fle] px-1 rounded b
    роса. (Цветные прямоугольники слева).
  </p>

  <div className="bg-[#0a0fle] rounded-xl
text-slate-300 shadow-inner overflow-x-auto
  <div className="absolute right-3 top-3
  <span className="text-slate-500">// 1.
  <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
  <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="

```

```

        const r = Math.floor(idx / matCols);
        const c = idx % matCols;
        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;

        let color = "bg-slate-800 text-white";
        let txt = ST2D[r][c][kx][ky].text;

        let badge = null;
        if (isTL) { color = "bg-rose-500 text-white"; }
        else if (isTR) { color = "bg-lime-300 text-white"; }
        else if (isBL) { color = "bg-emerald-300 text-white"; }
        else if (isBR) { color = "bg-amber-300 text-white"; }

        return (
            <div key={idx} className={color}
            transition-all duration-300 relative shrink-0 ${cx} ${cy}
                {txt}
                {badge && <div className="text-white font-bold rounded-sm px-1 py-1"
                r-slate-700 text-white px-1 font-bold rounded-sm border-1 border-white
                </div>}
            )
        )
    }
}
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white text-center py-3"
    >
    <div className="text-center font-bold text-white"
    wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border-1 border-white
        <span className="text-slate-400 font-sans text-sm">

```

```
document.getElementById('btn-mode-wtf').className =
  mode === 'wtf' ? 'px-3 py-1 rounded text-sm font-bold' : 'px-3 py-1 rounded text-sm font-bold';

const aside = document.querySelector('aside');
const headerMain = document.querySelector('header.main');
const questionsContainer = document.getElementById('questions-container');
const wtfContainer = document.getElementById('wtf-container');

if (mode === 'wtf') {
  document.body.classList.add('wtf-mode');
  if (aside) aside.style.display = 'none';
  if (headerMain) headerMain.style.display = 'none';
  if (questionsContainer) questionsContainer.style.display = 'none';
  if (wtfContainer) wtfContainer.classList.remove('hidden');

  // Remove the URL hash to avoid scrolling
  history.pushState("", document.title, window.location.pathname + window.search);
  window.scrollTo(0, 0);
} else {
  document.body.classList.remove('wtf-mode');
  if (aside) aside.style.display = '';
  if (headerMain) headerMain.style.display = '';
  if (questionsContainer) questionsContainer.style.display = '';
  if (wtfContainer) wtfContainer.classList.add('hidden');
}

function setTheme(theme) {
  const body = document.body;
  body.classList.remove('theme-dark', 'theme-light');
  body.classList.add('theme-' + theme);

  // Just some visual feedback on buttons
  document.querySelectorAll('[id^="theme-"]').forEach(btn => {
    btn.classList.remove('outline', 'outline-1');
  });
}
```

```

        if (drawerOverlay) {
            drawerOverlay.classList.add("open");
            setTimeout(() => drawerOverlay.classList.remove("open"), 300);
        }
        document.body.style.overflow = "";
    }
}

if (mobileMenuBtn && mobileDrawer) {
    mobileMenuBtn.addEventListener("click", () => {
        toggleDrawer();
    });
}
if (closeDrawerBtn && mobileDrawer) {
    closeDrawerBtn.addEventListener("click", () => {
        toggleDrawer();
    });
}
if (drawerOverlay) {
    drawerOverlay.addEventListener("click", () => {
        toggleDrawer();
    });
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
    link.addEventListener("click", () => {
        if (mobileDrawer && !mobileDrawer.classList.contains("open")) {
            toggleDrawer();
        }
    });
});

const observerOptions = {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: 0
};

const observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
        if (entry.isIntersecting && !document.querySelector("#mobile-drawer a.active")) {
            // Highlight active TOC link location
            const id = entry.target.id;
            document.querySelector(`a[href="#${id}"]`).classList.add("active");
        }
    });
});

```

```

        displayMath: [['$$', '$$'], ['\\[', '\\[
    }
  };
</script>
<style>
  @import url('https://fonts.googleapis.com/css2?
  @reference "../src/index.css";
  .math-font { font-family: 'Fira Code', monospace;
  .uno-card {
    width: 70px; height: 105px; border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
    align-items: center; justify-content: center; text-align: center;
    color: white; overflow: hidden; flex-shrink: 0;
    text-shadow: 2px 2px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
    font-family: 'Arial Black', Impact, sans-serif;
  }
  .uno-card::before, .uno-card::after {
    content: attr(data-val); position: absolute; top: 50%; left: 50%;
    text-shadow: 1px 1px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
  }
  .uno-card::before { top: 2px; left: 4px; }
  .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
  .uno-card.mini { width: 45px !important; height: 60px !important; }
  .uno-card.mini::before, .uno-card.mini::after {
    font-size: 0.8em;
  }

  /* Global formula scroll settings */
  .formula-scroll {
    overflow-x: auto;
    overflow-y: hidden;
    -webkit-overflow-scrolling: touch;
    scrollbar-width: none; /* Firefox */
    -ms-overflow-style: none; /* IE and Edge */
  }
  .formula-scroll::-webkit-scrollbar {
    display: none;
  }

  mjx-container {

```

```

.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-
.card-blue { background-color: #004dcf; } .card
.card-green { background-color: #339e35; } .card
.card-yellow { background-color: #ffc400; } .ca
.card-black { background-color: #222; } .card-b
.card-black::before, .card-black::after { text-
.info-block { @apply bg-slate-800/80 border-l-4
.info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
  ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
  king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
  r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
  background-color: #f8fafc !important;
  color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-ligh
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-ligh
  important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
  background-color: #fcf8eb !important; /* li
  color: #3b2818 !important; /* brown ink */
  background-image: repeating-linear-gradient

```

```

    }
  </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

  <div class="sticky top-0 z-50 w-full bg-slate-950/90
    center items-center gap-4 sm:gap-8 transition-colors">
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase text-slate-400">Тема</span>
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
        <button onclick="setMode('normal')" id="btn-normal"
          transition-colors">Список тем</button>
        <button onclick="setMode('wtf')" id="btn-wtf"
          transition-colors">WTF</button>
      </div>
    </div>
    <div class="flex items-center gap-3">
      <span class="text-sm font-bold uppercase text-slate-400">Тема</span>
      <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
        <button onclick="setTheme('dark')" id="btn-dark"
          transition-colors">Темная</button>
        <button onclick="setTheme('light')" id="btn-light"
          transition-colors">Светлая</button>
        <button onclick="setTheme('notebook')" id="btn-notebook"
          transition-colors">Notebook</button>
      </div>
    </div>
  </div>

  <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
  <svg width="0" height="0" class="absolute">
    <defs>
      <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
        color="yellow" orient="auto">
        <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d" />
      </marker>
      <marker id="arrow-orange" markerWidth="6" markerHeight="6"
        color="orange" orient="auto">
        <path d="M0,0 L6,3 L0,6 Z" fill="#f97316" />
      </marker>
    </defs>
  </svg>

```

```

        <rect x="5" y="40" width="40" height="20" fill="#333" />
        <!-- Ручка -->
        <path d="M 45 50 Q 60 50 60 30 L 75 30" stroke="#333" stroke-width="2" />
        <circle cx="75" cy="30" r="4" fill="#333" />
        <!-- Выход -->
        <path d="M 5 45 L -5 45 L -5 55 L 5 55" stroke="#333" stroke-width="2" />
        <circle cx="25" cy="50" r="15" fill="#fff" />
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="10" fill="#333" />
        <rect x="3" y="5" width="14" height="5" fill="#333" />
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20" fill="#fff" />
        <rect x="6" y="-5" width="8" height="6" fill="#333" />
        <rect x="26" y="-5" width="8" height="6" fill="#333" />
        <line x1="0" y1="2" x2="40" y2="2" stroke="#333" stroke-width="2" />
        <text x="20" y="14" fill="#475569" font-weight="bold">
            ЛЕГО
        </text>
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed top-0 right-0 p-4 z-50 hover:bg-blue-500 transition-transform duration-300">
    <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" />
    </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню -->
<div id="drawer-overlay" class="lg:hidden fixed inset-0 bg-black bg-opacity-50 z-40">
    </div>

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-direction-column">
    <!-- Левое меню (Оглавление) -->
    <aside id="mobile-drawer" class="fixed inset-y-0 left-0 z-50 w-64 p-4 transition-transform duration-300">

```



```
<div class="pl-6 space-y-2 text
  <a href="#section-2-1" class="b
  <a href="#section-2-2" class="b
  <a href="#section-2-3" class="b
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-3" class=
er-teal-500 pl-3 font-bold text-teal-300">
    3. Тригонометрическая форма
  </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#section-3-1" class="b
    <a href="#section-3-2" class="b
    <a href="#section-3-3" class="b
    <a href="#section-3-4" class="b
  </div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-4" class=
er-amber-500 pl-3 font-bold text-amber-300">
    4. Степени и корни
  </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q4-def" class="block l
    <a href="#q4-moivre" class="blo
    <a href="#q4-root" class="block
    <a href="#q4-count" class="bloc
  </div>
</details>

<details class="group">
```

```

        </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q7-1" class="bloc
        <a href="#q7-2" class="bloc
        <a href="#q7-3" class="bloc
        <a href="#q7-4" class="bloc
        <a href="#q7-5" class="bloc
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-8" class:
er-violet-500 pl-3 font-bold text-violet-300">
        8. Сравнимость и Поля В
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q8-1" class="bloc
        <a href="#q8-2" class="bloc
        <a href="#q8-3" class="bloc
        <a href="#q8-4" class="bloc
        <a href="#q8-5" class="bloc
        <a href="#q8-6" class="bloc
        <a href="#q8-7" class="bloc
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-9" class:
er-green-500 pl-3 font-bold text-green-400">
        9. Кольцо многочленов
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q9-1" class="bloc
```

```
        </div>
      </details>

      <details class="group">
        <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
          12. Взаимно простые мно
        </a>
        </summary>
        <div class="pl-6 space-y-2 text-blue-500">
          <a href="#q12-1" class="block">
          <a href="#q12-2" class="block">
          <a href="#q12-3" class="block">
            еление)</a>
          </div>
        </details>

      <details class="group">
        <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
          13. Корни многочленов.
        </a>
        </summary>
        <div class="pl-6 space-y-2 text-emerald-500">
          <a href="#q13-1" class="block">
          <a href="#q13-2" class="block">
          <a href="#q13-3" class="block">
            </div>
        </details>

      <details class="group">
        <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
          14. Кратность корня. Ко
        </a>
```

```

        </details>

        <div class="mt-4 mb-2 text-xs font-l
        <a href="#question-17" class="block
-500 pl-3 text-slate-400">
            <span class="font-bold text-tea
        </a>
        <a href="#question-18" class="block
-500 pl-3 text-slate-400">
            <span class="font-bold text-blur
        </a>
        <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
            <span class="font-bold text-eme
        </a>
        <a href="#question-20" class="block
-500 pl-3 text-slate-400">
            <span class="font-bold text-ros
        </a>
        <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
            <span class="font-bold text-ind
        </a>
        <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
            <span class="font-bold text-fuch
        </a>

        <a href="#proof-algorithms" class="l
-red-500 pl-3 mt-4 text-red-500 font-bold m
        ▲ Ликбез: Алгоритмы выживания
        </a>

    </nav>
    <script>
        document.querySelectorAll('details.
        detail.addEventListener('toggle
            if (detail.open) {

```

```

        // Open the parent
        const correspondingLi =
        if (correspondingLi) {
            const details =
            if (details &&
                details.open) {
                document.querySelector(
                    if (other) {
                        other
                    }
                }
            }
        }
    }
}
});
}, {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: [0, 0.5]
});

sections.forEach(section => {
    observer.observe(section);
});
});

function fallbackModal(contentRaw,
    const modal = document.createElement('div');
    modal.style.position = 'fixed';
    modal.style.top = '0';
    modal.style.left = '0';
    modal.style.width = '100vw';
    modal.style.height = '100vh';
    modal.style.backgroundColor = 'black';
    modal.style.zIndex = '99999';
    modal.style.display = 'flex';
    modal.style.flexDirection = 'column';

```

```

copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = 'white';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.writeText(textarea.value);
        copyBtn.innerText = 'Скопировано';
    } else {
        textarea.select();
        document.execCommand('copy');
        copyBtn.innerText = 'Скопировано';
    }
    setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = 'white';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
    btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);

```

```

function setTheme(theme) {

    const body = document.body;
    body.classList.remove('bg-slate-500');

    document.getElementById('theme-dark');
    document.getElementById('theme-light');
    document.getElementById('theme-style');

    let oldStyle = document.getElementById('theme-style');
    if (oldStyle) oldStyle.remove();
    const style = document.createElement('style');
    style.id = 'theme-style';

    if (theme === 'dark') {
        body.classList.add('bg-slate-500');
        document.getElementById('theme-dark').innerHTML = `
            body.bg-slate-500 section header { color: white; }
            body.bg-slate-500 main { color: white; }
            body.bg-slate-500 main p { color: white; }
            /* Universal text color */
            body.bg-slate-500 .text-slate-500 { color: white; }
            body.bg-slate-500 .text-slate-400 { color: white; }
            body.bg-slate-500 .text-slate-300 { color: white; }
            body.bg-slate-500 [class*="text-slate-"] { color: white; }

            /* Specific backgrounds */
            body.bg-slate-500 aside > div { background-color: white; }
            body.bg-slate-500 [class*="bg-slate-"] { background-color: white; }
        `;
    } else if (theme === 'light') {
        body.classList.add('bg-white');
        document.getElementById('theme-light').innerHTML = `
            body.bg-white section header { color: black; }
            body.bg-white main { color: black; }
            body.bg-white main p { color: black; }
            /* Universal text color */
            body.bg-white .text-slate-500 { color: black; }
            body.bg-white .text-slate-400 { color: black; }
            body.bg-white .text-slate-300 { color: black; }
            body.bg-white [class*="text-slate-"] { color: black; }

            /* Specific backgrounds */
            body.bg-white aside > div { background-color: black; }
            body.bg-white [class*="bg-slate-"] { background-color: black; }
        `;
    }

    style.innerHTML = `
        body.bg-white section header { color: black; }
        body.bg-white main { color: black; }
        body.bg-white main p { color: black; }
        /* Universal text color */
        body.bg-white .text-slate-500 { color: black; }
        body.bg-white .text-slate-400 { color: black; }
        body.bg-white .text-slate-300 { color: black; }
        body.bg-white [class*="text-slate-"] { color: black; }

        /* Specific backgrounds */
        body.bg-white aside > div { background-color: black; }
        body.bg-white [class*="bg-slate-"] { background-color: black; }
    `;

    document.getElementById('theme-style').innerHTML = style.innerHTML;
}

```

```

        `;
    } else if (theme === 'terminal') {
        body.classList.add('bg-black');
        document.getElementById('theme-label').innerHTML = `
            body.bg-black .bg-slate-950 {
                background-color: transparent;
            }
            body.bg-black section header {
                color: #4ade80 !important;
            }, body.bg-black button {
                color: #4ade80 !important;
                border-color: #22c55e !important;
                font-family: monospace;
            }
            body.bg-black svg path,
            body.bg-black .bg-slate-950 {
                stroke: #22c55e !important;
                fill: transparent !important;
            }
        `;
    }
}

document.head.appendChild(style);
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border-1" style="border-radius: 10px; padding: 10px;">
    <!-- Кнопки Конвертации / Экспорта
    <div class="flex flex-col gap-2">
        <div class="text-xs text-slate-400">
            <button onclick="downloadWord()">
                <span> Word </span>
            </button>
        </div>
    </div>

```



```
<h1 class="text-4xl md:text-5xl font-bl
4">
    ВЫСШАЯ АЛГЕБРА
</h1>
<p class="text-slate-400 italic text-lg

<!-- Print Table of Contents -->
<div class="hidden print:block mt-8 tex
fter: always;">
    <h2 class="font-bold text-xl mb-4">
    <ul class="text-base space-y-2">
        <li><a href="#question-1">1. Ал
        <li><a href="#question-2">2. Ко
        <li><a href="#question-3">3. Тр
        <li><a href="#question-4">4. Ст
        <li><a href="#question-5">5. Гр
        <li><a href="#question-6">6. Де
        <li><a href="#question-7">7. Ал
        <li><a href="#question-8">8. Кл
        <li><a href="#question-9">9. Ко
        <li><a href="#question-10">10.
        <li><a href="#question-11">11.
        <li><a href="#question-12">12.
        <li><a href="#question-13">13.
        <li><a href="#question-14">14.
        <li><a href="#question-15">15.
        <li><a href="#question-17">17.
        <li><a href="#question-18">18.
        <li><a href="#question-19">19.
        <li><a href="#question-20">20.
        <li><a href="#question-21">21.
        <li><a href="#question-22">22.
    </ul>
</div>
</header>

<!-- ===== WTF CONTAINER =====
<div id="wtf-container" class="hidden">
```

```

        <div class="font-bold text-slate-500">
      </a>
      <a href="#question-3" onclick="setMainQuestion(3)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">
          <div class="font-bold text-slate-500">
        </a>

      <!-- Row 3: НОД и НОК / Алгоритм Евклида -->
      <a href="#question-7" onclick="setMainQuestion(7)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-yellow-500">
          <div class="font-bold text-slate-500">
        </a>
      <a href="#question-12" onclick="setMainQuestion(12)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
          <div class="font-bold text-slate-500">
        </a>
      <!-- Empty 3rd row cell -->
      <div class="hidden md:block"></div>

      <!-- Row 4: Факторизация / Структурная теория -->
      <a href="#question-8" onclick="setMainQuestion(8)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-yellow-500">
          <div class="font-bold text-slate-500">
        </a>
      <a href="#question-11" onclick="setMainQuestion(11)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
          <div class="font-bold text-slate-500">
        </a>
      <a href="#question-5" onclick="setMainQuestion(5)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">
          <div class="font-bold text-slate-500">
        </a>

```

```

        <div class="text-xs text-teal-500">
        <div class="font-bold text-slate-700">
    </a>
    <a href="#question-18" onclick="setQuestion(18)">
- l-4 border-blue-500 hover:bg-slate-700 transition
        <div class="text-xs text-blue-500">
        <div class="font-bold text-slate-700">
    </a>
    <a href="#question-19" onclick="setQuestion(19)">
- l-4 border-emerald-500 hover:bg-slate-700 transition
        <div class="text-xs text-emerald-500">
        <div class="font-bold text-slate-700">
    </a>

    <a href="#question-20" onclick="setQuestion(20)">
- l-4 border-rose-500 hover:bg-slate-700 transition
        <div class="text-xs text-rose-500">
        <div class="font-bold text-slate-700">
    </a>
    <a href="#question-21" onclick="setQuestion(21)">
- l-4 border-indigo-500 hover:bg-slate-700 transition
        <div class="text-xs text-indigo-500">
        <div class="font-bold text-slate-700">
    </a>
    <a href="#question-22" onclick="setQuestion(22)">
- l-4 border-fuchsia-500 hover:bg-slate-700 transition
        <div class="text-xs text-fuchsia-500">
        <div class="font-bold text-slate-700">
    </a>

    <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ -->
    <div class="col-span-1 md:col-span-3 lg:col-span-3"
border-b border-red-500/30 pb-2">Кластер 4:

    <a href="#proof-algorithms" onclick="setQuestion(23)">
:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4 border-t-4 border-b-4
        <div class="text-xs text-red-500">
        <div class="font-bold text-slate-700">

```

```

<!-- От противного -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-fuchsia-400">От противного
  <p class="text-sm text-slate-400">Предположим, что
  <ul class="list-disc pl-5 text-slate-400">
    <li><span class="text-white">...
i>
    <li><span class="text-white">...
    т.д.).</li>
    <li><span class="text-white">...
что так и задумано: "Ой, противоречие, теорема неверна".
  </ul>
</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-emerald-400">Единственность
  <p class="text-sm text-slate-400">Пусть
  <ul class="list-disc pl-5 text-slate-400">
    <li><span class="text-white">...
,  $r_2$ ).</li>
    <li><span class="text-white">...
    <li><span class="text-white">...
счет ограничений (размер остатка) докажи, что
  </ul>
</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-yellow-400">Индукция
  <p class="text-sm text-slate-400">Базис
  <ul class="list-disc pl-5 text-slate-400">
    <li><span class="text-white">...
валось.</li>
    <li><span class="text-white">...
Просто перепиши формулу с буквой  $k$ .</li>
    <li><span class="text-white">...

```

```
        <div class="bg-slate-800 p-4">
          <h4 class="text-red-400">
            <p class="text-sm text-slate-400">
              <p class="text-xs font-slate-400">
                ициентов (они делятся на $p$), кроме старшего
                зложить.</p>
              </p>
            </h4>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```

ициентов (они делятся на \$p\$), кроме старшего  
зложить.</p>

```
        <div class="bg-slate-800 p-4">
          <h4 class="text-red-400">
            <p class="text-sm text-slate-400">
              <p class="text-xs font-slate-400">
                . Чаще всего после этого Фейс-контроль $p$ .
              </p>
            </h4>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```

. Чаще всего после этого Фейс-контроль \$p\$ .

```
    <h2 class="text-2xl font-bold bg-slate-800 p-4">
      (Правила экстрасенса)</h2>
    <p class="text-sm text-slate-400">
      тношения), а ты их не помнишь. Юзай базовые
      нейность) для делимости.</p>
  </div>
</div>
```

(Правила экстрасенса)</h2>

<p class="text-sm text-slate-400">  
тношения), а ты их не помнишь. Юзай базовые  
нейность) для делимости.</p>

```
    <ul class="space-y-4 pb-10">
      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
          <div>
            <h4 class="text-cyan-400">
              <p class="text-sm text-slate-400">
                <code class="text-xs font-slate-400">
                  </div>
            </h4>
          </div>
        </li>
      </ul>
    </div>
  </div>
</div>
```

<ul class="space-y-4 pb-10">  
<li class="bg-slate-800/50 p-4">  
<span class="text-2xl">  
<div>  
<h4 class="text-cyan-400">  
<p class="text-sm text-slate-400">  
<code class="text-xs font-slate-400">  
</div>  
</h4>  
</div>  
</li>  
</ul>

```
    <li class="bg-slate-800/50 p-4">
      <span class="text-2xl">
        <div>
          <h4 class="text-cyan-400">
            <p class="text-sm text-slate-400">
              </div>
          </h4>
        </div>
      </li>
    </ul>
  </div>
</div>
```

<li class="bg-slate-800/50 p-4">  
<span class="text-2xl">  
<div>  
<h4 class="text-cyan-400">  
<p class="text-sm text-slate-400">  
</div>  
</h4>  
</div>  
</li>  
</ul>

ФАЙЛ 74/82: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 75/82: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, интернеты
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — их
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
  const parts: string[] = [];
  for (const sheet of Array.from(document.styleSheets)) {
    try {
```

```

    details > div, details > p { display: block !important;
    pre { white-space: pre-wrap; }
  }
`;

```

```

const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(

```

```

/**

```

```

 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */

```

```

function prepareBody(html: string): { body: string; terms: string[] } {
  const host = document.createElement("div");
  host.innerHTML = html;

```

```

  try {
    annotateTerms(host);
  } catch {
    /* если браузер не осилил регулярку – выгружаем текст как есть */
  }

```

```

  const ids: string[] = [];
  host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
    const id = el.dataset.termId;
    if (!id) return;
    if (!ids.includes(id)) ids.push(id);
    const link = document.createElement("a");
    link.className = "term-hint";
    link.href = `#term-${id}`;
    link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
    link.textContent = el.textContent ?? "";
    el.replaceWith(link);
  });

```

```

// <details> в файле лучше раскрыть: иначе код и разброс
host.querySelectorAll("details").forEach((d) => d.setOpen(true));

```

```

<div class="export-shell">
  <header class="export-head">
    <div style="font-size:.7rem;text-transform:uppercase">
      <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">
        <div style="font-size:.72rem;color:#475569;margin-top:10px">
      </div>
    </div>

    <div class="export-note">
      Это автономная копия: стили внутри файла, интернет не нужен.
      Интерактивные тренажёры и анимации в статичный HTML превращены,
      они живут в онлайн-версии: <a href="${escapeHtml(opts.url)}">онлайн</a>
    </div>

    <article class="prose prose-invert max-w-none">
      ${opts.body}
    </article>

    ${opts.gloss}

    <footer class="export-foot">Универсальное пособие по созданию HTML-файлов
  </div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

/\*\* Имя файла: «04-splay-derevo.html» — без кириллицы и пробелов



```
    });

    const body = `
```

---

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

---

---

ФАЙЛ 76/82: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

---

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 *   node scripts/audit-coverage.mjs           # отчёт
 *   node scripts/audit-coverage.mjs --md      # markd
 */
import fs from "node:fs";
```

```

        chars: html.length,
    });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.svg;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
    console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
    console.log("| --- | --- | --- | --- | --- |");
    for (const r of rows) {
        console.log(
            `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
        );
    }
} else {
    console.log(`Глав в пособии: ${rows.length}\n`);
    const dump = (title, list) => {
        console.log(`${title} (${list.length}):`);
        list.forEach((r) => console.log(`  * ${r.title} [${r.num}]`));
        console.log();
    };
    dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
    dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
    dump("НЕТ 2D (аналогия есть)", gapViz);
    console.log(`Пропущенные номера билетов 1-24: ${missingNums.join(", ")}`);
    console.log(`Визуализаторы без главы: ${orphanViz.join(", ")}`);
}

```

```

const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
    .replace(/<[^\>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlights).padStart(3),
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
  );
}
console.log("\nГлав без единой подсказки:", bad.length,);
console.log("Терминов в глоссарии:", GLOSSARY.length,);
console.log("Не встретились:", GLOSSARY.filter((g) => !used.has(g)).length,);

```

---

```
    Author: "CI: rebuild-pdf",
    Subject: "Экспорт всего кода проекта (код -> txt
    CreationDate: new Date(),
  },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
  doc.addPage({ size: [A4.width, A4.height], margin: 10,
  doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width,
  }

doc.end();
await new Promise((resolve, reject) => {
  stream.on("finish", resolve);
  stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

---

ФАЙЛ 79/82: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.2 КБ

---

/\*\*

\* ШАГ 1: КОД -> TXT

```
/** Порядок разделов в документе. */
```

```
const ORDER = [  
  "Конфигурация проекта",  
  "Ядро приложения",  
  "Контент и данные пособия",  
  "Билеты (учебный контент)",  
  "Интерактивные компоненты и симуляторы",  
  "Визуализаторы (вложенные)",  
  "HTML-разметка (layout)",  
  "Утилиты",  
  "Скрипты сборки и экспорта",  
  "CI / автоматизация",  
  "Прочее",  
];
```

```
function listFiles() {  
  let files;  
  try {  
    files = execFileSync("git", ["ls-files", "--cached"],  
      cwd: ROOT,  
      encoding: "utf8",  
    )  
      .split("\n")  
      .filter(Boolean);  
  } catch {  
    files = [];  
    const walk = (dir) => {  
      for (const e of fs.readdirSync(dir, { withFileType: true })) {  
        const abs = path.join(dir, e.name);  
        const rel = path.relative(ROOT, abs).split(path.sep);  
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;  
        if (e.isDirectory()) walk(abs);  
        else files.push(rel);  
      }  
    };  
    walk(ROOT);  
  }  
}
```

```

const out = [];
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { ti
const totalBytes = files.reduce((s, f) => s + fs.stat

push(HR);
push("          УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
push("          ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalByt
push(`Глав/билетов в пособии:        ${chapters.length}`)
push();

push(HR);
push("          ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")}: ${c.titl
});
push();

push(HR);
push("          ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`)

```

```
}  
  
main();  
  
-----
```

ФАЙЛ 80/82: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```
-----  
/**  
 * Общие настройки и утилиты пайплайна экспорта.  
 *  
 * код -> full_code_all_pages.txt      (collect-code.mjs)  
 * txt  -> page-XXXX.png              (render-pages.mjs)  
 * png  -> full_code_all_pages.pdf     (build-pdf.mjs)  
 */  
import fs from "node:fs";  
import path from "node:path";  
import { fileURLToPath } from "node:url";  
import { execFileSync } from "node:child_process";  
  
export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),  
  "../..");  
  
/** Куда кладём финальные артефакты (эта папка отдаётся клиенту)  
export const OUT_DIR = path.join(ROOT, "public", "export");  
  
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают)  
export const PAGES_DIR = path.join(ROOT, "tmp", "export");  
  
export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");  
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");  
  
/** Параметры вёрстки страницы (подобраны под A4 @150dpi)  
export const PAGE = {  
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100  
  density: Number(process.env.EXPORT_DENSITY ?? 150),  
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1  
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",  
};
```

---

ФАЙЛ 81/82: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ: 1.8 КБ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize,
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
 * function wrap(line) {
 *   const expanded = line.replace(/\t/g, "    ").replace(
 *   if (expanded.length <= PAGE.cols) return [expanded];
 *   const parts = [];
 *   const indent = "    ".repeat(Math.min((expanded.match(/^
 *   let rest = expanded;
 *   let first = true;
 *   while (rest.length > 0) {
 *     const width = first ? PAGE.cols : PAGE.cols - indent;
 *     parts.push((first ? "" : indent) + rest.slice(0, width));
 *     rest = rest.slice(width);
 *     first = false;
 *   }
 *   return parts;
 * }
```



```

    "-density", String(PAGE.density),
    "-page", PAGE.size,
    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
    "-pointsize", String(PAGE.pointsized),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done}\n`);
    }
  }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(", ")}`);

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");

```

```
- "public/export/**"
- "**/*.md"
workflow_dispatch:
  inputs:
    density:
      description: "DPI растеризации страниц (по умолчанию)"
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${github.ref}`
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
```

```
    echo "| PNG-страниц | ${ls tmp/export-pages.  
} >> "$GITHUB_STEP_SUMMARY"
```

- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v4  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v4  
with:  
 name: full-code-pages-png  
 path: tmp/export-pages/\*.png  
 if-no-files-found: error  
 retention-days: 7
- name: Commit rebuilt export back to the branch  
run: |  
 git config user.name "github-actions[bot]"  
 git config user.email "41898282+github-action@users.noreply.github.com"  
 git add -- public/export/full\_code\_all\_pages.  
 if git diff --cached --quiet; then  
 echo "Экспорт не изменился — коммит не нужен"  
 exit 0  
 fi  
 git commit -m "chore(export): пересборка full-code-export"  
 git push origin "HEAD:\${GITHUB\_REF\_NAME}"